

# Human review packet: DGD26AdmissionsPredictability

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

**Paper:** DGD26AdmissionsPredictability

**Paper id:** DGD26AdmissionsPredictability

**Source version:** AAAI 2026 paper; pinned arXiv TeX source archive

**Source record:** audit/paper\_statement\_map.json

**Rows:** 42

**Generated:** 2026-09-06

**Source URL:** [official source](#)

## How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

## Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) AppliedModelingLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-repository-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper DGD26AdmissionsPredictability --serve
```

Open <http://127.0.0.1:8765/> in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's local reviewer-owned review record.

## Contents

- [Governing Lean declarations \(81; supporting context\)](#)
- [Semantic library prerequisites \(22; not paper claims\)](#)
  - [AppliedModelingLib.FiniteChoice.ChoiceRule](#)
  - [AppliedModelingLib.FiniteChoice.Consistent](#)
  - [AppliedModelingLib.FiniteChoice.choiceGainTerm](#)
  - [AppliedModelingLib.FiniteChoice.choiceLossTerm](#)
  - [AppliedModelingLib.FiniteChoice.choiceDistance](#)
  - [AppliedModelingLib.FiniteChoice.DUnstable](#)
  - [AppliedModelingLib.FiniteChoice.Feasible](#)
  - [AppliedModelingLib.FiniteChoice.borderlineSet](#)
  - [AppliedModelingLib.FiniteChoice.waitlistedSet](#)
  - [AppliedModelingLib.FiniteChoice.GeneralVariabilityExactly](#)
  - [AppliedModelingLib.FiniteChoice.Independent](#)
  - [AppliedModelingLib.FiniteChoice.Monotonic](#)
  - [AppliedModelingLib.FiniteChoice.QAcceptant](#)
  - [AppliedModelingLib.FiniteChoice.StrictTotalOrder](#)
  - [AppliedModelingLib.FiniteChoice.QRepresentative](#)
  - [AppliedModelingLib.FiniteChoice.Substitutable](#)
  - [AppliedModelingLib.FiniteChoice.TightlyDUnstable](#)
  - [AppliedModelingLib.FiniteChoice.VariabilityAtMost](#)
  - [AppliedModelingLib.FiniteChoice.VariabilityExactly](#)
  - [AppliedModelingLib.FiniteChoice.ZeroUnstable](#)
  - [AppliedModelingLib.FiniteChoice.linearTopQChoice](#)
  - [AppliedModelingLib.FiniteChoice.sequentialComposition](#)
- [Paper-specific semantic prerequisites \(6; not paper claims\)](#)
  - [DGD26AdmissionsPredictability.LAP.Assignment.SameSlotOrder](#)
  - [DGD26AdmissionsPredictability.LAP.Assignment.SlotBelow](#)

- DGD26AdmissionsPredictability.LAP.Assignment.choiceRuleOfAssignment
- DGD26AdmissionsPredictability.PaperInterface.paper\_binary\_classifier\_model\_statementSpec
- DGD26AdmissionsPredictability.lapModel
- DGD26AdmissionsPredictability.paperRepresentsChoice
- Source claims (42)
  - paper\_fixed\_threshold\_formula\_statementSpec
  - paper\_definition\_choice\_labelSpec
  - paper\_rank\_threshold\_formula\_statementSpec
  - paper\_independent\_zero\_unstable\_statementSpec
  - paper\_independent\_zero\_unstable\_corollary\_statementSpec
  - paper\_no\_consistent\_positive\_tightly\_even\_statementSpec
  - paper\_zero\_distance\_statementSpec
  - paper\_calculating\_instability\_statementSpec
  - paper\_corrected\_consistency\_of\_removable\_sets\_statementSpec
  - paper\_lap\_strictly\_higher\_slot\_applicant\_assigned\_statementSpec
  - paper\_lap\_assigned\_strictly\_outranks\_rejected\_statementSpec
  - paper\_monotonicity\_term\_statementSpec
  - paper\_monotonicity\_q\_acceptance\_incompatible\_statementSpec
  - paper\_acceptant\_one\_instability\_variability\_borderline\_eq\_waitlisted\_after\_changing\_insert\_statementSpec
  - paper\_substitutability\_term\_statementSpec
  - paper\_non\_substitutable\_single\_add\_statementSpec
  - paper\_ml\_fixed\_threshold\_representation\_zero\_unstable\_statementSpec
  - paper\_ml\_rank\_threshold\_can\_represent\_exact\_one\_statementSpec
  - paper\_ml\_rank\_threshold\_representation\_instability\_bound\_statementSpec
  - paper\_ml\_rank\_threshold\_representation\_variability\_bound\_statementSpec
  - paper\_program\_classes\_one\_instability\_statementSpec
  - paper\_screened\_open\_variability\_one\_statementSpec
  - paper\_screened\_open\_dia\_variability\_two\_statementSpec
  - paper\_educational\_option\_variability\_three\_statementSpec
  - paper\_educational\_option\_dia\_variability\_six\_statementSpec
  - paper\_sequential\_additive\_variability\_bound\_statementSpec
  - paper\_append\_remove\_variability\_exact\_equivalence\_statementSpec
  - paper\_choice\_distance\_triangle\_statementSpec
  - paper\_even\_distance\_iff\_inconsistent\_statementSpec
  - paper\_independent\_substitutable\_monotonic\_statementSpec
  - paper\_no\_zero\_instability\_under\_capacity\_statementSpec
  - paper\_substitutability\_one\_instability\_equivalence\_statementSpec
  - paper\_theorem1\_tight\_all\_d\_statementSpec
  - paper\_lap\_assignment\_one\_instability\_statementSpec
  - paper\_lap\_assignment\_slot\_order\_class\_variability\_of\_unique\_global\_optima\_statementSpec
  - paper\_q\_representative\_forward\_exact\_statementSpec
  - paper\_q\_representative\_converse\_exact\_statementSpec
  - paper\_sequential\_composition\_substitutable\_statementSpec
  - paper\_qacceptant\_substitutable\_iff\_one\_statementSpec
  - paper\_q\_acceptant\_substitutable\_consistent\_statementSpec
  - paper\_sequential\_q\_representative\_variability\_range\_statementSpec
  - paper\_variability\_one\_iff\_single\_order\_statementSpec

# Governing Lean declarations

These exact graph-bound declaration bodies are retained by the displayed specifications or reviewed prerequisites. They are supporting context and do not add source-result rows or semantic judgments.

## AppliedModelingLib.FiniteChoice.GeneralVariabilityAtMost

**Declaration location:** reusable library

### Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → [Fintype α] → ℕ → AppliedModelingLib.FiniteChoice.ChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] [Fintype α] (m : ℕ) (C : AppliedModelingLib.FiniteChoice.ChoiceRule α) =>
  ∀ (X : Finset α),
    (AppliedModelingLib.FiniteChoice.borderlineSet C X).card ≤ m ∧
    (AppliedModelingLib.FiniteChoice.waitlistedSet C X).card ≤ m
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)
- [AppliedModelingLib.FiniteChoice.borderlineSet](#)
- [AppliedModelingLib.FiniteChoice.waitlistedSet](#)

## DGD26AdmissionsPredictability.LAP.Assignment

**Declaration location:** paper-local

### Lean declaration body

```
field matchSlot:
{α : Type u_3} → {σ : Type u_4} → DGD26AdmissionsPredictability.LAP.Assignment α σ → σ → Option α
```

## DGD26AdmissionsPredictability.LAP.Assignment.Assigned

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → {σ : Type u_2} → DGD26AdmissionsPredictability.LAP.Assignment α σ → α → Prop

value:
fun {α : Type u_1} {σ : Type u_2} (A : DGD26AdmissionsPredictability.LAP.Assignment α σ) (a : α) =>
  ∃ (s : σ), A.matchSlot s = some a
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)

## DGD26AdmissionsPredictability.LAP.Assignment.AssignedFrom

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → {σ : Type u_2} → Finset α → DGD26AdmissionsPredictability.LAP.Assignment α σ → Prop

value:
fun {α : Type u_1} {σ : Type u_2} (X : Finset α) (A : DGD26AdmissionsPredictability.LAP.Assignment α σ) =>
  ∀ {s : σ} {a : α}, A.matchSlot s = some a → a ∈ X
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)

## DGD26AdmissionsPredictability.LAP.Assignment.NoDuplicateApplicants

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → {σ : Type u_2} → DGD26AdmissionsPredictability.LAP.Assignment α σ → Prop

value:
fun {α : Type u_1} {σ : Type u_2} (A : DGD26AdmissionsPredictability.LAP.Assignment α σ) =>
  ∀ {s t : σ} {a : α}, A.matchSlot s = some a → A.matchSlot t = some a → s = t
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)

## DGD26AdmissionsPredictability.LAP.Assignment.Feasible

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → {σ : Type u_2} → Finset α → DGD26AdmissionsPredictability.LAP.Assignment α σ → Prop

value:
fun {α : Type u_1} {σ : Type u_2} (X : Finset α) (A : DGD26AdmissionsPredictability.LAP.Assignment α σ) =>
  DGD26AdmissionsPredictability.LAP.Assignment.AssignedFrom X A ∧ A.NoDuplicateApplicants
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.AssignedFrom](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.NoDuplicateApplicants](#)

## DGD26AdmissionsPredictability.LAP.Assignment.Rejected

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → {σ : Type u_2} → Finset α → DGD26AdmissionsPredictability.LAP.Assignment α σ → α → Prop

value:
fun {α : Type u_1} {σ : Type u_2} (X : Finset α) (A : DGD26AdmissionsPredictability.LAP.Assignment α σ) (a : α) =>
  a ∈ X ∧ ¬A.Assigned a
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.Assigned](#)

## DGD26AdmissionsPredictability.LAP.Assignment.SlotNoTies

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → {σ : Type u_2} → (α → σ → ℝ) → σ → Prop

value:
fun {α : Type u_1} {σ : Type u_2} (w : α → σ → ℝ) (s : σ) => ∀ {a b : α}, a ≠ b → w a s ≠ w b s
```

## DGD26AdmissionsPredictability.LAP.Assignment.chosenSet

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} →
{σ : Type u_2} → [DecidableEq α] → [Fintype σ] → DGD26AdmissionsPredictability.LAP.Assignment α σ → Finset α

value:
fun {α : Type u_1} {σ : Type u_2} [DecidableEq α] [Fintype σ] (A : DGD26AdmissionsPredictability.LAP.Assignment α σ) =>
  Finset.univ.biUnion fun (s : σ) =>
    match A.matchSlot s with
    | none => ∅
    | some a => {a}
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)

## DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} →
{σ : Type u_2} → [DecidableEq α] → [Fintype σ] → Finset α → DGD26AdmissionsPredictability.LAP.Assignment α σ → Prop

value:
fun {α : Type u_1} {σ : Type u_2} [DecidableEq α] [Fintype σ] (X : Finset α)
  (A : DGD26AdmissionsPredictability.LAP.Assignment α σ) =>
  (X.card ≤ Fintype.card σ → X ⊆ A.chosenSet) ∧ (Fintype.card σ ≤ X.card → ∀ (s : σ), ∃ (a : α), A.matchSlot s = some a)
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.chosenSet](#)

## DGD26AdmissionsPredictability.LAP.Assignment.sameSlotOrderSetoid

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → {σ : Type u_2} → [DecidableEq σ] → [Fintype σ] → (α → σ → ℝ) → Setoid σ

value:
fun {α : Type u_1} {σ : Type u_2} [DecidableEq σ] [Fintype σ] (w : α → σ → ℝ) =>
  { r := DGD26AdmissionsPredictability.LAP.Assignment.SameSlotOrder w,
    iseqv := DGD26AdmissionsPredictability.LAP.Assignment.sameSlotOrderSetoid._proof_1 w }
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment.SameSlotOrder](#)

## DGD26AdmissionsPredictability.LAP.Assignment.slotOrderClass

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} →
{σ : Type u_2} →
[inst : DecidableEq σ] →
[inst_1 : Fintype σ] →
(w : α → σ → ℝ) → σ → Quotient (DGD26AdmissionsPredictability.LAP.Assignment.sameSlotOrderSetoid w)

value:
fun {α : Type u_1} {σ : Type u_2} [DecidableEq σ] [Fintype σ] (w : α → σ → ℝ) (s : σ) => [s]
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment.sameSlotOrderSetoid](#)

## DGD26AdmissionsPredictability.LAP.Assignment.slotValue

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → {σ : Type u_2} → (α → σ → ℝ) → DGD26AdmissionsPredictability.LAP.Assignment α σ → σ → ℝ

value:
fun {α : Type u_1} {σ : Type u_2} (w : α → σ → ℝ) (A : DGD26AdmissionsPredictability.LAP.Assignment α σ) (s : σ) =>
  match A.matchSlot s with
  | none => 0
  | some a => w a s
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)

## DGD26AdmissionsPredictability.LAP.Assignment.objective

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → {σ : Type u_2} → [Fintype σ] → (α → σ → ℝ) → DGD26AdmissionsPredictability.LAP.Assignment α σ → ℝ

value:
fun {α : Type u_1} {σ : Type u_2} [Fintype σ] (w : α → σ → ℝ) (A : DGD26AdmissionsPredictability.LAP.Assignment α σ) =>
  ∑ s : σ, DGD26AdmissionsPredictability.LAP.Assignment.slotValue w A s
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.slotValue](#)

## DGD26AdmissionsPredictability.LAP.Assignment.ObjectiveOptimal

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} →
{σ : Type u_2} →
[DecidableEq α] → [Fintype σ] → Finset α → (α → σ → ℝ) → DGD26AdmissionsPredictability.LAP.Assignment α σ → Prop

value:
fun {α : Type u_1} {σ : Type u_2} [DecidableEq α] [Fintype σ] (X : Finset α) (w : α → σ → ℝ)
  (A : DGD26AdmissionsPredictability.LAP.Assignment α σ) =>
  ∀ (B : DGD26AdmissionsPredictability.LAP.Assignment α σ),
  DGD26AdmissionsPredictability.LAP.Assignment.Feasible X B →
  DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling X B →
  DGD26AdmissionsPredictability.LAP.Assignment.objective w B ≤
  DGD26AdmissionsPredictability.LAP.Assignment.objective w A
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.Feasible](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.objective](#)

## DGD26AdmissionsPredictability.LAP.Assignment.UniqueChosenSetObjectiveOptimal

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} →
{σ : Type u_2} →
[DecidableEq α] → [Fintype σ] → Finset α → (α → σ → ℝ) → DGD26AdmissionsPredictability.LAP.Assignment α σ → Prop

value:
fun {α : Type u_1} {σ : Type u_2} [DecidableEq α] [Fintype σ] (X : Finset α) (w : α → σ → ℝ) =>
(A : DGD26AdmissionsPredictability.LAP.Assignment α σ) =>
DGD26AdmissionsPredictability.LAP.Assignment.ObjectiveOptimal X w A ∧
∀ (B : DGD26AdmissionsPredictability.LAP.Assignment α σ),
DGD26AdmissionsPredictability.LAP.Assignment.Feasible X B →
DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling X B →
DGD26AdmissionsPredictability.LAP.Assignment.ObjectiveOptimal X w B → B.chosenSet = A.chosenSet
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.Feasible](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.ObjectiveOptimal](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.chosenSet](#)

## DGD26AdmissionsPredictability.LAP.Assignment.WellPosedObjective

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → {σ : Type u_2} → [DecidableEq α] → [Fintype σ] → (α → σ → ℝ) → Prop

value:
fun {α : Type u_1} {σ : Type u_2} [DecidableEq α] [Fintype σ] (w : α → σ → ℝ) =>
∀ (X : Finset α),
∃ (A : DGD26AdmissionsPredictability.LAP.Assignment α σ),
DGD26AdmissionsPredictability.LAP.Assignment.Feasible X A ∧
DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling X A ∧
DGD26AdmissionsPredictability.LAP.Assignment.UniqueChosenSetObjectiveOptimal X w A
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.Feasible](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.UniqueChosenSetObjectiveOptimal](#)

## DGD26AdmissionsPredictability.LAP.Assignment.canonicalOptimalAssignment

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} →
{σ : Type u_2} →
[inst : DecidableEq α] →
[inst_1 : Fintype σ] →
(w : α → σ → ℝ) →
DGD26AdmissionsPredictability.LAP.Assignment.WellPosedObjective w →
Finset α → DGD26AdmissionsPredictability.LAP.Assignment α σ

value:
fun {α : Type u_1} {σ : Type u_2} [DecidableEq α] [Fintype σ] (w : α → σ → ℝ)
(hwell : DGD26AdmissionsPredictability.LAP.Assignment.WellPosedObjective w) (X : Finset α) =>
Classical.choose (hwell X)
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.Feasible](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.UniqueChosenSetObjectiveOptimal](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.WellPosedObjective](#)

## DGD26AdmissionsPredictability.PaperChoiceRule

**Declaration location:** paper-local

### Lean declaration body

```
type:
( $\alpha$  : Type u_2) → [DecidableEq  $\alpha$ ] → Type u_2

value:
fun ( $\alpha$  : Type u_2) [DecidableEq  $\alpha$ ] => AppliedModelingLib.FiniteChoice.ChoiceRule  $\alpha$ 
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)

## DGD26AdmissionsPredictability.PaperPoolPredictor

**Declaration location:** paper-local

### Lean declaration body

```
type:
Type u_2 → Type u_2

value:
fun ( $\alpha$  : Type u_2) => Finset  $\alpha$  →  $\alpha$  → Prop
```

## DGD26AdmissionsPredictability.ProgramClasses.queueRank

**Declaration location:** paper-local

### Lean declaration body

```
type:
( $n$  :  $\mathbb{N}$ ) → Fin n → Fin (2 * n) →  $\mathbb{N}$ 

value:
fun ( $n$  :  $\mathbb{N}$ ) (j : Fin n) (a : Fin (2 * n)) => if  $\uparrow a = n + \uparrow j$  then 0 else if  $\uparrow a = \uparrow j$  then 1 else 2 +  $\uparrow a$ 
```

## DGD26AdmissionsPredictability.ProgramClasses.queueChoice

**Declaration location:** paper-local

### Lean declaration body

```
type:
( $n$  :  $\mathbb{N}$ ) → Fin n → Finset (Fin (2 * n)) → Finset (Fin (2 * n))

value:
fun ( $n$  :  $\mathbb{N}$ ) (j : Fin n) (a : Finset (Fin (2 * n))) =>
  {a_1 ∈ a |
    ∀ b ∈ a,
      DGD26AdmissionsPredictability.ProgramClasses.queueRank n j a_1 ≤
      DGD26AdmissionsPredictability.ProgramClasses.queueRank n j b}
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.ProgramClasses.queueRank](#)

## DGD26AdmissionsPredictability.ProgramClasses.queues

**Declaration location:** paper-local

### Lean declaration body

```
type:
( $n$  :  $\mathbb{N}$ ) → List (AppliedModelingLib.FiniteChoice.ChoiceRule (Fin (2 * n)))

value:
fun ( $n$  :  $\mathbb{N}$ ) => List.ofFn fun (j : Fin n) => DGD26AdmissionsPredictability.ProgramClasses.queueChoice n j
```



## Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)
- [DGD26AdmissionsPredictability.ProgramClasses.queueChoice](#)

## DGD26AdmissionsPredictability.ProgramClasses.choice

**Declaration location:** paper-local

## Lean declaration body

```
type:
(n : ℕ) → Finset (Fin (2 * n)) → Finset (Fin (2 * n))

value:
fun (n : ℕ) (a : Finset (Fin (2 * n))) =>
  AppliedModelingLib.FiniteChoice.sequentialComposition (DGD26AdmissionsPredictability.ProgramClasses.queues n) a
```

## Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.sequentialComposition](#)
- [DGD26AdmissionsPredictability.ProgramClasses.queues](#)

## DGD26AdmissionsPredictability.paperChoiceLabel

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → DGD26AdmissionsPredictability.PaperChoiceRule α → Finset α → α → ℕ

value:
fun {α : Type u_1} [DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) (X : Finset α) (x : α) =>
  if x ∈ C X then 1 else 0
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)

## DGD26AdmissionsPredictability.paperEducationalOptionDIAProgramChoice

**Declaration location:** paper-local

## Lean declaration body

```
type:
Finset (Fin 12) → Finset (Fin 12)

value:
fun (a : Finset (Fin 12)) => DGD26AdmissionsPredictability.ProgramClasses.choice 6 a
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.ProgramClasses.choice](#)

## DGD26AdmissionsPredictability.paperEducationalOptionProgramChoice

**Declaration location:** paper-local

## Lean declaration body

```
type:
Finset (Fin 6) → Finset (Fin 6)

value:
fun (a : Finset (Fin 6)) => DGD26AdmissionsPredictability.ProgramClasses.choice 3 a
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.ProgramClasses.choice](#)

## DGD26AdmissionsPredictability.paperFixedThresholdChoice

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → [DecidableEq α] → (α → ℝ) → ℝ → Finset α → Finset α

value:
fun {α : Type u_1} [DecidableEq α] (score : α → ℝ) (threshold : ℝ) (a : Finset α) => {x ∈ a | threshold ≤ score x}
```

## DGD26AdmissionsPredictability.paperFixedThresholdPredictor

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → (α → ℝ) → ℝ → Finset α → α → Prop

value:
fun {α : Type u_1} (score : α → ℝ) (threshold : ℝ) (a : Finset α) (a_1 : α) => threshold ≤ score a_1
```

## DGD26AdmissionsPredictability.paperLAPChoiceRule

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} →
[inst : DecidableEq α] →
{σ : Type u_2} →
[DecidableEq σ] →
[inst_2 : Fintype σ] →
(w : α → σ → ℝ) → DGD26AdmissionsPredictability.LAP.Assignment.WellPosedObjective w → Finset α → Finset α

value:
fun {α : Type u_1} [DecidableEq α] {σ : Type u_2} [DecidableEq σ] [Fintype σ] (w : α → σ → ℝ)
(hwell : DGD26AdmissionsPredictability.LAP.Assignment.WellPosedObjective w) (a : Finset α) =>
DGD26AdmissionsPredictability.LAP.Assignment.choiceRuleOfAssignment
(DGD26AdmissionsPredictability.LAP.Assignment.canonicalOptimalAssignment w hwell) a
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment.WellPosedObjective](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.canonicalOptimalAssignment](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.choiceRuleOfAssignment](#)

## DGD26AdmissionsPredictability.paperOrderByScore

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → (score : α → ℝ) → Function.Injective score → LinearOrder α

value:
fun {α : Type u_1} (score : α → ℝ) (hinjective : Function.Injective score) =>
LinearOrder.lift' (fun (a : α) => OrderDual.toDual (score a))
fun {{a b : α}} (h : (fun (a : α) => OrderDual.toDual (score a)) a = (fun (a : α) => OrderDual.toDual (score a)) b) =>
hinjective h
```

## DGD26AdmissionsPredictability.paperRankThresholdChoice

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → [DecidableEq α] → ℕ → (score : α → ℝ) → Function.Injective score → Finset α → Finset α

value:
fun {α : Type u_1} [DecidableEq α] (q : ℕ) (score : α → ℝ) (hinjective : Function.Injective score) (a : Finset α) =>
AppliedModelingLib.FiniteChoice.linearTopQChoice q a
```

## Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.linearTopQChoice](#)
- [DGD26AdmissionsPredictability.paperOrderByScore](#)

## DGD26AdmissionsPredictability.paperRankThresholdPredictor

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [DecidableEq α] → ℕ → (score : α → ℝ) → Function.Injective score → Finset α → α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (q : ℕ) (score : α → ℝ) (hinjective : Function.Injective score) (a : Finset α)
  (a_1 : α) =>
  a_1 ∈ DGD26AdmissionsPredictability.paperRankThresholdChoice q score hinjective a
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.paperRankThresholdChoice](#)

## DGD26AdmissionsPredictability.paperScreenedOpenDIAProgramChoice

**Declaration location:** paper-local

## Lean declaration body

```
type:
Finset (Fin 4) → Finset (Fin 4)

value:
fun (a : Finset (Fin 4)) => DGD26AdmissionsPredictability.ProgramClasses.choice 2 a
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.ProgramClasses.choice](#)

## DGD26AdmissionsPredictability.paperScreenedOpenProgramChoice

**Declaration location:** paper-local

## Lean declaration body

```
type:
Finset (Fin 2) → Finset (Fin 2)

value:
fun (a : Finset (Fin 2)) => DGD26AdmissionsPredictability.ProgramClasses.choice 1 a
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.ProgramClasses.choice](#)

## DGD26AdmissionsPredictability.paper\_borderline\_set

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} →
[inst : DecidableEq α] → [Fintype α] → DGD26AdmissionsPredictability.PaperChoiceRule α → Finset α → Finset α

value:
fun {α : Type u_1} [DecidableEq α] [Fintype α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) (X : Finset α) =>
  AppliedModelingLib.FiniteChoice.borderlineSet C X
```

## Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.borderlineSet](#)
- [DGD26AdmissionsPredictability.PaperChoiceRule](#)

## DGD26AdmissionsPredictability.paper\_choiceDistance

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → DGD26AdmissionsPredictability.PaperChoiceRule α → Finset α → Finset α → ℕ

value:
fun {α : Type u_1} [DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) (X1 X2 : Finset α) =>
  ((X1 ∩ C X2) \ C X1).card + (C X1 \ C X2).card
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)

## DGD26AdmissionsPredictability.paper\_consistent

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  AppliedModelingLib.FiniteChoice.Consistent C
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.Consistent](#)
- [DGD26AdmissionsPredictability.PaperChoiceRule](#)

## DGD26AdmissionsPredictability.paper\_d\_unstable

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → ℕ → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (d : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  AppliedModelingLib.FiniteChoice.DUnstable d C
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.DUnstable](#)
- [DGD26AdmissionsPredictability.PaperChoiceRule](#)

## DGD26AdmissionsPredictability.paper\_definition\_borderline\_set

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} →
[inst : DecidableEq α] → [Fintype α] → DGD26AdmissionsPredictability.PaperChoiceRule α → Finset α → Finset α

value:
fun {α : Type u_1} [DecidableEq α] [Fintype α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) (X : Finset α) =>
  DGD26AdmissionsPredictability.paper_borderline_set C X
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_borderline\\_set](#)

## DGD26AdmissionsPredictability.paper\_definition\_choice\_distance

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → DGD26AdmissionsPredictability.PaperChoiceRule α → Finset α → Finset α → ℕ

value:
fun {α : Type u_1} [DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) (X1 X2 : Finset α) =>
  DGD26AdmissionsPredictability.paper_choiceDistance C X1 X2
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choiceDistance](#)

## DGD26AdmissionsPredictability.paper\_definition\_consistency

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  DGD26AdmissionsPredictability.paper_consistent C
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_consistent](#)

## DGD26AdmissionsPredictability.paper\_definition\_d\_instability

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → ℕ → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (d : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  DGD26AdmissionsPredictability.paper_d_unstable d C
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_d\\_unstable](#)

## DGD26AdmissionsPredictability.paper\_definition\_educational\_option\_dia\_program\_choice

**Declaration location:** paper-local

### Lean declaration body

```
type:
Finset (Fin 12) → Finset (Fin 12)

value:
fun (a : Finset (Fin 12)) => DGD26AdmissionsPredictability.paperEducationalOptionDIAProgramChoice a
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.paperEducationalOptionDIAProgramChoice](#)

## DGD26AdmissionsPredictability.paper\_definition\_educational\_option\_program\_choice

**Declaration location:** paper-local

### Lean declaration body

```
type:
Finset (Fin 6) → Finset (Fin 6)

value:
fun (a : Finset (Fin 6)) => DGD26AdmissionsPredictability.paperEducationalOptionProgramChoice a
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.paperEducationalOptionProgramChoice](#)

## DGD26AdmissionsPredictability.paper\_definition\_fixed\_threshold\_predictor

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → (α → ℝ) → ℝ → Finset α → α → Prop

value:
fun {α : Type u_1} (score : α → ℝ) (threshold : ℝ) (a : Finset α) (a_1 : α) =>
  DGD26AdmissionsPredictability.paperFixedThresholdPredictor score threshold a a_1
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.paperFixedThresholdPredictor](#)

## DGD26AdmissionsPredictability.paper\_definition\_lap\_assigned

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} →
{σ : Type u_2} → [DecidableEq σ] → [Fintype σ] → DGD26AdmissionsPredictability.LAP.Assignment α σ → α → Prop

value:
fun {α : Type u_1} {σ : Type u_2} [DecidableEq σ] [Fintype σ] (A : DGD26AdmissionsPredictability.LAP.Assignment α σ)
(x : α) =>
  A.Assigned x
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.Assigned](#)

## DGD26AdmissionsPredictability.paper\_definition\_lap\_assignment\_feasible

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} →
{σ : Type u_2} → [DecidableEq σ] → [Fintype σ] → Finset α → DGD26AdmissionsPredictability.LAP.Assignment α σ → Prop

value:
fun {α : Type u_1} {σ : Type u_2} [DecidableEq σ] [Fintype σ] (X : Finset α)
(A : DGD26AdmissionsPredictability.LAP.Assignment α σ) =>
  DGD26AdmissionsPredictability.LAP.Assignment.Feasible X A
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.Feasible](#)

## DGD26AdmissionsPredictability.paper\_definition\_lap\_capacity\_filling

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} →
[DecidableEq α] →
{σ : Type u_2} → [DecidableEq σ] → [Fintype σ] → Finset α → DGD26AdmissionsPredictability.LAP.Assignment α σ → Prop

value:
fun {α : Type u_1} [DecidableEq α] {σ : Type u_2} [DecidableEq σ] [Fintype σ] (X : Finset α)
(A : DGD26AdmissionsPredictability.LAP.Assignment α σ) =>
DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling X A
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling](#)

## DGD26AdmissionsPredictability.paper\_definition\_lap\_objective\_optimal

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} →
[DecidableEq α] →
{σ : Type u_2} →
[DecidableEq σ] → [Fintype σ] → Finset α → (α → σ → ℝ) → DGD26AdmissionsPredictability.LAP.Assignment α σ → Prop

value:
fun {α : Type u_1} [DecidableEq α] {σ : Type u_2} [DecidableEq σ] [Fintype σ] (X : Finset α) (w : α → σ → ℝ)
(A : DGD26AdmissionsPredictability.LAP.Assignment α σ) =>
DGD26AdmissionsPredictability.LAP.Assignment.ObjectiveOptimal X w A
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.ObjectiveOptimal](#)

## DGD26AdmissionsPredictability.paper\_definition\_lap\_slot\_below

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} → {σ : Type u_2} → [DecidableEq σ] → [Fintype σ] → (α → σ → ℝ) → σ → α → α → Prop

value:
fun {α : Type u_1} {σ : Type u_2} [DecidableEq σ] [Fintype σ] (w : α → σ → ℝ) (s : σ) (a b : α) =>
DGD26AdmissionsPredictability.LAP.Assignment.SlotBelow w s a b
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment.SlotBelow](#)

## DGD26AdmissionsPredictability.paper\_definition\_ml\_representation

**Declaration location:** paper-local

### Lean declaration body

```
type:
{α : Type u_1} →
[inst : DecidableEq α] →
DGD26AdmissionsPredictability.PaperPoolPredictor α → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (predicts : DGD26AdmissionsPredictability.PaperPoolPredictor α)
(C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
DGD26AdmissionsPredictability.paperRepresentsChoice predicts C
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.PaperPoolPredictor](#)
- [DGD26AdmissionsPredictability.paperRepresentsChoice](#)

## DGD26AdmissionsPredictability.paper\_definition\_rank\_threshold\_predictor

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [DecidableEq α] → ℕ → (score : α → ℝ) → Function.Injective score → Finset α → α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (q : ℕ) (score : α → ℝ) (hinjective : Function.Injective score) (a : Finset α)
  (a_1 : α) =>
  DGD26AdmissionsPredictability.paperRankThresholdPredictor q score hinjective a a_1
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.paperRankThresholdPredictor](#)

## DGD26AdmissionsPredictability.paper\_definition\_screened\_open\_dia\_program\_choice

**Declaration location:** paper-local

## Lean declaration body

```
type:
Finset (Fin 4) → Finset (Fin 4)

value:
fun (a : Finset (Fin 4)) => DGD26AdmissionsPredictability.paperScreenedOpenDIAProgramChoice a
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.paperScreenedOpenDIAProgramChoice](#)

## DGD26AdmissionsPredictability.paper\_definition\_screened\_open\_program\_choice

**Declaration location:** paper-local

## Lean declaration body

```
type:
Finset (Fin 2) → Finset (Fin 2)

value:
fun (a : Finset (Fin 2)) => DGD26AdmissionsPredictability.paperScreenedOpenProgramChoice a
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.paperScreenedOpenProgramChoice](#)

## DGD26AdmissionsPredictability.paper\_feasible

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  AppliedModelingLib.FiniteChoice.Feasible C
```



## Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.Feasible](#)
- [DGD26AdmissionsPredictability.PaperChoiceRule](#)

## DGD26AdmissionsPredictability.paper\_choice\_function\_feasible

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  DGD26AdmissionsPredictability.paper_feasible C
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_feasible](#)

## DGD26AdmissionsPredictability.paper\_general\_variability\_exactly

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → [Fintype α] → ℕ → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] [Fintype α] (m : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  AppliedModelingLib.FiniteChoice.GeneralVariabilityExactly m C
```

## Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.GeneralVariabilityExactly](#)
- [DGD26AdmissionsPredictability.PaperChoiceRule](#)

## DGD26AdmissionsPredictability.paper\_definition\_general\_variability\_exactly

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → [Fintype α] → ℕ → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] [Fintype α] (m : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  DGD26AdmissionsPredictability.paper_general_variability_exactly m C
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_general\\_variability\\_exactly](#)

## DGD26AdmissionsPredictability.paper\_independent

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  AppliedModelingLib.FiniteChoice.Independent C
```

## Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.Independent](#)
- [DGD26AdmissionsPredictability.PaperChoiceRule](#)

## DGD26AdmissionsPredictability.paper\_definition\_independence

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  DGD26AdmissionsPredictability.paper_independent C
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_independent](#)

## DGD26AdmissionsPredictability.paper\_monotonic

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  AppliedModelingLib.FiniteChoice.Monotonic C
```

## Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.Monotonic](#)
- [DGD26AdmissionsPredictability.PaperChoiceRule](#)

## DGD26AdmissionsPredictability.paper\_definition\_monotonicity

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  DGD26AdmissionsPredictability.paper_monotonic C
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_monotonic](#)

## DGD26AdmissionsPredictability.paper\_q\_acceptant

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → ℕ → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (q : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  AppliedModelingLib.FiniteChoice.QAcceptant q C
```

## Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.QAcceptant](#)
- [DGD26AdmissionsPredictability.PaperChoiceRule](#)

## DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → ℕ → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (q : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  DGD26AdmissionsPredictability.paper_q_acceptant q C
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_q\\_acceptant](#)

## DGD26AdmissionsPredictability.paper\_q\_representative

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → ℕ → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (q : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  AppliedModelingLib.FiniteChoice.QRepresentative q C
```

## Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.QRepresentative](#)
- [DGD26AdmissionsPredictability.PaperChoiceRule](#)

## DGD26AdmissionsPredictability.paper\_definition\_q\_representativeness

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → ℕ → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (q : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  DGD26AdmissionsPredictability.paper_q_representative q C
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_q\\_representative](#)

## DGD26AdmissionsPredictability.paper\_sequential\_composition

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → List (DGD26AdmissionsPredictability.PaperChoiceRule α) → Finset α → Finset α

value:
fun {α : Type u_1} [DecidableEq α] (Cs : List (DGD26AdmissionsPredictability.PaperChoiceRule α)) (a : Finset α) =>
  AppliedModelingLib.FiniteChoice.sequentialComposition Cs a
```

## Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.sequentialComposition](#)
- [DGD26AdmissionsPredictability.PaperChoiceRule](#)

## DGD26AdmissionsPredictability.paper\_definition\_sequential\_composition

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → List (DGD26AdmissionsPredictability.PaperChoiceRule α) → Finset α → Finset α

value:
fun {α : Type u_1} [DecidableEq α] (Cs : List (DGD26AdmissionsPredictability.PaperChoiceRule α)) (a : Finset α) =>
  DGD26AdmissionsPredictability.paper_sequential_composition Cs a
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_sequential\\_composition](#)

## DGD26AdmissionsPredictability.paper\_substitutable

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  AppliedModelingLib.FiniteChoice.Substitutable C
```

## Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.Substitutable](#)
- [DGD26AdmissionsPredictability.PaperChoiceRule](#)

## DGD26AdmissionsPredictability.paper\_definition\_substitutability

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  DGD26AdmissionsPredictability.paper_substitutable C
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_substitutable](#)

## DGD26AdmissionsPredictability.paper\_tightly\_d\_unstable

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → ℕ → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (d : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  AppliedModelingLib.FiniteChoice.TightlyDUnstable d C
```

## Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.TightlyDUnstable](#)
- [DGD26AdmissionsPredictability.PaperChoiceRule](#)

## DGD26AdmissionsPredictability.paper\_definition\_tight\_d\_instability

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → ℕ → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (d : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  DGD26AdmissionsPredictability.paper_tightly_d_unstable d C
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_tightly\\_d\\_unstable](#)

## DGD26AdmissionsPredictability.paper\_variability\_at\_most

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → [Fintype α] → ℕ → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] [Fintype α] (m : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  AppliedModelingLib.FiniteChoice.VariabilityAtMost m C
```

## Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.VariabilityAtMost](#)
- [DGD26AdmissionsPredictability.PaperChoiceRule](#)

## DGD26AdmissionsPredictability.paper\_definition\_variability\_at\_most

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → [Fintype α] → ℕ → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] [Fintype α] (m : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  DGD26AdmissionsPredictability.paper_variability_at_most m C
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_variability\\_at\\_most](#)

## DGD26AdmissionsPredictability.paper\_variability\_exactly

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → [Fintype α] → ℕ → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] [Fintype α] (m : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  AppliedModelingLib.FiniteChoice.VariabilityExactly m C
```

## Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.VariabilityExactly](#)
- [DGD26AdmissionsPredictability.PaperChoiceRule](#)

## DGD26AdmissionsPredictability.paper\_definition\_variability\_exactly

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → [Fintype α] → ℕ → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] [Fintype α] (m : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  DGD26AdmissionsPredictability.paper_variability_exactly m C
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_variability\\_exactly](#)

## DGD26AdmissionsPredictability.paper\_waitlisted\_set

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} →
[inst : DecidableEq α] → [Fintype α] → DGD26AdmissionsPredictability.PaperChoiceRule α → Finset α → Finset α

value:
fun {α : Type u_1} [DecidableEq α] [Fintype α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) (X : Finset α) =>
  AppliedModelingLib.FiniteChoice.waitlistedSet C X
```

## Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.waitlistedSet](#)
- [DGD26AdmissionsPredictability.PaperChoiceRule](#)

## DGD26AdmissionsPredictability.paper\_definition\_waitlisted\_set

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} →
[inst : DecidableEq α] → [Fintype α] → DGD26AdmissionsPredictability.PaperChoiceRule α → Finset α → Finset α

value:
fun {α : Type u_1} [DecidableEq α] [Fintype α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) (X : Finset α) =>
  DGD26AdmissionsPredictability.paper_waitlisted_set C X
```

## Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_waitlisted\\_set](#)

## DGD26AdmissionsPredictability.paper\_zero\_unstable

**Declaration location:** paper-local

## Lean declaration body

```
type:
{α : Type u_1} → [inst : DecidableEq α] → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
  AppliedModelingLib.FiniteChoice.ZeroUnstable C
```

## Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ZeroUnstable](#)
- [DGD26AdmissionsPredictability.PaperChoiceRule](#)

## DGD26AdmissionsPredictability.paper\_definition\_zero\_instability

**Declaration location:** paper-local

## Lean declaration body

type:  
{ $\alpha$  : Type u\_1} → [inst : DecidableEq  $\alpha$ ] → DGD26AdmissionsPredictability.PaperChoiceRule  $\alpha$  → Prop

value:  
fun { $\alpha$  : Type u\_1} [DecidableEq  $\alpha$ ] (C : DGD26AdmissionsPredictability.PaperChoiceRule  $\alpha$ ) =>  
DGD26AdmissionsPredictability.paper\_zero\_unstable C

## Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_zero\\_unstable](#)

# Material library prerequisites

## AppliedModelingLib.FiniteChoice.ChoiceRule

**Source locator:** cited publication, lines 172-181

### Verbatim paper-source connection

\subsection{Choice Functions}

Many admissions processes have capacity constraints (either perceived or actual). This fact gives rise to what we call \textit{cohort-dependence}: an applicant's acceptance decision depends on not only their own characteristics, but also the number and characteristics of other applicants.

\textit{Choice functions} formalize admission decision processes and allow modeling such dependence.

A choice function describes the behavior of ``choosing'' elements from a set of available options, and is commonly used in economic models. Formally, a choice function over the universe  $\mathcal{X}$  is a function  $\text{choice}: 2^{\mathcal{X}} \rightarrow 2^{\mathcal{X}}$  that maps all subsets  $X$  of  $\mathcal{X}$  to the accepted set  $\text{choice}(X)$ , where  $\text{choice}(X) \subseteq X$ . As our motivation focuses on real applicant datasets, we consider finite subsets.

Recalling admissions data, the binary admissions label  $y$  for student  $x$  is 1 if  $x \in \text{choice}(X)$  and 0 otherwise.

To aid in our discussion, we will slightly abuse notation to relate pairs  $(x, y)$  to the choice function  $\text{choice}(X)$  where  $x \in X$ .

Define  $\text{choice}(X)_i$  as 1 if  $x_i \in \text{choice}(X)$  and 0 otherwise; this allows us to write the label  $y_i = \text{choice}(X)_i$ .

### Lean-expanded library semantic target

type:

$(\alpha : \text{Type } u_2) \rightarrow [\text{DecidableEq } \alpha] \rightarrow \text{Type } u_2$

value:

$\text{fun } (\alpha : \text{Type } u_2) [\text{DecidableEq } \alpha] \Rightarrow \text{Finset } \alpha \rightarrow \text{Finset } \alpha$

### Exact Lean library declaration

*-- A finite choice rule maps every finite feasible set to a finite chosen set. -/*

*abbrev ChoiceRule*  $(\alpha : \text{Type}^*) [\text{DecidableEq } \alpha] := \text{Finset } \alpha \rightarrow \text{Finset } \alpha$

### Recorded source-to-library screening Verdict: matches

**Reason:** The source model's finite-set map is represented by  $\text{Finset } \alpha \rightarrow \text{Finset } \alpha$ ; its separate chosen-subset requirement is the distinct Feasible predicate reviewed from the same model source.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

### Reviewer annotation



## AppliedModelingLib.FiniteChoice.Consistent

**Source locator:** cited publication, lines 492-495

### Verbatim paper-source connection

```
\begin{restatable}[Consistency]{definition}{consistency}
\label{def:consistency}
Choice function  $\text{choice}$  is \textit{consistent} if  $\text{choice}(X_2) \subseteq X_1 \subseteq X_2$  implies  $\text{choice}(X_2) = \text{choice}(X_1)$ .
\end{restatable}
```

### Lean-expanded library semantic target

```
type:
{α : Type u_1} → [inst : DecidableEq α] → AppliedModelingLib.FiniteChoice.ChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (C : AppliedModelingLib.FiniteChoice.ChoiceRule α) =>
  ∀ {X₁ X₂ : Finset α}, C X₂ ⊆ X₁ → X₁ ⊆ X₂ → C X₂ = C X₁
```

### Exact Lean library declaration

```
/-- Consistency: removing rejected alternatives does not change the chosen set. -/
def Consistent (C : ChoiceRule α) : Prop :=
  ∀ {X₁ X₂}, C X₂ ⊆ X₁ → X₁ ⊆ X₂ → C X₂ = C X₁
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)

### Recorded source-to-library screening Verdict: matches

**Reason:** Both state that whenever  $C X_2 \subseteq X_1 \subseteq X_2$ , removing the rejected alternatives leaves the chosen set unchanged:  $C X_2 = C X_1$ .

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

#### Reviewer annotation

## AppliedModelingLib.FiniteChoice.choiceGainTerm

**Source locator:** cited publication, lines 213-218

### Verbatim paper-source connection

```
\begin{restatable}[Choice Distance]{definition}{distance} \label{def:distance}
For a choice function  $\text{\choice}$  and sets  $X_1 \subseteq X_2$ , we define the choice distance as:
\begin{equation}
r_{\text{\choice}}(X_1, X_2) := |X_1 \cap \text{\choice}(X_2) \setminus \text{\choice}(X_1)| + |\text{\choice}(X_1) \setminus \text{\choice}(X_2)|
\end{equation}
\end{restatable}
```

### Lean-expanded library semantic target

```
type:
{α : Type u_1} → [inst : DecidableEq α] → AppliedModelingLib.FiniteChoice.ChoiceRule α → Finset α → Finset α → ℕ

value:
fun {α : Type u_1} [DecidableEq α] (C : AppliedModelingLib.FiniteChoice.ChoiceRule α) (X₁ X₂ : Finset α) =>
((X₁ ∩ C X₂) \ C X₁).card
```

### Exact Lean library declaration

```
/-- The rejection-to-acceptance term in finite choice distance. -/
def choiceGainTerm (C : ChoiceRule α) (X₁ X₂ : Finset α) : ℕ :=
((X₁ ∩ C X₂) \ C X₁).card
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)

### Recorded source-to-library screening Verdict: matches

**Reason:** The first Lean summand is exactly the source's cardinality of earlier-pool applicants newly selected in the later pool.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

#### Reviewer annotation

## AppliedModelingLib.FiniteChoice.choiceLossTerm

**Source locator:** cited publication, lines 213-218

### Verbatim paper-source connection

```
\begin{restatable}[Choice Distance]{definition}{distance} \label{def:distance}
For a choice function  $\text{\choice}$  and sets  $X_1 \subseteq X_2$ , we define the choice distance as:
\begin{equation}
r_{\text{\choice}}(X_1, X_2) := |X_1 \cap \text{\choice}(X_2) \setminus \text{\choice}(X_1)| + |\text{\choice}(X_1) \setminus \text{\choice}(X_2)|
\end{equation}
\end{restatable}
```

### Lean-expanded library semantic target

```
type:
{α : Type u_1} → [inst : DecidableEq α] → AppliedModelingLib.FiniteChoice.ChoiceRule α → Finset α → Finset α → ℕ

value:
fun {α : Type u_1} [DecidableEq α] (C : AppliedModelingLib.FiniteChoice.ChoiceRule α) (X₁ X₂ : Finset α) =>
(C X₁ \ C X₂).card
```

### Exact Lean library declaration

```
/-- The acceptance-to-rejection term in finite choice distance. -/
def choiceLossTerm (C : ChoiceRule α) (X₁ X₂ : Finset α) : ℕ :=
(C X₁ \ C X₂).card
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)

### Recorded source-to-library screening Verdict: matches

**Reason:** The second Lean summand is exactly the source's cardinality of applicants selected in the earlier pool but not the later pool.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

#### Reviewer annotation

## AppliedModelingLib.FiniteChoice.choiceDistance

**Source locator:** cited publication, lines 213-218

### Verbatim paper-source connection

```
\begin{restatable}[Choice Distance]{definition}{distance} \label{def:distance}
For a choice function  $\text{\choice}$  and sets  $X_1 \subseteq X_2$ , we define the choice distance as:
\begin{equation}
r_{\text{\choice}}(X_1, X_2) := |X_1 \cap \text{\choice}(X_2) \setminus \text{\choice}(X_1)| + |\text{\choice}(X_1) \setminus \text{\choice}(X_2)|
\end{equation}
\end{restatable}
```

### Lean-expanded library semantic target

```
type:
{α : Type u_1} → [inst : DecidableEq α] → AppliedModelingLib.FiniteChoice.ChoiceRule α → Finset α → Finset α → ℕ

value:
fun {α : Type u_1} [DecidableEq α] (C : AppliedModelingLib.FiniteChoice.ChoiceRule α) (X₁ X₂ : Finset α) =>
  AppliedModelingLib.FiniteChoice.choiceGainTerm C X₁ X₂ + AppliedModelingLib.FiniteChoice.choiceLossTerm C X₁ X₂
```

### Exact Lean library declaration

```
/--
Choice distance: the number of existing decisions whose label changes when the
offered set expands from `X₁` to `X₂`.
-/
def choiceDistance (C : ChoiceRule α) (X₁ X₂ : Finset α) : ℕ :=
  choiceGainTerm C X₁ X₂ + choiceLossTerm C X₁ X₂
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)
- [AppliedModelingLib.FiniteChoice.choiceGainTerm](#)
- [AppliedModelingLib.FiniteChoice.choiceLossTerm](#)

### Recorded source-to-library screening Verdict: matches

**Reason:** For the source's nested inputs, Lean defines the choice distance as the same sum of the rejection-to-acceptance and acceptance-to-rejection cardinalities.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

#### Reviewer annotation

## AppliedModelingLib.FiniteChoice.DUnstable

**Source locator:** cited publication, lines 222-225

### Verbatim paper-source connection

```
\begin{restatable}[$d$-Instability]{definition}{defstability}
\label{def:d-instability}
Choice function  $\text{\textit{choice}}$  is  $d$ -$-unstable if for all  $X$ , and  $X' = X \cup \{x'\}$ , we have  $r_{\text{\textit{choice}}}(X, X') \leq d$  for any nonnegative integer  $d$ .
 $\leftrightarrow$   $\text{\textit{choice}}$  is called tightly  $d$ -$-unstable if it is  $d$ -$-unstable but not  $(d-1)$ -$-unstable.
\end{restatable}
```

### Lean-expanded library semantic target

```
type:
{α : Type u_1} → [inst : DecidableEq α] → ℕ → AppliedModelingLib.FiniteChoice.ChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (d : ℕ) (C : AppliedModelingLib.FiniteChoice.ChoiceRule α) =>
  ∀ (X : Finset α), ∀ x ∉ X, AppliedModelingLib.FiniteChoice.choiceDistance C X (insert x X) ≤ d
```

### Exact Lean library declaration

```
/-- `d`-instability for one fresh added alternative. -/
def DUnstable (d : ℕ) (C : ChoiceRule α) : Prop :=
  ∀ X x, x ∉ X → choiceDistance C X (insert x X) ≤ d
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)
- [AppliedModelingLib.FiniteChoice.choiceDistance](#)

**Recorded source-to-library screening Verdict:** matches

**Reason:** The source requires  $r_{\text{\textit{C}}}(X, X \cup \{x'\}) \leq d$  for every newly appended applicant; Lean quantifies finite  $X$  and fresh  $x \notin X$  with the same distance and bound.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## AppliedModelingLib.FiniteChoice.Feasible

**Source locator:** cited publication, lines 172-181

### Verbatim paper-source connection

\subsection{Choice Functions}

Many admissions processes have capacity constraints (either perceived or actual). This fact gives rise to what we call \textit{cohort-dependence}: an applicant's acceptance decision depends on not only their own characteristics, but also the number and characteristics of other applicants.

\textit{Choice functions} formalize admission decision processes and allow modeling such dependence.

A choice function describes the behavior of ``choosing'' elements from a set of available options, and is commonly used in economic models. Formally, a

choice function over the universe  $\mathcal{X}$  is a function  $\text{choice} : 2^{\mathcal{X}} \rightarrow 2^{\mathcal{X}}$  that maps all subsets  $X$  of  $\mathcal{X}$  to the accepted set  $\text{choice}(X)$ , where  $\text{choice}(X) \subseteq X$ . As our motivation focuses on real applicant datasets, we consider finite subsets.

Recalling admissions data, the binary admissions label  $y$  for student  $x$  is 1 if  $x \in \text{choice}(X)$  and 0 otherwise.

To aid in our discussion, we will slightly abuse notation to relate pairs  $(x_i, y_i)$  to the choice function  $\text{choice}(X)$  where  $x_i \in X$ .

Define  $C(X)_i$  as 1 if  $x_i \in \text{choice}(X)$  and 0 otherwise; this allows us to write the label  $y_i = C(X)_i$ .

### Lean-expanded library semantic target

type:

$\{\alpha : \text{Type } u_1\} \rightarrow [\text{inst} : \text{DecidableEq } \alpha] \rightarrow \text{AppliedModelingLib.FiniteChoice.ChoiceRule } \alpha \rightarrow \text{Prop}$

value:

$\text{fun } \{\alpha : \text{Type } u_1\} [\text{DecidableEq } \alpha] (C : \text{AppliedModelingLib.FiniteChoice.ChoiceRule } \alpha) \Rightarrow \forall (X : \text{Finset } \alpha), C X \subseteq X$

### Exact Lean library declaration

*-- Feasibility: the chosen set is always contained in the offered set. -/*

*def Feasible (C : ChoiceRule  $\alpha$ ) : Prop :=*

*$\forall X, C X \subseteq X$*

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)

**Recorded source-to-library screening Verdict:** matches

**Reason:** The source model requires  $C(X) \subseteq X$  for every offered set, exactly the universal Finset-subset condition in Feasible.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## AppliedModelingLib.FiniteChoice.borderlineSet

**Source locator:** cited publication, lines 826-830

### Verbatim paper-source connection

As per \Cref{def:variability}, when appending a new element to the input set  $X$ , different applicants may become rejected: let the set of ``borderline  $\hookrightarrow$  admits''  $\mathcal{V} \setminus \text{choice}(X)$  be

$$\mathcal{V} \setminus \text{choice}(X) = \bigcup \limits_{x \in \mathcal{A} \setminus X} \setminus \text{choice}(X) \setminus \text{choice}(X \cup \{x\})$$

which we shall call the ``borderline set'' for short. Notice that this is bounded by  $q$ , as  $|\setminus \text{choice}(X)| = q$ .

### Lean-expanded library semantic target

```
type:
{α : Type u_1} →
[inst : DecidableEq α] → [Fintype α] → AppliedModelingLib.FiniteChoice.ChoiceRule α → Finset α → Finset α

value:
fun {α : Type u_1} [DecidableEq α] [Fintype α] (C : AppliedModelingLib.FiniteChoice.ChoiceRule α) (X : Finset α) =>
  Finset.univ.biUnion fun (x : α) => C X \ C (insert x X)
```

### Exact Lean library declaration

```
/-- Borderline admits displaced by some single added applicant. -/
def borderlineSet [Fintype α] (C : ChoiceRule α) (X : Finset α) : Finset α :=
  Finset.univ.biUnion fun x => C X \ C (insert x X)
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)

**Recorded source-to-library screening** **Verdict:** matches

**Reason:** The source union over every applicant  $x'$  in the universe of the chosen applicants displaced by adding  $x'$  is exactly Lean's finite-universe `biUnion`.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## AppliedModelingLib.FiniteChoice.waitlistedSet

**Source locator:** cited publication, lines 832-836

### Verbatim paper-source connection

Conversely, when removing an applicant from  $X$ , let the set of applicants that might be newly accepted be

$$A \setminus \text{choice}(X) = \bigcup_{x^* \in X} (\text{choice}(X \setminus \{x^*\}) \setminus \text{choice}(X))$$

the set of "waitlisted rejections"; likewise, we shall call this the "waitlisted set".

### Lean-expanded library semantic target

```
type:
{α : Type u_1} →
[inst : DecidableEq α] → [Fintype α] → AppliedModelingLib.FiniteChoice.ChoiceRule α → Finset α → Finset α

value:
fun {α : Type u_1} [DecidableEq α] [Fintype α] (C : AppliedModelingLib.FiniteChoice.ChoiceRule α) (X : Finset α) =>
X.biUnion fun x => C (X.erase x) \ C X
```

### Exact Lean library declaration

```
/-- Waitlisted rejections newly accepted after removing some current applicant. -/
def waitlistedSet [Fintype α] (C : ChoiceRule α) (X : Finset α) : Finset α :=
X.biUnion fun x => C (X.erase x) \ C X
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)

**Recorded source-to-library screening Verdict:** matches

**Reason:** Both union over each currently offered applicant the newly accepted applicants obtained by removing that applicant; Lean's erase is the finite-set removal in the source formula.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**



## AppliedModelingLib.FiniteChoice.GeneralVariabilityExactly

**Source locator:** cited publication, lines 838-842

### Verbatim paper-source connection

```
\begin{definition}[Variability, General]
\label{def:variability-general}
A choice function  $\text{\textit{\textit{choice}}}$  has  $\text{\textit{\textit{variability}}}$   $m$  where:

$$m := \max\left\{\max_{X \subset \text{\textit{\textit{calX}}}} \left|V_{\text{\textit{\textit{choice}}}(X)}\right|, \max_{X \subset \text{\textit{\textit{calX}}} \left|A_{\text{\textit{\textit{choice}}}(X)}\right| \right\}$$

\end{definition}
```

### Lean-expanded library semantic target

```
type:
{α : Type u_1} → [inst : DecidableEq α] → [Fintype α] → ℕ → AppliedModelingLib.FiniteChoice.ChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] [Fintype α] (m : ℕ) (C : AppliedModelingLib.FiniteChoice.ChoiceRule α) =>
  AppliedModelingLib.FiniteChoice.GeneralVariabilityAtMost m C ∧
  ((∃ (X : Finset α), (AppliedModelingLib.FiniteChoice.borderlineSet C X).card = m) ∨
   ∃ (X : Finset α), (AppliedModelingLib.FiniteChoice.waitlistedSet C X).card = m)
```

### Exact Lean library declaration

```
/-- Appendix exact general variability, stated by upper bounds and a witnessing side. -/
def GeneralVariabilityExactly [Fintype α] (m : ℕ) (C : ChoiceRule α) : Prop :=
  GeneralVariabilityAtMost m C ∧
  ((∃ X, (borderlineSet C X).card = m) ∨
   (∃ X, (waitlistedSet C X).card = m))
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)
- [AppliedModelingLib.FiniteChoice.GeneralVariabilityAtMost](#)
- [AppliedModelingLib.FiniteChoice.borderlineSet](#)
- [AppliedModelingLib.FiniteChoice.waitlistedSet](#)

**Recorded source-to-library screening** **Verdict:** matches

**Reason:** Under the displayed finite-universe convention, Lean's two all-pool upper bounds plus a borderline-or-waitlist equality witness are equivalent to the source's maximum of the two maxima being  $m$ .

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## AppliedModelingLib.FiniteChoice.Independent

**Source locator:** cited publication, lines 671-679

### Verbatim paper-source connection

```
\begin{restatable}{definition}{Independence}
\label{def:independence}
 $\$choice\$$  is independent if for
  for all  $x$ ,
  either  $x \in choice(X)$ 
  or  $x \notin choice(X)$  for all  $X$  where  $x \in X$ .
\end{restatable}
```

In other words, any element is either always accepted, or always rejected, when available, irrespective of the other elements available in a set  $S$ .

### Lean-expanded library semantic target

```
type:
{α : Type u_1} → [inst : DecidableEq α] → AppliedModelingLib.FiniteChoice.ChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (C : AppliedModelingLib.FiniteChoice.ChoiceRule α) =>
  ∀ (x : α), (∀ (X : Finset α), x ∈ X → x ∈ C X) ∨ ∀ (X : Finset α), x ∈ X → x ∉ C X
```

### Exact Lean library declaration

```
--
Independence: each alternative is either always chosen whenever available or
always rejected whenever available.
-/
def Independent (C : ChoiceRule α) : Prop :=
  ∀ x, (∀ X, x ∈ X → x ∈ C X) ∨ (∀ X, x ∈ X → x ∉ C X)
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)

**Recorded source-to-library screening** **Verdict:** matches

**Reason:** Both require that each applicant, on every pool in which they occur, is either always selected or always rejected.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## AppliedModelingLib.FiniteChoice.Monotonic

**Source locator:** cited publication, lines 484-487

### Verbatim paper-source connection

```
\begin{restatable}[Monotonicity]{definition}{monotonicity}
\label{def:monotonicity}
Choice function  $\text{choice}$  is \textit{monotonic} if  $X_1 \subseteq X_2$  implies  $\text{choice}(X_1) \subseteq \text{choice}(X_2)$ .
\end{restatable}
```

### Lean-expanded library semantic target

```
type:
{α : Type u_1} → [inst : DecidableEq α] → AppliedModelingLib.FiniteChoice.ChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (C : AppliedModelingLib.FiniteChoice.ChoiceRule α) =>
  ∀ {X₁ X₂ : Finset α}, X₁ ⊆ X₂ → C X₁ ⊆ C X₂
```

### Exact Lean library declaration

```
/-- Monotonicity: expanding the offered set preserves every earlier choice. -/
def Monotonic (C : ChoiceRule α) : Prop :=
  ∀ {X₁ X₂}, X₁ ⊆ X₂ → C X₁ ⊆ C X₂
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)

**Recorded source-to-library screening Verdict:** matches

**Reason:** The source implication  $X_1 \subseteq X_2 \rightarrow C(X_1) \subseteq C(X_2)$  is exactly the Lean Monotonic target.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## AppliedModelingLib.FiniteChoice.QAcceptant

**Source locator:** cited publication, lines 186-189

### Verbatim paper-source connection

```
\begin{restatable}[$q$-Acceptance]{definition}{qacceptance}
\label{def:q-acceptance}
Choice function  $\text{\textit{choice}}$  is  $q$ -acceptant if  $|\text{\textit{choice}}(X)| = \min\{q, |X|\}$  for all input sets  $X$ .
\end{restatable}
```

[Next verbatim source excerpt]

```
\qacceptance*
\textit{Intuition}: Capacity of  $q$ .

\substitutability*
```

### Lean-expanded library semantic target

```
type:
{α : Type u_1} → [inst : DecidableEq α] → ℕ → AppliedModelingLib.FiniteChoice.ChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (q : ℕ) (C : AppliedModelingLib.FiniteChoice.ChoiceRule α) =>
  ∀ (X : Finset α), (C X).card = min q X.card
```

### Exact Lean library declaration

```
/-- `q`-acceptance: choose as many alternatives as possible, up to capacity `q`. -/
def QAcceptant (q : ℕ) (C : ChoiceRule α) : Prop :=
  ∀ X, (C X).card = min q X.card
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)

**Recorded source-to-library screening Verdict:** matches

**Reason:** Both require every offered finite pool to have exactly  $\min(q, |X|)$  chosen applicants.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## AppliedModelingLib.FiniteChoice.StrictTotalOrder

**Source locator:** cited publication, lines 193-195

### Verbatim paper-source connection

```
\begin{definition}[Total order] \label{def:total_order}
  A \emph{total order} is a transitive, asymmetric, and complete binary relation  $\text{\textbackslash succ}$  over elements of  $\text{\textbackslash calX}$ ; that is,  $x_1 \text{\textbackslash succ } x_2$  implies  $x_2$ 
   $\rightarrow$   $\text{\textbackslash not succ } x_1$ ,  $x_1 \text{\textbackslash succ } x_2$  and  $x_2 \text{\textbackslash succ } x_3$  implies  $x_1 \text{\textbackslash succ } x_3$ , and all  $x_1, x_2$  are comparable.
\end{definition}
```

**Formalized review target** The archival source above is not asserted equivalent to this formalized target.

A strict total order is asymmetric and transitive, and compares every distinct pair of applicants.

**Archival source anchor:** cited publication, lines 193-195

### Lean-expanded library semantic target

```
type:
{α : Type u_1} → (α → α → Prop) → Prop

value:
fun {α : Type u_1} (r : α → α → Prop) =>
  (∀ (x : α), ¬ r x x) ∧ (∀ {x y z : α}, r x y → r y z → r x z) ∧ ∀ {x y : α}, x ≠ y → r x y ∨ r y x
```

### Exact Lean library declaration

```
/-- A strict total order in the asymmetric/transitive/complete sense. -/
def StrictTotalOrder (r : α → α → Prop) : Prop :=
  (∀ x, ¬ r x x) ∧
  (∀ {x y z}, r x y → r y z → r x z) ∧
  (∀ {x y}, x ≠ y → r x y ∨ r y x)
```

**Recorded source-to-library screening Verdict:** matches approved corrected target

**Reason:** Compared the archival total-order sentence, the complete approval-pinned corrected target DGD26-TOTAL-ORDER-DISTINCT-01, and the actual StrictTotalOrder body. Lean requires irreflexivity, transitivity and comparability of every distinct pair. Irreflexivity plus transitivity is equivalent to the corrected target's asymmetry plus transitivity, and the distinct-pair quantifier is exact. The displayed QRepresentative and four paper uses retain that same relation. The queue expressly disclaims archival equivalence and supplies the correction statement, scope, defect id and approval excerpt, so the approved-corrected-target verdict is required.

**Reviewer check:** ☐ Matches formalized target

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## AppliedModelingLib.FiniteChoice.QRepresentative

**Source locator:** cited publication, lines 500-507

### Verbatim paper-source connection

```
\begin{restatable}[$q$-representativeness]{definition}{qrepresentativeness}
\label{def:q-representative}
$q$-acceptant choice function $\textit{choice}$ is $q$-representative if there exists a strict total ordering $\textit{succ}$ over all $x \in \textit{calX}$ and $\textit{choice}(X)$ is the
 $\hookrightarrow$  top $q$ elements of $X$ for all $X \subseteq \textit{calX}$.
\end{restatable}
```

$\textit{Intuition}$ : Students are selected according to an exact ranking.

$\textit{Note}$ : We refer to this property in the main body as being ``characterized by a total order'', or a ``queue'', but this language --- and definition as  $\hookrightarrow$  a property in its own right --- is seen in, e.g., \citet{echenique2015control}.

[Next verbatim source excerpt]

Total orders are frequently used in modeling choice mechanisms; they are often called  $\textit{preference orderings}$  or  $\textit{priority orders}$  in economic models. Intuitively, total orders induce a ranking over candidates and could result from, e.g., ordering applicants based on their GPA. We will say that a \$q\$-acceptant choice function \$\textit{C}\$ is  $\textit{characterized by a total order}$  if there exists a total order \$\textit{succ}\$ such that for any input \$X\$, the accepted applicants are the top-\$q\$ elements of \$X\$ ordered by \$\textit{succ}\$. In other words, \$x\_1 \textit{succ} x\_2\$ for all \$x\_1 \in \textit{choice}(X)\$, \$x\_2 \notin \textit{choice}(X)\$. We will refer to the process of sorting elements according to a total order \$\textit{succ}\$ and selecting the top \$q\$ as a  $\textit{queue}$ .

### Lean-expanded library semantic target

```
type:
{α : Type u_1} → [inst : DecidableEq α] → ℕ → AppliedModelingLib.FiniteChoice.ChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (q : ℕ) (C : AppliedModelingLib.FiniteChoice.ChoiceRule α) =>
  ∃ (r : α → α → Prop),
    AppliedModelingLib.FiniteChoice.StrictTotalOrder r ∧
    AppliedModelingLib.FiniteChoice.Feasible C ∧
    AppliedModelingLib.FiniteChoice.QAcceptant q C ∧ ∀ {X : Finset α} {x y : α}, x ∈ C X → y ∈ X → y ∉ C X → r x y
```

### Exact Lean library declaration

```
/-
`C` is represented by a single priority order if it chooses only offered
applicants, is q-acceptant, and every chosen applicant is above every rejected
applicant in each finite pool.
-/
def QRepresentative (q : ℕ) (C : ChoiceRule α) : Prop :=
  ∃ r : α → α → Prop,
    StrictTotalOrder r ∧ Feasible C ∧ QAcceptant q C ∧
    ∀ {X x y}, x ∈ C X → y ∈ X → y ∉ C X → r x y
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)
- [AppliedModelingLib.FiniteChoice.Feasible](#)
- [AppliedModelingLib.FiniteChoice.QAcceptant](#)
- [AppliedModelingLib.FiniteChoice.StrictTotalOrder](#)

### Recorded source-to-library screening Verdict: matches

**Reason:** Independent source-first review: the predicate requires one strict total order, q-acceptance, and the source's selected-above-unselected condition for every finite pool. Its explicit Feasible C conjunct ensures every selected applicant belongs to the offered pool, as required by the source's top-q-of-X language; without it, the priority implication could be vacuous for an off-pool applicant.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

### Reviewer annotation

## AppliedModelingLib.FiniteChoice.Substitutable

**Source locator:** cited publication, lines 273-276

### Verbatim paper-source connection

```
\begin{restatable}[Substitutability]{definition}{substitutability}
\label{def:substitutability}
Choice function  $\text{\textit{choice}}$  is substitutable if  $X_1 \subseteq X_2$  implies  $X_1 \cap \text{\textit{choice}}(X_2) \subseteq \text{\textit{choice}}(X_1)$ .
\end{restatable}
```

[Next verbatim source excerpt]

```
\substitutability*
\textit{Intuition}: Students selected from a larger pool remain selected when the pool shrinks.
```

### Lean-expanded library semantic target

```
type:
{α : Type u_1} → [inst : DecidableEq α] → AppliedModelingLib.FiniteChoice.ChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (C : AppliedModelingLib.FiniteChoice.ChoiceRule α) =>
  ∀ {X₁ X₂ : Finset α}, X₁ ⊆ X₂ → X₁ ∩ C X₂ ⊆ C X₁
```

### Exact Lean library declaration

```
-- Substitutability: shrinking the offered set does not hurt retained chosen elements. -/
def Substitutable (C : ChoiceRule α) : Prop :=
  ∀ {X₁ X₂}, X₁ ⊆ X₂ → X₁ ∩ C X₂ ⊆ C X₁
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)

**Recorded source-to-library screening Verdict:** matches

**Reason:** Both say that for  $X_1 \subseteq X_2$ , every member of  $X_1$  selected from  $X_2$  remains selected from  $X_1$ .

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## AppliedModelingLib.FiniteChoice.TightlyDUnstable

**Source locator:** cited publication, lines 222-225

### Verbatim paper-source connection

```
\begin{restatable}[$d$-Instability]{definition}{defstability}
\label{def:d-instability}
Choice function  $\text{\textit{choice}}$  is  $d$ -unstable if for all  $X$ , and  $X' = X \cup \{x'\}$ , we have  $r_{\text{\textit{choice}}}(X, X') \leq d$  for any nonnegative integer  $d$ .
 $\hookrightarrow$   $\text{\textit{choice}}$  is called tightly  $d$ -unstable if it is  $d$ -unstable but not  $(d-1)$ -unstable.
\end{restatable}
```

**Settled source reading** The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE\_CLARIFICATIONS.md).

### Lean-expanded library semantic target

```
type:
{α : Type u_1} → [inst : DecidableEq α] → ℕ → AppliedModelingLib.FiniteChoice.ChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (d : ℕ) (C : AppliedModelingLib.FiniteChoice.ChoiceRule α) =>
  AppliedModelingLib.FiniteChoice.DUnstable d C ∧ ∀ k < d, ¬AppliedModelingLib.FiniteChoice.DUnstable k C
```

### Exact Lean library declaration

```
/-- Tight `d`-instability: `d` is the least one-step instability bound. -/
def TightlyDUnstable (d : ℕ) (C : ChoiceRule α) : Prop :=
  DUnstable d C ∧ ∀ k, k < d → ¬ DUnstable k C
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)
- [AppliedModelingLib.FiniteChoice.DUnstable](#)

**Recorded source-to-library screening** **Verdict:** matches

**Reason:** On the approved positive- $d$  source domain, Lean's  $DUnstable\ d$  plus failure of every lower bound is equivalent to source  $d$ -unstable but not  $(d-1)$ -unstable, since the distance bound is monotone in  $d$ .

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**



## AppliedModelingLib.FiniteChoice.VariabilityAtMost

**Source locator:** cited publication, lines 998-1003

### Verbatim paper-source connection

```
\begin{restatable}[Additive Variability Bound]{theorem}{addvar}
\label{thm:additive-variability}
If  $\text{\textit{choice}}$  is a sequential composition of  $q$ -acceptant,  $l$ -unstable functions with variabilities  $m_1, \dots, m_n$ , then  $\text{\textit{choice}}$  has variability at
 $\hookrightarrow$  most  $\sum_{i=1}^n m_i$ .
\end{restatable}
```

Note that this is only an upper bound; we could trivially construct cases where this is not tight. For example, compose a  $\text{\textit{choice}}$  of two  $q$ -representative choice functions with the same underlying ordering. On the other hand, it is also easy to construct instances in which this upper bound is tight. Consider the composition of  $m$  queues which each rank a student by a different subject area exam. Then the addition of a new student to the applicant pool could displace any of  $m$  students, depending on the subject area.

### Lean-expanded library semantic target

```
type:
{α : Type u_1} → [inst : DecidableEq α] → [Fintype α] → ℕ → AppliedModelingLib.FiniteChoice.ChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] [Fintype α] (m : ℕ) (C : AppliedModelingLib.FiniteChoice.ChoiceRule α) =>
  ∀ (X : Finset α), (AppliedModelingLib.FiniteChoice.borderlineSet C X).card ≤ m
```

### Exact Lean library declaration

```
/-- Main-text variability upper-bound form. -/
def VariabilityAtMost [Fintype α] (m : ℕ) (C : ChoiceRule α) : Prop :=
  ∀ X, (borderlineSet C X).card ≤ m
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)
- [AppliedModelingLib.FiniteChoice.borderlineSet](#)

**Recorded source-to-library screening Verdict:** matches

**Reason:** The selected theorem's conclusion that variability is at most a bound is exactly that every finite-pool borderline set has cardinality at most that bound; the finite-universe convention makes the source maximum literal.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## AppliedModelingLib.FiniteChoice.VariabilityExactly

**Source locator:** cited publication, lines 235-243

### Verbatim paper-source connection

We formalize<sup>footnote</sup>{For a more general definition of variability (beyond 1-unstable and  $q$ -acceptant), see \Cref{sec:variability-appendix}.} this intuition with the following definition.

```
\begin{restatable}[Variability]{definition}{defvariability}
\label{def:variability}
A  $1$ -unstable,  $q$ -acceptant choice function  $\text{\textit{choice}}$  has variability  $m$  where
\begin{equation}
m \coloneqq \max_{X \subset \text{\textit{cal}}X} \left| \bigcup_{x \in \text{\textit{cal}}X} \text{\textit{choice}}(X) \setminus \text{\textit{choice}}(X \cup \{x\}) \right|
\end{equation}
\end{restatable}
```

### Lean-expanded library semantic target

```
type:
{α : Type u_1} → [inst : DecidableEq α] → [Fintype α] → ℕ → AppliedModelingLib.FiniteChoice.ChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] [Fintype α] (m : ℕ) (C : AppliedModelingLib.FiniteChoice.ChoiceRule α) =>
  AppliedModelingLib.FiniteChoice.VariabilityAtMost m C ∧
  ∃ (X : Finset α), (AppliedModelingLib.FiniteChoice.borderlineSet C X).card = m
```

### Exact Lean library declaration

```
/-- Main-text exact variability, stated as an upper bound with a witnessing pool. -/
def VariabilityExactly [Fintype α] (m : ℕ) (C : ChoiceRule α) : Prop :=
  VariabilityAtMost m C ∧ ∃ X, (borderlineSet C X).card = m
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)
- [AppliedModelingLib.FiniteChoice.VariabilityAtMost](#)
- [AppliedModelingLib.FiniteChoice.borderlineSet](#)

**Recorded source-to-library screening Verdict:** matches

**Reason:** With the approved finite universe, Lean's all-pool upper bound and one pool attaining  $m$  are equivalent to the source definition  $m = \max_X |V_C(X)|$ .

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## AppliedModelingLib.FiniteChoice.ZeroUnstable

**Source locator:** cited publication, lines 654-657

### Verbatim paper-source connection

```
\begin{restatable}[$0$-Instability]{definition}{zerostability}
\label{def:zero-instability}
Choice function $\text{choice}$ is 0-unstable if and only if $\text{r\_choice}(X_1, X_2) = 0$ for all $X_1 \subseteq X_2$.
\end{restatable}
```

### Lean-expanded library semantic target

```
type:
{α : Type u_1} → [inst : DecidableEq α] → AppliedModelingLib.FiniteChoice.ChoiceRule α → Prop

value:
fun {α : Type u_1} [DecidableEq α] (C : AppliedModelingLib.FiniteChoice.ChoiceRule α) =>
  ∀ {X₁ X₂ : Finset α}, X₁ ⊆ X₂ → AppliedModelingLib.FiniteChoice.choiceDistance C X₁ X₂ = 0
```

### Exact Lean library declaration

```
/-- Zero instability for arbitrary nested offered sets. -/
def ZeroUnstable (C : ChoiceRule α) : Prop :=
  ∀ {X₁ X₂}, X₁ ⊆ X₂ → choiceDistance C X₁ X₂ = 0
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)
- [AppliedModelingLib.FiniteChoice.choiceDistance](#)

**Recorded source-to-library screening Verdict:** matches

**Reason:** The source's zero-instability condition is precisely zero choice distance for every nested pair of offered sets, as in the Lean target.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## AppliedModelingLib.FiniteChoice.linearTopQChoice

**Source locator:** cited publication, lines 253-259

### Verbatim paper-source connection

Before stating this result, consider a straightforward way to adapt a model~\eqref{eq:ml-independent} to the capacity-constrained setting. Given that the ML model is based on continuous scores, we can adjust the decisions to account for a known capacity constraint if we have available the set  $\hookrightarrow$  of all test-time applications.

In particular, we can rank the applicants by their scores and then select the top  $q$ .

This process results in decisions determined by a cohort dependent threshold  $t_q(X)$  which is the score of the  $q$ th ranked applicant:

$$f(x; X) = \mathbf{1}\{s(x) \geq t_q(X)\}.$$

**Settled source reading** The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE\_CLARIFICATIONS.md).

### Lean-expanded library semantic target

```
type:
{α : Type u_1} → [DecidableEq α] → [LinearOrder α] → ℕ → Finset α → Finset α

value:
fun {α : Type u_1} [DecidableEq α] [LinearOrder α] (q : ℕ) (a : Finset α) =>
  if h : q ≤ a.card then
    Finset.image
      (fun (i : Fin q) =>
        ↑((a.orderIsoOffFin (AppliedModelingLib.FiniteChoice.linearTopQChoice._proof_1 a))
          (↑i, AppliedModelingLib.FiniteChoice.linearTopQChoice._proof_2 q a h i)))
      Finset.univ
  else a
```

### Exact Lean library declaration

```
/--
The q-acceptant rule that chooses the first `q` elements of the offered set in
the ambient linear order, or everyone if the pool has size below capacity.
-/
noncomputable def linearTopQChoice (q : ℕ) : ChoiceRule α :=
  fun X =>
    if h : q ≤ X.card then
      Finset.univ.image
        (fun i : Fin q =>
          ((X.orderIsoOffFin (by rfl)
            (i.1, lt_of_lt_of_le i.2 h) : X) : α))
    else
      X
```

**Recorded source-to-library screening** **Verdict:** matches

**Reason:** The approved rank-threshold context resolves equal scores by a fixed ex-ante order; Lean takes that order and selects its first  $q$  offered elements, or all elements below capacity, exactly as the source ranking rule requires.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## AppliedModelingLib.FiniteChoice.sequentialComposition

**Source locator:** cited publication, lines 307-311

### Verbatim paper-source connection

```
\begin{restatable}[Sequential Composition]{definition}{seqcomposition} \label{def:seq-composition}
A choice function  $\text{\textit{\textit{choice}}}$  is a  $\text{\textit{\textit{sequential composition}}}$  of functions  $\text{\textit{\textit{choice}}}_n, \text{\textit{\textit{choice}}}_1$  if:
 $\text{\textit{\textit{choice}}}(X) = \text{\textit{\textit{choice}}}_1(X) \cup \text{\textit{\textit{choice}}}_2(X_{\{2\}}) \cup \dots \cup \text{\textit{\textit{choice}}}_n(X_n)$ 
where  $X_{\{i+1\}} = X_{\{i\}} \setminus \text{\textit{\textit{choice}}}_i(X_{\{i\}})$  and  $X_1 = X$ .
\end{restatable}
```

[Next verbatim source excerpt]

```
\subsubsection{Sequential Composition} \label{sec:seq-proofs}
A useful class of choice functions are choice functions that can be broken down or defined as a sequential composition of functions  $\text{\textit{\textit{choice}}}_n,$ 
 $\text{\textit{\textit{choice}}}_{n-1}, \dots, \text{\textit{\textit{choice}}}_1$ ; that is,
 $\text{\textit{\textit{choice}}}(X) = \text{\textit{\textit{choice}}}_n(X) \cup \text{\textit{\textit{choice}}}_{n-1}(X_{\{n-1\}}) \cup \dots \cup \text{\textit{\textit{choice}}}_1(X_1)$  where  $X_i = X_{\{i+1\}} \setminus \text{\textit{\textit{choice}}}_{i+1}(X_{\{i+1\}})$  and  $X_n =$ 
 $X$ . Sequential composition is particularly useful because it preserves a variety of useful properties. Sequences of  $q_i$ -acceptant functions are
 $q$ -acceptant, because no duplicate elements are selected by multiple functions, and therefore preserves  $q$ -acceptance when each function in the
sequence has capacity  $q_i$  (i.e.,  $q = q_n + \dots + q_1$ .)
\seqcomposition*
```

### Lean-expanded library semantic target

```
type:
{α : Type u_1} → [inst : DecidableEq α] → List (AppliedModelingLib.FiniteChoice.ChoiceRule α) → Finset α → Finset α

value:
fun {α : Type u_1} [DecidableEq α] (a : List (AppliedModelingLib.FiniteChoice.ChoiceRule α)) (a_1 : Finset α) =>
List.brecOn a AppliedModelingLib.FiniteChoice.sequentialComposition._f a_1
```

### Exact Lean library declaration

```
/--
Sequential composition of choice rules: run the first rule on the current
remaining set, remove its chosen alternatives, then continue with the rest.
-/
def sequentialComposition : List (ChoiceRule α) → ChoiceRule α
| [] => fun _ => ∅
| C :: Cs =>
  fun X =>
    let chosen := C X
    chosen ∪ sequentialComposition Cs (X \ chosen)
```

### Direct retained dependencies

- [AppliedModelingLib.FiniteChoice.ChoiceRule](#)

### Recorded source-to-library screening Verdict: matches

**Reason:** The approved source reading is exactly Lean's recursion: choose the head rule on the current pool, remove its choices, recurse on the tail, union the stages, and return empty for an empty list.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

#### Reviewer annotation

# Paper-specific semantic prerequisites

## DGD26AdmissionsPredictability.LAP.Assignment.SameSlotOrder

### Paper-local declaration:

DGD26AdmissionsPredictability.LAP.Assignment.SameSlotOrder

**Source locator:** cited publication, lines 1081-1083

### Verbatim paper-source connection

Consider all edges connecting to slot  $s$ ; then for each element  $x_j$  in  $X$  there is a real number weight  $w(x_j, s)$ . Then sorting  $x$  in  $\text{cal}X$  by  $\hookrightarrow w(x, s)$  creates a total ordering, which we call  $\text{succ}_s$ . Let slot  $s$ 's be  $\text{equivalent}$  to slot  $s$  if and only if  $\text{succ}_{s'} = \text{succ}_s$ . We will assume for simplicity that weights are defined such that ties do not occur.

**Settled source reading** The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE\_CLARIFICATIONS.md).

### Lean-expanded paper semantic target

```
type:
{α : Type u_1} → {σ : Type u_2} → (α → σ → ℝ) → σ → σ → Prop

value:
fun {α : Type u_1} {σ : Type u_2} (w : α → σ → ℝ) (s t : σ) => ∀ (a b : α), w a s < w b s ↔ w a t < w b t
```

### Recorded source-to-declaration screening Verdict: matches

**Reason:** Compared audit/cited publication:1081-1083 with the complete SameSlotOrder body: for every applicant pair  $a, b$ ,  $w(a, s) < w(b, s)$  iff  $w(a, t) < w(b, t)$ . Extensional equality of these strict real-weight relations is exactly equality of the source slot orders. The displayed sameSlotOrderSetoid uses this relation unchanged, and the slot-order-class use has explicit no-within-slot-ties and operational-objective assumptions governed by DGD26-LAP-DETERMINATE-TIEFREE-01. The target neither requires raw-primary uniqueness nor changes the source equivalence relation.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

### Reviewer annotation

## DGD26AdmissionsPredictability.LAP.Assignment.SlotBelow

### Paper-local declaration:

DGD26AdmissionsPredictability.LAP.Assignment.SlotBelow

**Source locator:** cited publication, lines 1081-1083

### Verbatim paper-source connection

Consider all edges connecting to slot  $s$ ; then for each element  $x_j \in X$  there is a real number weight  $w(x_j, s)$ . Then sorting  $x \in \text{cal}X$  by  $\hookrightarrow w(x, s)$  creates a total ordering, which we call  $\text{succ}_s$ . Let slot  $s'$  be  $\text{equivalent}$  to slot  $s$  if and only if  $\text{succ}_{s'} = \text{succ}_s$ . We will assume for simplicity that weights are defined such that ties do not occur.

**Settled source reading** The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE\_CLARIFICATIONS.md).

### Lean-expanded paper semantic target

type:

$\{\alpha : \text{Type } u_1\} \rightarrow \{\sigma : \text{Type } u_2\} \rightarrow (\alpha \rightarrow \sigma \rightarrow \mathbb{R}) \rightarrow \sigma \rightarrow \alpha \rightarrow \alpha \rightarrow \text{Prop}$

value:

$\text{fun } \{\alpha : \text{Type } u_1\} \{\sigma : \text{Type } u_2\} (w : \alpha \rightarrow \sigma \rightarrow \mathbb{R}) (s : \sigma) (a b : \alpha) \Rightarrow w a s < w b s$

**Recorded source-to-declaration screening Verdict:** matches

**Reason:** Compared audit/cited publication:1081-1083 with SlotBelow(w,s,a,b)=w(a,s)<w(b,s), precisely the lower-to-higher orientation of the strict order induced by real slot weights. The displayed uses pass rejected x and assigned y in that order to assert x below y, and reverse the comparison for an offered applicant strictly above the assignee. The no-ties condition is explicit where strict rejection ordering needs it. This general real-weight comparison specializes exactly to the source tie-free model under DGD26-LAP-DETERMINATE-TIEFREE-01.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## DGD26AdmissionsPredictability.LAP.Assignment.choiceRuleOfAssignment

### Paper-local declaration:

DGD26AdmissionsPredictability.LAP.Assignment.choiceRuleOfAssignment

**Source locator:** cited publication, lines 1084-1089

### Verbatim paper-source connection

Let assignment  $A$  be the set of pairs  $\{(x_{s_1}, s_1), \dots, (x_{s_q}, s_q)\}$ .

```
\begin{restatable}{lemma}{LAP Ordering}
```

```
\label{lem:laptop-ordering}
```

Let  $x_s$  denote the element of  $\text{choice}(X)$  assigned to slot  $s$ . Then for all  $x \in X$ ,  $x \in \text{choice}(X)$  if  $x \text{succ}_s x_s$ , and  $x_s \text{succ}_s x$  if  $x \not\in \text{choice}(X)$ .

```
\end{restatable}
```

**Settled source reading** The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE\_CLARIFICATIONS.md).

### Lean-expanded paper semantic target

type:

```
{α : Type u_1} →
```

```
{σ : Type u_2} →
```

```
[DecidableEq α] → [Fintype σ] → (Finset α → DGD26AdmissionsPredictability.LAP.Assignment α σ) → Finset α → Finset α
```

value:

```
fun {α : Type u_1} {σ : Type u_2} [DecidableEq α] [Fintype σ]
```

```
(select : Finset α → DGD26AdmissionsPredictability.LAP.Assignment α σ) (a : Finset α) =>
```

```
(select a).chosenSet
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.chosenSet](#)

### Recorded source-to-declaration screening Verdict: matches

**Reason:** Compared audit/cited publication:1084-1089 with choiceRuleOfAssignment and the complete Assignment/chosenSet bodies. For pool  $X$  the function returns the union, over finite slots, of the singleton applicant in each occupied slot, omitting unoccupied slots. Membership is exactly being assigned to a slot in  $\text{select}(X)$ , as in the source assignment-to-choice interpretation. The helper is general in its selector; the displayed paperLAPChoiceRule uses and their feasible/capacity-filling/unique-chosen-set operational-objective context are consistent with the bound DGD26-LAP-DETERMINATE-TIEFREE-01 convention. This assignment-to-set conversion introduces no extra restriction.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

### Reviewer annotation



**Paper-local declaration:**

DGD26AdmissionsPredictability.PaperInterface.paper\_binary\_classifier\_model\_statementSpec

**Source locator:** cited publication, lines 157-160

**Verbatim paper-source connection**

Data from admissions processes are often used for supervised machine learning as follows: each applicant is a data point and admissions decisions are  $\hookrightarrow$  labels.

Applicants are represented by the features  $x \in \text{mathcal{X}}$  on their application, and labels  $y \in \{0, 1\}$  are binary (accept or reject).

Then the relevant task is binary classification, i.e., generating a model

$\{f\} : \text{calX} \rightarrow \{0, 1\}$  using data  $X = \{x_1, x_2, \dots, x_n\}$  and  $Y = \{y_1, y_2, \dots, y_n\}$  from past admissions processes.

**Lean-expanded paper semantic target**

type:

$\{\alpha : \text{Type } u_1\} \rightarrow (\alpha \rightarrow \mathbb{N}) \rightarrow \text{Prop}$

value:

$\text{fun } \{\alpha : \text{Type } u_1\} (f : \alpha \rightarrow \mathbb{N}) \Rightarrow \forall (x : \alpha), f\ x = 0 \vee f\ x = 1$

**Recorded source-to-declaration screening Verdict:** matches

**Reason:** The pinned source defines the introductory classifier as  $f : \mathcal{X} \rightarrow \{0, 1\}$ ; the expanded Lean target exactly states that every output of  $f$  is 0 or 1. The training-data narrative is contextual and adds no premise to this codomain statement.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## DGD26AdmissionsPredictability.lapModel

### Paper-local declaration:

DGD26AdmissionsPredictability.lapModel

**Source locator:** cited publication, lines 1073-1084

### Verbatim paper-source connection

`\subsubsection{Linear Assignment Problems} \label{sec:lap-proofs}`

We highlight `\textit{linear assignment problems}` as a generalization of sequential composition.

The linear sum assignment problem can be represented as a bipartite graph matching.

One one side of the graph are applicants, and on the other are available admissions slots.

Edges connect these nodes according to how well suited each applicant is for each slot.

The admissions decision is made by selecting edges which have maximal sum, subject to the constraint that each applicant and each slot is selected only once.

Consider all edges connecting to slot  $s$ ; then for each element  $x_j$  in  $X$  there is a real number weight  $w(x_j, s)$ . Then sorting  $x$  in  $\text{cal}X$  by

$\hookrightarrow w(x, s)$  creates a total ordering, which we call  $\text{succ}_s$ . Let slot  $s$  be `\textit{equivalent}` to

to slot  $s'$  if and only if  $\text{succ}_s = \text{succ}_{s'}$ .

We will assume for simplicity that weights are defined such that ties do not occur.

Let assignment  $A$  be the set of pairs  $\{(x_{s_1}, s_1), \dots, (x_{s_q}, s_q)\}$ .

**Settled source reading** The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE\_CLARIFICATIONS.md).

### Lean-expanded paper semantic target

type:

```
{α : Type u_1} →  
{σ : Type u_2} →  
[DecidableEq σ] → [Fintype σ] → Finset α → (α → σ → ℝ) → DGD26AdmissionsPredictability.LAP.Assignment α σ → Prop
```

value:

```
fun {α : Type u_1} {σ : Type u_2} [DecidableEq σ] [Fintype σ] (X : Finset α) (w : α → σ → ℝ)  
  (A : DGD26AdmissionsPredictability.LAP.Assignment α σ) =>  
  DGD26AdmissionsPredictability.paper_definition_lap_assignment_feasible X A ∧  
  ∀ (B : DGD26AdmissionsPredictability.LAP.Assignment α σ),  
  DGD26AdmissionsPredictability.paper_definition_lap_assignment_feasible X B →  
  DGD26AdmissionsPredictability.LAP.Assignment.objective w B ≤  
  DGD26AdmissionsPredictability.LAP.Assignment.objective w A
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.LAP.Assignment](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.objective](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_lap\\_assignment\\_feasible](#)

### Recorded source-to-declaration screening Verdict: matches

**Reason:** Compared audit/cited publication:1073-1084 with lapModel and all displayed feasibility/objective bodies. Each slot contains at most one applicant; AssignedFrom restricts assignments to X and NoDuplicateApplicants prevents one applicant occupying two slots. The objective sums the real weight of each occupied slot, using zero for an empty slot. lapModel requires feasibility of A and objective(B) ≤ objective(A) for every feasible assignment B, exactly the source maximum-weight bipartite matching constraint and objective. The comparison does not restrict the competing matching family. DGD26-LAP-DETERMINATE-TIEFREE-01 is honored within its stated operational choice/tie scope.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## DGD26AdmissionsPredictability.paperRepresentsChoice

### Paper-local declaration:

DGD26AdmissionsPredictability.paperRepresentsChoice

**Source locator:** cited publication, lines 248-251

### Verbatim paper-source connection

We first relate the concepts of instability and variability to the practice of machine learning.

We focus on the representation capabilities of ML models: when is it possible for an ML model to faithfully represent an admissions process?

Formally, we say that an ML model can represent a choice function  $\mathcal{C}$  if there exists a function  $f$  such that for all applicant sets  $X$ ,  $f(x_i) = \mathcal{C}(X)_i$ .

{Note that by focusing on representation, our theory is distribution agnostic, and applies to settings where applicant distributions are affected  
considerations such as strategic behavior and matching algorithms.}

### Lean-expanded paper semantic target

type:

```
{α : Type u_1} →  
[inst : DecidableEq α] →  
DGD26AdmissionsPredictability.PaperPoolPredictor α → DGD26AdmissionsPredictability.PaperChoiceRule α → Prop
```

value:

```
fun {α : Type u_1} [DecidableEq α] (predicts : DGD26AdmissionsPredictability.PaperPoolPredictor α)  
  (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>  
  ∀ (X : Finset α), ∀ x ∈ X, predicts X x ↔ x ∈ C X
```

### Direct retained dependencies

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.PaperPoolPredictor](#)

### Recorded source-to-declaration screening Verdict: matches

**Reason:** Compared audit/cited publication:248-251 under DGD26-ML-POOL-CONTEXT-01 with the full paperRepresentsChoice, PaperPoolPredictor, PaperChoiceRule and alias bodies. For every finite pool  $X$  and offered applicant  $x$ ,  $\text{predicts}(X, x)$  iff  $x$  belongs to  $C(X)$ , exactly equality of binary admissions labels. This is the representation relation for a supplied predictor  $f$ ; existentially choosing  $f$  gives the source ability-to-represent statement. The pool argument is the expressly bound reading of ambient  $X$ . The displayed fixed- and rank-threshold uses preserve the same relation with feasibility and their score-domain assumptions visible; no distributional premise or extra offered-applicant restriction is introduced.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

### Reviewer annotation

# Source claims

## Review row 1

**Source locator:** cited publication, lines 162-170

### Verbatim source input

Training a machine learning model means selecting a  $f$  from a set of possible models, i.e., a hypothesis class. Generally, this is done by finding a model which achieves low error on available data (a train set). It is typical to use continuous optimization algorithms to first train a continuous  $\text{score}$  function  $s: \mathcal{X} \rightarrow [0,1]$  by minimizing the error. Then the classification model is defined as

$$f(x) = \mathbf{1}_{\{s(x) \geq t\}}$$

based on a fixed threshold value  $t$ , often  $t=0.5$ . Notice that models of the form  $\sim \text{eqref{eq:ml-independent}}$  predict outcomes independently for each applicant. At test time inference, they do not consider the overall applicant pool -- they implicitly model the training time pool, through its effect on the training labels.

### Expanded PaperInterface specification

$\forall \{\alpha : \text{Type } u_1\} [\text{inst} : \text{DecidableEq } \alpha] (\text{score} : \alpha \rightarrow \mathbb{R}) (\text{threshold} : \mathbb{R}),$   
 $(\forall (x : \alpha), 0 \leq \text{score } x \wedge \text{score } x \leq 1) \rightarrow$   
 $\forall (X : \text{Finset } \alpha) (x : \alpha),$   
 $x \in \text{DGD26AdmissionsPredictability.paperFixedThresholdChoice score threshold } X \leftrightarrow$   
 $x \in X \wedge \text{DGD26AdmissionsPredictability.paper\_definition\_fixed\_threshold\_predictor score threshold } X x$

### Retained governing declarations

- [DGD26AdmissionsPredictability.paperFixedThresholdChoice](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_fixed\\_threshold\\_predictor](#)

### Lean-elaborated claim roles

1. parameter:  $\text{Type } u_1$
2. parameter:  $\text{DecidableEq } \alpha$
3. parameter:  $\alpha \rightarrow \mathbb{R}$
4. parameter:  $\mathbb{R}$
5. assumption:  $\forall (x : \alpha), 0 \leq \text{score } x \wedge \text{score } x \leq 1$
6. parameter:  $\text{Finset } \alpha$
7. parameter:  $\alpha$
8. conclusion:  $x \in \text{DGD26AdmissionsPredictability.paperFixedThresholdChoice score threshold } X \leftrightarrow$   
 $x \in X \wedge \text{DGD26AdmissionsPredictability.paper\_definition\_fixed\_threshold\_predictor score threshold } X x$

### Lean proof endpoint (built separately)

`DGD26AdmissionsPredictability.PaperInterface.paper_fixed_threshold_formula_statement`

### Recorded source-to-Spec screening Verdict: matches

**Reason:** Matches: with the source [0,1] score normalization, an offered applicant is selected exactly when its fixed score meets the threshold; the pool argument only enforces offered-applicant membership.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

### Reviewer annotation

## Review row 2

**Source locator:** cited publication, lines 179-181

### Verbatim source input

Recalling admissions data, the binary admissions label  $y$  for student  $x$  is 1 if  $x \in \text{choice}(X)$  and 0 otherwise. To aid in our discussion, we will slightly abuse notation to relate pairs  $(x, y)$  to the choice function  $\text{choice}(X)$  where  $x \in X$ . Define  $\text{choice}(X)_i$  as 1 if  $x_i \in \text{choice}(X)$  and 0 otherwise; this allows us to write the label  $y_i = \text{choice}(X)_i$ .

### Expanded PaperInterface specification

$\forall \{\alpha : \text{Type } u_1\} [\text{inst} : \text{DecidableEq } \alpha] (C : \text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha) (X : \text{Finset } \alpha),$   
 $\forall x \in X, \text{DGD26AdmissionsPredictability.paperChoiceLabel } C \ X \ x = \text{if } x \in C \ X \text{ then } 1 \text{ else } 0$

### Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paperChoiceLabel](#)

### Lean-elaborated claim roles

1. parameter:  $\text{Type } u_1$
2. parameter:  $\text{DecidableEq } \alpha$
3. parameter:  $\text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha$
4. parameter:  $\text{Finset } \alpha$
5. parameter:  $\alpha$
6. assumption:  $x \in X$
7. conclusion:  $\text{DGD26AdmissionsPredictability.paperChoiceLabel } C \ X \ x = \text{if } x \in C \ X \text{ then } 1 \text{ else } 0$

### Lean proof endpoint (built separately)

$\text{DGD26AdmissionsPredictability.PaperInterface.paper\_definition\_choice\_label}$

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Matches: for an offered applicant, the target gives label one exactly when the applicant is chosen and zero otherwise.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## Review row 3

**Source locator:** cited publication, lines 253-259

### Verbatim source input

Before stating this result, consider a straightforward way to adapt a model~\eqref{eq:ml-independent} to the capacity-constrained setting. Given that the ML model is based on continuous scores, we can adjust the decisions to account for a known capacity constraint if we have available the set  $\hookrightarrow$  of all test-time applications.

In particular, we can rank the applicants by their scores and then select the top  $q$ .

This process results in decisions determined by a cohort dependent threshold  $t_q(X)$  which is the score of the  $q$ th ranked applicant:

$$\begin{aligned} \text{\label{eq:ml-rank}} \\ f(x; X) = \mathbf{1}\{s(x) \geq t_q(X)\}. \end{aligned}$$

**Settled source reading** The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE\_CLARIFICATIONS.md).

### Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (q : ℕ) (operationalScore : α → ℝ)
(hinjective : Function.Injective operationalScore),
0 < q →
  ∀ (X : Finset α),
  q ≤ X.card →
    ∃ threshold ∈ Finset.image operationalScore X,
      {x ∈ X | threshold ≤ operationalScore x}.card = q ∧
        ∀ x ∈ X,
          DGD26AdmissionsPredictability.paper_definition_rank_threshold_predictor q operationalScore hinjective X
            x ↔
              threshold ≤ operationalScore x
```

### Retained governing declarations

- [DGD26AdmissionsPredictability.paper\\_definition\\_rank\\_threshold\\_predictor](#)

### Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: DecidableEq α
3. parameter: ℕ
4. parameter: α → ℝ
5. assumption: Function.Injective operationalScore
6. assumption: 0 < q
7. parameter: Finset α
8. assumption: q ≤ X.card
9. conclusion: ∃ threshold ∈ Finset.image operationalScore X,
  {x ∈ X | threshold ≤ operationalScore x}.card = q ∧
    ∀ x ∈ X,
      DGD26AdmissionsPredictability.paper_definition_rank_threshold_predictor q operationalScore hinjective X x ↔
        threshold ≤ operationalScore x
```

### Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_rank\_threshold\_formula\_statement

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Matches: for positive  $q$  and a pool of at least  $q$  applicants, the target gives the  $q$ -th-score threshold formula and exactly  $q$  selected applicants. The queue and Spec explicitly limit injective operationalScore to a fixed tie-break, not a raw-score uniqueness assumption.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## Review row 4

**Source locator:** cited publication, lines 704-707

### Verbatim source input

```
\begin{restatable}{corollary}{Only Independent Rules are 0-unstable}
\label{thm:independent-zero-unstable}
$\text{choice}$ is 0-unstable if and only if it is independent, and therefore no $\text{q}$-acceptant $\text{choice}$ can be 0-unstable.
\end{restatable}
```

### Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α),
  DGD26AdmissionsPredictability.paper_choice_function_feasible C →
  (DGD26AdmissionsPredictability.paper_definition_independence C ↔
   DGD26AdmissionsPredictability.paper_definition_zero_instability C)
```

### Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_independence](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_zero\\_instability](#)

### Lean-elaborated claim roles

1. parameter: Type u\_1
2. parameter: DecidableEq α
3. parameter: DGD26AdmissionsPredictability.PaperChoiceRule α
4. assumption: DGD26AdmissionsPredictability.paper\_choice\_function\_feasible C
5. conclusion: DGD26AdmissionsPredictability.paper\_definition\_independence C ↔ DGD26AdmissionsPredictability.paper\_definition\_zero\_instability C

### Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_independent\_zero\_unstable\_statement

### Recorded source-to-Spec screening Verdict: matches

**Reason:** Matches: the target is the source corollary's equivalence between independence and 0-instability for a feasible choice rule.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

### Reviewer annotation

## Review row 5

**Source locator:** cited publication, lines 704-707

### Verbatim source input

```
\begin{restatable}{\corollary}{Only Independent Rules are 0-unstable}
\label{thm:independent-zero-unstable}
$\choice$ is $0$-unstable if and only if it is independent, and therefore no $q$-acceptant $\choice$ can be $0$-unstable.
\end{restatable}
```

**Formalized review target** The archival source above is not asserted equivalent to this formalized target.

For a feasible  $q$ -acceptant choice rule  $C$  on a universe with an input pool strictly larger than positive capacity  $q$ ,  $C$  is 0-unstable if and only if it is  
 $\hookrightarrow$  independent, and  $C$  is not 0-unstable.

**Archival source anchor:** cited publication, lines 704-707

### Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {q : ℕ} {C : DGD26AdmissionsPredictability.PaperChoiceRule α},
  DGD26AdmissionsPredictability.paper_choice_function_feasible C →
  DGD26AdmissionsPredictability.paper_definition_q_acceptance q C →
  0 < q →
  ∀ {U : Finset α},
    q < U.card →
    (DGD26AdmissionsPredictability.paper_definition_zero_instability C ↔
     DGD26AdmissionsPredictability.paper_definition_independence C) ∧
    ¬DGD26AdmissionsPredictability.paper_definition_zero_instability C
```

### Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_independence](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_acceptance](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_zero\\_instability](#)

### Lean-elaborated claim roles

1. parameter: Type u\_1
2. parameter: DecidableEq α
3. parameter: ℕ
4. parameter: DGD26AdmissionsPredictability.PaperChoiceRule α
5. assumption: DGD26AdmissionsPredictability.paper\_choice\_function\_feasible C
6. assumption: DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance q C
7. assumption: 0 < q
8. parameter: Finset α
9. assumption: q < U.card
10. conclusion: (DGD26AdmissionsPredictability.paper\_definition\_zero\_instability C ↔  
DGD26AdmissionsPredictability.paper\_definition\_independence C) ∧  
¬DGD26AdmissionsPredictability.paper\_definition\_zero\_instability C

### Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_independent\_zero\_unstable\_corollary\_statement

**Recorded source-to-Spec screening Verdict:** matches\_approved\_corrected\_target

**Reason:** Matches the approved corrected target: positive capacity and a pool larger than capacity make the source corollary's no-zero clause nonvacuous, and the Spec states both its independence equivalence and nonzero conclusion.

**Reviewer check:** ☐ Matches formalized target

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**



## Review row 6

**Source locator:** cited publication, lines 796-801

### Verbatim source input

```
\begin{restatable}{\corollary}{No Consistent Tightly Even Instability}
\label{cor:even-inconsistency}
There is no  $q$ -acceptant and consistent  $\text{\choice}$  that is tightly  $dk$ -unstable for even values of  $d$ .
\end{restatable}
```

This follows directly from  $\text{\Cref{thm:even-inconsistency}}$ .

**Formalized review target** The archival source above is not asserted equivalent to this formalized target.

For every positive integer  $k$ , no  $q$ -acceptant consistent choice rule is tightly  $2k$ -unstable.

**Archival source anchor:** cited publication, lines 796-801

### Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {q k : ℕ} {C : DGD26AdmissionsPredictability.PaperChoiceRule α},
0 < k →
  DGD26AdmissionsPredictability.paper_choice_function_feasible C →
  DGD26AdmissionsPredictability.paper_definition_q_acceptance q C →
  DGD26AdmissionsPredictability.paper_definition_consistency C →
  ¬DGD26AdmissionsPredictability.paper_definition_tight_d_instability (2 * k) C
```

### Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_consistency](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_acceptance](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_tight\\_d\\_instability](#)

### Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: DecidableEq α
3. parameter: ℕ
4. parameter: ℕ
5. parameter: DGD26AdmissionsPredictability.PaperChoiceRule α
6. assumption: 0 < k
7. assumption: DGD26AdmissionsPredictability.paper_choice_function_feasible C
8. assumption: DGD26AdmissionsPredictability.paper_definition_q_acceptance q C
9. assumption: DGD26AdmissionsPredictability.paper_definition_consistency C
10. assumption: DGD26AdmissionsPredictability.paper_definition_tight_d_instability (2 * k) C
11. conclusion: False
```

### Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_no\_consistent\_positive\_tightly\_even\_statement

**Recorded source-to-Spec screening Verdict:** matches approved corrected target

**Reason:** Matches the approved corrected target: the undefined printed  $dk$  is documented as the positive even parameter  $2k$ , and the Spec states precisely that no feasible  $q$ -acceptant consistent rule is tightly  $2k$ -unstable.

**Reviewer check:** ☐ Matches formalized target

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 7

**Source locator:** cited publication, lines 661-667

## Verbatim source input

```
\begin{restatable}{corollary}{Zero-Distance}
\label{thm:sub-mon-zero}
Choice function  $\text{\textit{choice}}$  is 0-unstable if and only if it is both substitutable and monotonic.
\end{restatable}
```

This immediately follows from the two terms comprising  $\text{\textit{r\_choice}}$  mapping onto substitutability and monotonicity.

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α),
  DGD26AdmissionsPredictability.paper_definition_zero_instability C ↔
  DGD26AdmissionsPredictability.paper_definition_substitutability C ∧
  DGD26AdmissionsPredictability.paper_definition_monotonicity C
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_monotonicity](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_substitutability](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_zero\\_instability](#)

## Lean-elaborated claim roles

- 1. parameter: Type u\_1
- 2. parameter: DecidableEq α
- 3. parameter: DGD26AdmissionsPredictability.PaperChoiceRule α
- 4. conclusion: DGD26AdmissionsPredictability.paper\_definition\_zero\_instability C ↔ DGD26AdmissionsPredictability.paper\_definition\_substitutability C ∧ DGD26AdmissionsPredictability.paper\_definition\_monotonicity C

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_zero\_distance\_statement

## Recorded source-to-Spec screening Verdict: matches

**Reason:** Matches: the target is exactly the source equivalence between 0-instability and the conjunction of substitutability and monotonicity.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

## Reviewer annotation

# Review row 8

**Source locator:** cited publication, lines 746-765

## Verbatim source input

```
\begin{restatable}[Calculating Instability]{lemma}{stablecalc}
\label{lem:unstable-calc}
For all  $q$ -acceptant  $\mathcal{X}$ , let  $\mathcal{X}' = \mathcal{X} \cup \{x'\}$  for  $|X| \geq q$  and  $n = |\mathcal{X}| \setminus |\mathcal{X}'|$ . Then
 $r_{\mathcal{X}}(\mathcal{X}, \mathcal{X}') = 2n - \mathbf{1}_{x' \in \mathcal{X}'}$ .
\end{restatable}

\begin{proof}
\textbf{Informal}
If input sets differ by one element, but the selected sets differ by  $n$  elements, then it must be that  $n$  previously accepted elements were rejected and
 $\hookrightarrow$  there must be  $n$  new accepted elements, at least  $n-1$  must have been previously rejected, because the size of the accepted set is fixed.

\textbf{Formal}
By  $q$ -acceptance,  $|\mathcal{X}| = |\mathcal{X}'| = q$ . Then  $|\mathcal{X}| \setminus |\mathcal{X}'| = |\mathcal{X}'| \setminus |\mathcal{X}| = q - |\mathcal{X}| \cap |\mathcal{X}'|$ .
 $\hookrightarrow |\mathcal{X}'| = q$ .
Second, note that  $|\mathcal{X}'| \setminus |\mathcal{X}| = (|\mathcal{X}'| \cap \mathcal{X}) \setminus |\mathcal{X}|$  if  $x' \notin \mathcal{X}'$ , and otherwise  $|\mathcal{X}'| \setminus |\mathcal{X}| = \{x'\} \cup \left( (|\mathcal{X}'| \cap \mathcal{X}) \setminus |\mathcal{X}| \right)$ .
 $\hookrightarrow |\mathcal{X}'| \setminus |\mathcal{X}| = \{x'\} \cup \left( (|\mathcal{X}'| \cap \mathcal{X}) \setminus |\mathcal{X}| \right)$ .
Putting the previous statements together, we get two cases:
\begin{itemize}
\item  $x' \notin \mathcal{X}'$ :  $|\mathcal{X}| \setminus |\mathcal{X}'| = |\mathcal{X}'| \setminus |\mathcal{X}| = |\mathcal{X}| \cap |\mathcal{X}'| \setminus |\mathcal{X}| = |\mathcal{X}| \cap |\mathcal{X}'| \setminus |\mathcal{X}| = 0$ 
\item  $x' \in \mathcal{X}'$ :  $|\mathcal{X}| \setminus |\mathcal{X}'| = |\mathcal{X}'| \setminus |\mathcal{X}| = |\mathcal{X}| \cap |\mathcal{X}'| \setminus |\mathcal{X}| = |\mathcal{X}| \cap |\mathcal{X}'| \setminus |\mathcal{X}| + 1$ 
\end{itemize}
Then  $r_{\mathcal{X}}(\mathcal{X}, \mathcal{X}') = |\mathcal{X}| \cap |\mathcal{X}'| \setminus |\mathcal{X}| + |\mathcal{X}'| \setminus |\mathcal{X}| = 2|\mathcal{X}| \cap |\mathcal{X}'| \setminus |\mathcal{X}| + \mathbf{1}_{x' \in \mathcal{X}'}$ .
 $\hookrightarrow \mathbf{1}_{x' \in \mathcal{X}'} = 2|\mathcal{X}| \cap |\mathcal{X}'| \setminus |\mathcal{X}| - \mathbf{1}_{x' \in \mathcal{X}'}$ 
\end{proof}
```

## Expanded PaperInterface specification

```

 $\forall \alpha : \text{Type } u_1 \{ \text{inst} : \text{DecidableEq } \alpha \} \{ q : \mathbb{N} \} \{ C : \text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha \},$ 
DGD26AdmissionsPredictability.paper_choice_function_feasible C  $\rightarrow$ 
DGD26AdmissionsPredictability.paper_definition_q_acceptance q C  $\rightarrow$ 
 $\forall \{ X : \text{Finset } \alpha \} \{ x : \alpha \},$ 
q  $\leq$  X.card  $\rightarrow$ 
x  $\notin$  X  $\rightarrow$ 
DGD26AdmissionsPredictability.paper_definition_choice_distance C X (insert x X) =
if x  $\in$  C (insert x X) then 2 * (C X \ C (insert x X)).card - 1 else 2 * (C X \ C (insert x X)).card

```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_choice\\_distance](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_acceptance](#)

## Lean-elaborated claim roles

1. parameter: Type  $u_1$
2. parameter: DecidableEq  $\alpha$
3. parameter:  $\mathbb{N}$
4. parameter: DGD26AdmissionsPredictability.PaperChoiceRule  $\alpha$
5. assumption: DGD26AdmissionsPredictability.paper\_choice\_function\_feasible C
6. assumption: DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance q C
7. parameter: Finset  $\alpha$
8. parameter:  $\alpha$
9. assumption: q  $\leq$  X.card
10. assumption: x  $\notin$  X
11. conclusion: DGD26AdmissionsPredictability.paper\_definition\_choice\_distance C X (insert x X) = if x  $\in$  C (insert x X) then 2 \* (C X \ C (insert x X)).card - 1 else 2 \* (C X \ C (insert x X)).card

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_calculating\_instability\_statement

## Recorded source-to-Spec screening Verdict: matches

**Reason:** Matches: for a fresh insertion into a pool of at least  $q$  applicants, the conditional Nat formula is precisely 2 times the number of lost accepts minus the indicator that the new applicant is chosen.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

## Reviewer annotation

# Review row 9

**Source locator:** cited publication, lines 885-896

## Verbatim source input

```
\begin{restatable}[Consistency of Removable Sets]{lemma}{consistentremove}
\label{lem:consistent-remove}
Let  $\text{\textit{choice}}$  be 1-unstable and  $q$ -acceptant with variability  $\text{\textit{sm}}$ . For any  $X_1, X_2$ , if  $\text{\textit{choice}}(X_1) = \text{\textit{choice}}(X_2)$ , then  $V_{\text{\textit{choice}}(X_1)} = V_{\text{\textit{choice}}(X_2)}$ .
\end{restatable}

\begin{proof}
Recall from \Cref{thm:sub-accept-consistent} that  $\text{\textit{choice}}$  must be consistent and so  $\text{\textit{choice}}(\text{\textit{choice}}(X)) = \text{\textit{choice}}(X)$  for any  $X$ .

Consider any  $x'$  associated with an element  $x^*$  of  $V_{\text{\textit{choice}}(X)}$ , such that  $\text{\textit{choice}}(X) \setminus \text{\textit{choice}}(x') = \{x^*\}$  (and even more precisely, by  $q$ -acceptant substitutability, that  $\text{\textit{choice}}(x') = \{x^*\} \cup \text{\textit{choice}}(X) \setminus \text{\textit{choice}}(x^*)$  exactly). Note that  $\text{\textit{choice}}(x') \subseteq \text{\textit{choice}}(X)$  and therefore  $\text{\textit{choice}}(\text{\textit{choice}}(X) \cup \{x^*\}) = \text{\textit{choice}}(x')$  by consistency. This applies to all possible  $x'$  (including when  $x' \in X$ ), and therefore,  $V_{\text{\textit{choice}}(X)} = V_{\text{\textit{choice}}(\text{\textit{choice}}(X))}$ .

As this set equality is exact, apply this logic to some  $X_2$  and see  $V_{\text{\textit{choice}}(X_1)} = V_{\text{\textit{choice}}(\text{\textit{choice}}(X_1))} = V_{\text{\textit{choice}}(\text{\textit{choice}}(X_2))} = V_{\text{\textit{choice}}(X_2)}$ .
\end{proof}
```

**Formalized review target** The archival source above is not asserted equivalent to this formalized target.

For a feasible  $q$ -acceptant substitutable choice rule  $C$ , if  $C(X_1)=C(X_2)$ , then the borderline sets  $V_C(X_1)$  and  $V_C(X_2)$  are equal.

**Archival source anchor:** cited publication, lines 885-896

## Expanded PaperInterface specification

```

 $\forall \{ \alpha : \text{Type } u_1 \} [\text{inst} : \text{DecidableEq } \alpha] [\text{inst}_1 : \text{Fintype } \alpha] \{ q : \mathbb{N} \}$ 
{ C : DGD26AdmissionsPredictability.PaperChoiceRule  $\alpha$  },
DGD26AdmissionsPredictability.paper_choice_function_feasible C  $\rightarrow$ 
DGD26AdmissionsPredictability.paper_definition_q_acceptance q C  $\rightarrow$ 
DGD26AdmissionsPredictability.paper_definition_substitutability C  $\rightarrow$ 
 $\forall \{ X_1 X_2 : \text{Finset } \alpha \},$ 
C  $X_1 = C X_2 \rightarrow$ 
DGD26AdmissionsPredictability.paper_definition_borderline_set C  $X_1 =$ 
DGD26AdmissionsPredictability.paper_definition_borderline_set C  $X_2$ 
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_borderline\\_set](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_acceptance](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_substitutability](#)

## Lean-elaborated claim roles

1. parameter:  $\text{Type } u_1$
2. parameter:  $\text{DecidableEq } \alpha$
3. parameter:  $\text{Fintype } \alpha$
4. parameter:  $\mathbb{N}$
5. parameter:  $\text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha$
6. assumption:  $\text{DGD26AdmissionsPredictability.paper\_choice\_function\_feasible } C$
7. assumption:  $\text{DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance } q \ C$
8. assumption:  $\text{DGD26AdmissionsPredictability.paper\_definition\_substitutability } C$
9. parameter:  $\text{Finset } \alpha$
10. parameter:  $\text{Finset } \alpha$
11. assumption:  $C \ X_1 = C \ X_2$
12. conclusion:  $\text{DGD26AdmissionsPredictability.paper\_definition\_borderline\_set } C \ X_1 = \text{DGD26AdmissionsPredictability.paper\_definition\_borderline\_set } C \ X_2$

## Lean proof endpoint (built separately)

$\text{DGD26AdmissionsPredictability.PaperInterface.paper\_corrected\_consistency\_of\_removable\_sets\_statement}$

**Recorded source-to-Spec screening Verdict:** matches\_approved\_corrected\_target

**Reason:** Matches the approved corrected target: the archived self-equality is documented as a defect, and the approved feasible  $q$ -acceptant substitutable cross-pool borderline-set equality is exactly the Spec.

**Reviewer check:** ☐ Matches formalized target

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 10

**Source locator:** cited publication, lines 1086-1095

## Verbatim source input

```
\begin{restatable}{lemma}{LAP Ordering}
\label{lem:lap-ordering}
Let  $x_s$  denote the element of  $\text{choice}(X)$  assigned to slot  $s$ . Then for all  $x \in X$ ,  $x \in \text{choice}(X)$  if  $x \text{succ}_s x_s$ , and  $x_s \text{succ}_s x$  if  $x \not\in \text{choice}(X)$ .
\end{restatable}

\textit{Intuition}: Linear assignment problems have a structure that resembles a combination of rankings, even though there is not a fixed sequential order.

\begin{proof}
This comes by the optimality of the assignment solution. For contradiction, say there is  $x^* \text{succ}_s x_s$  where  $x^* \not\in \text{choice}(X)$ . Then  $w(x^*, s) > w(x_s, s)$ , and therefore  $\{x^*, s\} \cup A \setminus \{x_s, s\}$  leads to a higher sum than  $\text{choice}(X)$ , contradicting the fact that  $A$  is optimal.
\end{proof}
```

**Settled source reading** The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE\_CLARIFICATIONS.md).

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {σ : Type u_2} [inst_1 : DecidableEq σ] [inst_2 : Fintype σ] {X : Finset α}
{w : α → σ → ℝ} {A : DGD26AdmissionsPredictability.LAP.Assignment α σ},
DGD26AdmissionsPredictability.paper_definition_lap_objective_optimal X w A →
DGD26AdmissionsPredictability.paper_definition_lap_capacity_filling X A →
DGD26AdmissionsPredictability.paper_definition_lap_assignment_feasible X A →
  ∀ {s : σ} {y x : α},
    A.matchSlot s = some y →
      x ∈ X →
        DGD26AdmissionsPredictability.paper_definition_lap_slot_below w s y x →
          DGD26AdmissionsPredictability.paper_definition_lap_assigned A x
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.LAP.Assignment](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_lap\\_assigned](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_lap\\_assignment\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_lap\\_capacity\\_filling](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_lap\\_objective\\_optimal](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_lap\\_slot\\_below](#)

## Lean-elaborated claim roles

1. parameter: Type u\_1
2. parameter: DecidableEq α
3. parameter: Type u\_2
4. parameter: DecidableEq σ
5. parameter: Fintype σ
6. parameter: Finset α
7. parameter: α → σ → ℝ
8. parameter: DGD26AdmissionsPredictability.LAP.Assignment α σ
9. assumption: DGD26AdmissionsPredictability.paper\_definition\_lap\_objective\_optimal X w A
10. assumption: DGD26AdmissionsPredictability.paper\_definition\_lap\_capacity\_filling X A
11. assumption: DGD26AdmissionsPredictability.paper\_definition\_lap\_assignment\_feasible X A
12. parameter: σ
13. parameter: α
14. parameter: α
15. assumption: A.matchSlot s = some y
16. assumption: x ∈ X
17. assumption: DGD26AdmissionsPredictability.paper\_definition\_lap\_slot\_below w s y x
18. conclusion: ∃ (s : σ), A.matchSlot s = some x

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_lap\_strictly\_higher\_slot\_applicant\_assigned\_statement

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Independent outcome-blind review: the Spec is the first direction of the source LAP Ordering lemma, with matching, optimality, capacity-filling, and no-ties premises explicit.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

|  |
|--|
|  |
|--|

# Review row 11

**Source locator:** cited publication, lines 1086-1095

## Verbatim source input

```
\begin{restatable}{lemma}{LAP Ordering}
\label{lem:lap-ordering}
Let  $x_s$  denote the element of  $\text{choice}(X)$  assigned to slot  $s$ . Then for all  $x \in X$ ,  $x \in \text{choice}(X)$  if  $x \text{succ}_s x_s$ , and  $x_s \text{succ}_s x$  if  $x \not\in \text{choice}(X)$ .
\end{restatable}

\textit{Intuition}: Linear assignment problems have a structure that resembles a combination of rankings, even though there is not a fixed sequential order.

\begin{proof}
This comes by the optimality of the assignment solution. For contradiction, say there is  $x^* \text{succ}_s x_s$  where  $x^* \not\in \text{choice}(X)$ . Then  $w(x^*, s) > w(x_s, s)$ , and therefore  $\{x^*, s\} \cup A \setminus \{x_s, s\}$  leads to a higher sum than  $\text{choice}(X)$ , contradicting the fact that  $A$  is optimal.
\end{proof}
```

**Settled source reading** The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE\_CLARIFICATIONS.md).

## Expanded PaperInterface specification

```

 $\forall \{ \alpha : \text{Type } u_1 \} [ \text{inst} : \text{DecidableEq } \alpha ] \{ \sigma : \text{Type } u_2 \} [ \text{inst}_1 : \text{DecidableEq } \sigma ] [ \text{inst}_2 : \text{Fintype } \sigma ] \{ X : \text{Finset } \alpha \}$ 
 $\{ w : \alpha \rightarrow \sigma \rightarrow \mathbb{R} \} \{ A : \text{DGD26AdmissionsPredictability.LAP.Assignment } \alpha \sigma \},$ 
DGD26AdmissionsPredictability.paper_definition_lap_objective_optimal  $X w A \rightarrow$ 
DGD26AdmissionsPredictability.paper_definition_lap_capacity_filling  $X A \rightarrow$ 
DGD26AdmissionsPredictability.paper_definition_lap_assignment_feasible  $X A \rightarrow$ 
 $\forall \{ s : \sigma \} \{ y x : \alpha \},$ 
A.matchSlot  $s = \text{some } y \rightarrow$ 
DGD26AdmissionsPredictability.LAP.Assignment.Rejected  $X A x \rightarrow$ 
DGD26AdmissionsPredictability.LAP.Assignment.SlotNoTies  $w s \rightarrow$ 
DGD26AdmissionsPredictability.paper_definition_lap_slot_below  $w s x y$ 
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.LAP.Assignment](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.Rejected](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.SlotNoTies](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_lap\\_assignment\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_lap\\_capacity\\_filling](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_lap\\_objective\\_optimal](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_lap\\_slot\\_below](#)

## Lean-elaborated claim roles

1. parameter:  $\text{Type } u_1$
2. parameter:  $\text{DecidableEq } \alpha$
3. parameter:  $\text{Type } u_2$
4. parameter:  $\text{DecidableEq } \sigma$
5. parameter:  $\text{Fintype } \sigma$
6. parameter:  $\text{Finset } \alpha$
7. parameter:  $\alpha \rightarrow \sigma \rightarrow \mathbb{R}$
8. parameter:  $\text{DGD26AdmissionsPredictability.LAP.Assignment } \alpha \sigma$
9. assumption:  $\text{DGD26AdmissionsPredictability.paper_definition_lap_objective_optimal } X w A$
10. assumption:  $\text{DGD26AdmissionsPredictability.paper_definition_lap_capacity_filling } X A$
11. assumption:  $\text{DGD26AdmissionsPredictability.paper_definition_lap_assignment_feasible } X A$
12. parameter:  $\sigma$
13. parameter:  $\alpha$
14. parameter:  $\alpha$
15. assumption:  $\text{A.matchSlot } s = \text{some } y$
16. assumption:  $\text{DGD26AdmissionsPredictability.LAP.Assignment.Rejected } X A x$
17. assumption:  $\text{DGD26AdmissionsPredictability.LAP.Assignment.SlotNoTies } w s$
18. conclusion:  $\text{Real.lt } (w x s) (w y s)$

## Lean proof endpoint (built separately)

`DGD26AdmissionsPredictability.PaperInterface.paper_lap_assigned_strictly_outranks_rejected_statement`

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Independent outcome-blind review: the Spec is the LAP Ordering lemma's rejected-applicant conclusion, with the paper's matching, capacity-filling, optimality, and no-ties conditions explicit.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

|  |
|--|
|  |
|--|



# Review row 12

**Source locator:** cited publication, lines 564-573

## Verbatim source input

```
\begin{restatable}[Monotonicity Term]{lemma}{mono-dist}
\label{lem:monotonicity-term}
Choice function  $\text{choice}$  is monotonic if and only if  $|\text{choice}(X_1) \setminus \text{choice}(X_2)| = 0$  for all  $X_1 \subseteq X_2$ .
\end{restatable}

\textit{Intuition}:
For a monotonic choice function, previously accepted candidates are never rejected after the addition of more applicants. Thus labels cannot flip from
 $\hookrightarrow$  positive to negative, which is what the second term counts.

\begin{proof}
Use the same set properties as per \Cref{lem:substitutability-term},  $|\text{choice}(X_1) \setminus \text{choice}(X_2)| = 0$  if and only if  $\text{choice}(X_1) \subseteq \text{choice}(X_2)$ , which is the definition of monotonicity.
\end{proof}
```

## Expanded PaperInterface specification

$\forall \{\alpha : \text{Type } u_1\} [\text{inst} : \text{DecidableEq } \alpha] (C : \text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha),$   
 $\text{DGD26AdmissionsPredictability.paper\_definition\_monotonicity } C \leftrightarrow \forall \{X_1 X_2 : \text{Finset } \alpha\}, X_1 \subseteq X_2 \rightarrow (C X_1 \setminus C X_2).card = 0$

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_monotonicity](#)

## Lean-elaborated claim roles

1. parameter:  $\text{Type } u_1$
2. parameter:  $\text{DecidableEq } \alpha$
3. parameter:  $\text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha$
4. conclusion:  $\text{DGD26AdmissionsPredictability.paper\_definition\_monotonicity } C \leftrightarrow \forall \{X_1 X_2 : \text{Finset } \alpha\}, X_1 \subseteq X_2 \rightarrow (C X_1 \setminus C X_2).card = 0$

## Lean proof endpoint (built separately)

$\text{DGD26AdmissionsPredictability.PaperInterface.paper\_monotonicity\_term\_statement}$

## Recorded source-to-Spec screening Verdict: matches

**Reason:** Matches: the target is exactly the source equivalence between monotonicity and a zero second choice-distance term on all nested pools.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

## Reviewer annotation

# Review row 13

**Source locator:** cited publication, lines 517-529

## Verbatim source input

```
\begin{restatable}[Incompatibility of Monotonicity and  $q$ -Acceptance]{lemma}{incompatibility}
\label{thm:mono-accept-mutex}
For  $|\mathcal{X}| > q \geq 1$ , no choice function can be both monotonic and  $q$ -acceptant.
\end{restatable}

\textit{Intuition}: %
For a choice function to be monotonic, it must never reject a previously accepted applicant as the applicant pool grows. This implies that the capacity
 $\hookrightarrow$  must be unbounded, which contradicts the strict capacity limit imposed by  $q$ -acceptance.%

\begin{proof}
Consider two different sets each of  $q$  elements. If  $\text{choice}$  is  $q$ -acceptant, every element must be chosen in each. Then consider their union. If
 $\hookrightarrow$   $\text{choice}$  is monotonic, every element in their union would also be accepted, but that would contradict the  $q$ -acceptant capacity constraint.

Consider two disjoint sets  $X_1, X_2 \subseteq \mathcal{X}$  with  $|X_1| = |X_2| = q$ . By  $q$ -acceptance,  $\text{choice}(X_1) = X_1$  and  $\text{choice}(X_2) = X_2$ . By
 $\hookrightarrow$  monotonicity, both  $\text{choice}(X_1)$  and  $\text{choice}(X_2)$  must be subsets of  $\text{choice}(X_1 \cup X_2)$ . But then  $|\text{choice}(X_1 \cup X_2)| \geq |X_1| +$ 
 $\hookrightarrow |X_2| = 2q > q$ , violating  $q$ -acceptance.
\end{proof}
```

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {q : ℕ} {C : DGD26AdmissionsPredictability.PaperChoiceRule α},
DGD26AdmissionsPredictability.paper_choice_function_feasible C →
DGD26AdmissionsPredictability.paper_definition_monotonicity C →
DGD26AdmissionsPredictability.paper_definition_q_acceptance q C → 0 < q → ∀ {U : Finset α}, q < U.card → False
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_monotonicity](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_acceptance](#)

## Lean-elaborated claim roles

1. parameter: Type u\_1
2. parameter: DecidableEq α
3. parameter: ℕ
4. parameter: DGD26AdmissionsPredictability.PaperChoiceRule α
5. assumption: DGD26AdmissionsPredictability.paper\_choice\_function\_feasible C
6. assumption: DGD26AdmissionsPredictability.paper\_definition\_monotonicity C
7. assumption: DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance q C
8. assumption: 0 < q
9. parameter: Finset α
10. assumption: q < U.card
11. conclusion: False

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_monotonicity\_q\_acceptance\_incompatible\_statement

## Recorded source-to-Spec screening Verdict: matches

**Reason:** Matches: positive  $q$ -acceptance and monotonicity are inconsistent whenever an offered pool has more than  $q$  applicants, the finite-pool form of the source condition  $|\mathcal{X}| > q \geq 1$ .

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

## Reviewer annotation

# Review row 14

**Source locator:** cited publication, lines 898-905

## Verbatim source input

```
\begin{restatable}{lemma}{Ranking m}
\label{lem:a-v-equal}
Let  $q$ -acceptant,  $1$ -unstable  $\text{\textit{choice}}$  have variability  $1$ . Then  $V_{\text{\textit{choice}}(X)} = A_{\text{\textit{choice}}(X \cup \{x\})}$  when  $\text{\textit{choice}}(X \cup \{x\}) \neq \text{\textit{choice}}(X)$ .
 $\hookrightarrow \text{\textit{choice}}(X)$ .
\end{restatable}

\begin{proof}
If  $\text{\textit{choice}}(X \cup \{x\}) \neq \text{\textit{choice}}(X)$ , then  $\text{\textit{choice}}(X \cup \{x\}) = \{x\} \cup \text{\textit{choice}}(X) \setminus x^*$  where  $V_{\text{\textit{choice}}(X)} = \{x^*\}$  (by  $\hookrightarrow$  consistency, 1-instability, and the definition of variability). Then  $X' \setminus \{x\} = X$  and so  $A_{\text{\textit{choice}}(X')} = \{x^*\}$ , completing the proof.
\end{proof}
```

## Expanded PaperInterface specification

```

 $\forall \{ \alpha : \text{Type } u_1 \} [\text{inst} : \text{DecidableEq } \alpha] [\text{inst}_1 : \text{Fintype } \alpha] \{ q : \mathbb{N} \}$ 
 $\{ C : \text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha \},$ 
 $\text{DGD26AdmissionsPredictability.paper\_choice\_function\_feasible } C \rightarrow$ 
 $\text{DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance } q \ C \rightarrow$ 
 $\text{DGD26AdmissionsPredictability.paper\_definition\_d\_instability } 1 \ C \rightarrow$ 
 $\text{DGD26AdmissionsPredictability.paper\_definition\_variability\_exactly } 1 \ C \rightarrow$ 
 $\forall \{ X : \text{Finset } \alpha \} \{ x : \alpha \},$ 
 $x \notin X \rightarrow$ 
 $C (\text{insert } x \ X) \neq C \ X \rightarrow$ 
 $\text{DGD26AdmissionsPredictability.paper\_definition\_borderline\_set } C \ X =$ 
 $\text{DGD26AdmissionsPredictability.paper\_definition\_waitlisted\_set } C (\text{insert } x \ X)$ 
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_borderline\\_set](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_d\\_instability](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_acceptance](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_variability\\_exactly](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_waitlisted\\_set](#)

## Lean-elaborated claim roles

1. parameter:  $\text{Type } u_1$
2. parameter:  $\text{DecidableEq } \alpha$
3. parameter:  $\text{Fintype } \alpha$
4. parameter:  $\mathbb{N}$
5. parameter:  $\text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha$
6. assumption:  $\text{DGD26AdmissionsPredictability.paper\_choice\_function\_feasible } C$
7. assumption:  $\text{DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance } q \ C$
8. assumption:  $\text{DGD26AdmissionsPredictability.paper\_definition\_d\_instability } 1 \ C$
9. assumption:  $\text{DGD26AdmissionsPredictability.paper\_definition\_variability\_exactly } 1 \ C$
10. parameter:  $\text{Finset } \alpha$
11. parameter:  $\alpha$
12. assumption:  $x \notin X$
13. assumption:  $C (\text{insert } x \ X) \neq C \ X$
14. conclusion:  $\text{DGD26AdmissionsPredictability.paper\_definition\_borderline\_set } C \ X = \text{DGD26AdmissionsPredictability.paper\_definition\_waitlisted\_set } C (\text{insert } x \ X)$

## Lean proof endpoint (built separately)

$\text{DGD26AdmissionsPredictability.PaperInterface.paper\_acceptant\_one\_instability\_variability\_borderline\_eq\_waitlisted\_after\_changing\_insert\_statement}$

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Matches: under  $q$ -acceptance, 1-instability, exact variability one, and a changing fresh insertion, the target is exactly the source lemma's equality of the borderline set with the enlarged-pool waitlisted set.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 15

**Source locator:** cited publication, lines 552-562

## Verbatim source input

```
\begin{restatable}[Substitutability Term]{lemma}{sub-dist}
\label{lem:substitutability-term}
Choice function  $\text{\choice}$  is substitutable if and only if  $|X_1 \cap \text{\choice}(X_2) \setminus \text{\choice}(X_1)| = 0$  for all  $X_1 \subseteq X_2$ .
\end{restatable}

\textit{Intuition}:
For a substitutable choice function, previously rejected candidates are never accepted after the addition of more applicants. Thus labels cannot flip from
 $\hookrightarrow$  negative to positive, which is what the first term counts.

\begin{proof}
For any sets  $A, B$ ,  $|A \setminus B| = 0$  if and only if  $A \setminus B = \emptyset$ , and furthermore  $A \setminus B = \emptyset$  if and only if  $A \subseteq B$ . So  $|X_1 \cap \text{\choice}(X_2) \setminus \text{\choice}(X_1)| = 0$  if and only if  $X_1 \cap \text{\choice}(X_2) \subseteq \text{\choice}(X_1)$ , which is the
 $\hookrightarrow$  definition of substitutability.
\end{proof}
```

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α),
  DGD26AdmissionsPredictability.paper_definition_substitutability C ↔
  ∀ {X₁ X₂ : Finset α}, X₁ ⊆ X₂ → ((X₁ ∩ C X₂) \ C X₁).card = 0
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_substitutability](#)

## Lean-elaborated claim roles

1. parameter: Type u\_1
2. parameter: DecidableEq α
3. parameter: DGD26AdmissionsPredictability.PaperChoiceRule α
4. conclusion: DGD26AdmissionsPredictability.paper\_definition\_substitutability C ↔  
 ∀ {X₁ X₂ : Finset α}, X₁ ⊆ X₂ → ((X₁ ∩ C X₂) \ C X₁).card = 0

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_substitutability\_term\_statement

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Matches: the target is exactly the source equivalence between substitutability and the zero first choice-distance term on all nested pools.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 16

**Source locator:** cited publication, lines 532-546

## Verbatim source input

```
\begin{restatable}{lemma}{unsubstitutable}
\label{lem:unsubstitutable}
If  $\$choice\$$  is not substitutable, there exists some  $\$x^*\$, \$x'\$,  $\$X\$$  and  $\$X' = X \cup \{x'\}$  where  $\$x^* \in X\$$  and  $\$x^* \in \text{choice}(X)\$$  but  $\$x^* \notin \text{choice}(X')\$$ .
 $\hookrightarrow \text{choice}(X)\$$ .
\end{restatable}

\textit{Intuition}:
If a choice function is not substitutable, then there is at least one situation where a decision ``flips'' from rejection to acceptance.
We argue that there has to be some single element  $\$x'\$$  that causes this ``flip'' when added.

\begin{proof}
 $\$choice\$$  is not substitutable if there exists some  $\$x^*\$$  and  $\$X_0 \subseteq X_k\$$  where  $\$x^* \in X_0\$$  and  $\$x^* \in \text{choice}(X_k)\$$  but  $\$x \notin \text{choice}(X_0)\$$ .
 $\hookrightarrow \text{choice}(X_0)\$$ .
Let  $\$k\$$  be such that  $\$k = |X_k \setminus X_0|\$$ .

We want to show the existence of  $\$x'\$$ . To do this, we identify the ``closest'' sets where this occurs. For each  $\$x \in X_k \setminus X_0\$$ , add elements to
 $\hookrightarrow \$X_0\$$ , constructing  $\$X_0 \subseteq X_1 \subseteq \dots \subseteq X_j \subseteq \dots \subseteq X_k\$$ , where  $\$X_1 = X_0 \cup \{x_1\}\$$  and so on. Let  $\$X_j\$$ 
 $\hookrightarrow$  be the first in this sequence of sets such that  $\$x^* \in \text{choice}(X_j)\$$  where  $\$x^* \in \text{choice}(X_j)\$$  but  $\$x^* \notin \text{choice}(X_{j-1})\$$ . Then let  $\$x' = x_j\$$ ,  $\$X = X_{j-1}\$, and  $\$X' = X_j\$$ .
\end{proof}$$ 
```

## Expanded PaperInterface specification

$\forall \{\alpha : \text{Type } u_1\} [\text{inst} : \text{DecidableEq } \alpha] (C : \text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha),$   
 $\neg \text{DGD26AdmissionsPredictability.paper\_definition\_substitutability } C \rightarrow$   
 $\exists (X : \text{Finset } \alpha) (x : \alpha) (xstar : \alpha), x \notin X \wedge xstar \in X \wedge xstar \in C (\text{insert } x X) \wedge xstar \notin C X$

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_substitutability](#)

## Lean-elaborated claim roles

1. parameter:  $\text{Type } u_1$
2. parameter:  $\text{DecidableEq } \alpha$
3. parameter:  $\text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha$
4. assumption:  $\neg \text{DGD26AdmissionsPredictability.paper\_definition\_substitutability } C$
5. conclusion:  $\exists (X : \text{Finset } \alpha) (x : \alpha) (xstar : \alpha), x \notin X \wedge xstar \in X \wedge xstar \in C (\text{insert } x X) \wedge xstar \notin C X$

## Lean proof endpoint (built separately)

$\text{DGD26AdmissionsPredictability.PaperInterface.paper\_non\_substitutable\_single\_add\_statement}$

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Matches: non-substitutability yields a single fresh insertion at which an existing applicant changes from rejected to accepted, exactly as in the source lemma.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 17

**Source locator:** cited publication, lines 261-265

## Verbatim source input

```
\begin{restatable}[ML Representation]{proposition}{mlrepresentation}
\label{prop:ml-representation}
A model of the form~\eqref{eq:ml-independent} which makes independent predictions can only represent 0-unstable choice functions.
A model of the form~\eqref{eq:ml-rank} which ranks applicants can represent a 1-variable, 1-unstable choice function. No such model can represent a
↪ choice function with variability or instability greater than one.
\end{restatable}
```

[Next verbatim source excerpt]

```
\subsection{Proof of \Cref{prop:ml-representation}} \label{sec:ml-prop-proof}
```

Recall \Cref{prop:ml-representation}:  
\mlrepresentation\*

First consider using an ML model to make decisions; then it can be seen as a choice function with various properties.  
Thus we say that an ML model can represent a choice function with various properties if it can have those properties when viewed as a choice function. %

The first statement follows by noting that any rule of the form~\eqref{eq:ml-independent} must either admit or reject an applicant regardless of the  
↪ applicant pool, so it must be independent and hence \$0\$-unstable by \Cref{thm:independent-zero-unstable}. %  
The second and third statements follow by noting that scores in~\eqref{eq:ml-rank} induce a total order by embedding applicants in \$\mathbb{R}\$ and  
↪ selecting accordingly. Therefore any rule of the form~\eqref{eq:ml-rank} is \$q\$-representative (by definition) and hence has instability and variability  
↪ exactly one (by \Cref{thm:ranking-variability}), but no greater.

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {C : DGD26AdmissionsPredictability.PaperChoiceRule α} (score : α → ℝ)
(threshold : ℝ),
(∀ (x : α), 0 ≤ score x ∧ score x ≤ 1) →
DGD26AdmissionsPredictability.paper_choice_function_feasible C →
DGD26AdmissionsPredictability.paper_definition_ml_representation
(DGD26AdmissionsPredictability.paper_definition_fixed_threshold_predictor score threshold) C →
DGD26AdmissionsPredictability.paper_definition_zero_instability C
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_fixed\\_threshold\\_predictor](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_ml\\_representation](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_zero\\_instability](#)

## Lean-elaborated claim roles

1. parameter: Type u\_1
2. parameter: DecidableEq α
3. parameter: DGD26AdmissionsPredictability.PaperChoiceRule α
4. parameter: α → ℝ
5. parameter: ℝ
6. assumption: ∀ (x : α), 0 ≤ score x ∧ score x ≤ 1
7. assumption: DGD26AdmissionsPredictability.paper\_choice\_function\_feasible C
8. assumption: DGD26AdmissionsPredictability.paper\_definition\_ml\_representation  
(DGD26AdmissionsPredictability.paper\_definition\_fixed\_threshold\_predictor score threshold) C
9. parameter: Finset α
10. parameter: Finset α
11. assumption: X<sub>1</sub> ⊆ X<sub>2</sub>
12. conclusion: AppliedModelingLib.FiniteChoice.choiceDistance C X<sub>1</sub> X<sub>2</sub> = 0

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_ml\_fixed\_threshold\_representation\_zero\_unstable\_statement

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Matches: a feasible choice rule represented by the source fixed-score, fixed-threshold predictor is 0-unstable; the pool context preserves the source model's pool independence.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 18

**Source locator:** cited publication, lines 261-269

## Verbatim source input

```
\begin{restatable}[ML Representation]{proposition}{mlrepresentation}
\label{prop:ml-representation}
A model of the form~\eqref{eq:ml-independent} which makes independent predictions can only represent 0-unstable choice functions.
A model of the form~\eqref{eq:ml-rank} which ranks applicants can represent a 1-variable, 1-unstable choice function. No such model can represent a
↪ choice function with variability or instability greater than one.
\end{restatable}
```

This result shows that standard ML practice coherently applies to admissions data only in limited settings.  
The intuition for this result hinges on the fact that all 1-unstable and 1-variable choice functions correspond to total preference orderings over applicants  
↪ (formalized in Theorem~\ref{thm:variability}).  
Then the proposition follows from noticing that total preference orderings correspond to embedding applicants in  $\mathbb{R}$  (by their scores).

[Next verbatim source excerpt]

```
\subsection{Proof of \Cref{prop:ml-representation}} \label{sec:ml-prop-proof}
```

Recall \Cref{prop:ml-representation}:  
\mlrepresentation\*

First consider using an ML model to make decisions; then it can be seen as a choice function with various properties.  
Thus we say that an ML model can represent a choice function with various properties if it can have those properties when viewed as a choice function. %

The first statement follows by noting that any rule of the form~\eqref{eq:ml-independent} must either admit or reject an applicant regardless of the  
↪ applicant pool, so it must be independent and hence 0-unstable by \Cref{thm:independent-zero-unstable}. %  
The second and third statements follow by noting that scores in~\eqref{eq:ml-rank} induce a total order by embedding applicants in  $\mathbb{R}$  and  
↪ selecting accordingly. Therefore any rule of the form~\eqref{eq:ml-rank} is  $q$ -representative (by definition) and hence has instability and variability  
↪ exactly one (by \Cref{thm:ranking-variability}), but no greater.

**Settled source reading** The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE\_CLARIFICATIONS.md).

## Expanded PaperInterface specification

```
∀ (q : ℕ),
  0 < q →
    ∃ (score : Fin (q + 1) → ℝ) (hinjective : Function.Injective score),
      DGD26AdmissionsPredictability.paper_definition_tight_d_instability 1
        (DGD26AdmissionsPredictability.paperRankThresholdChoice q score hinjective) ∧
        DGD26AdmissionsPredictability.paper_definition_variability_exactly 1
        (DGD26AdmissionsPredictability.paperRankThresholdChoice q score hinjective)
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.paperRankThresholdChoice](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_tight\\_d\\_instability](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_variability\\_exactly](#)

## Lean-elaborated claim roles

```
1. parameter: ℕ
2. assumption: 0 < q
3. conclusion: ∃ (score : Fin (q + 1) → ℝ) (hinjective : Function.Injective score),
  DGD26AdmissionsPredictability.paper_definition_tight_d_instability 1
    (DGD26AdmissionsPredictability.paperRankThresholdChoice q score hinjective) ∧
    DGD26AdmissionsPredictability.paper_definition_variability_exactly 1
    (DGD26AdmissionsPredictability.paperRankThresholdChoice q score hinjective)
```

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_ml\_rank\_threshold\_can\_represent\_exact\_one\_statement

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Matches: for each positive capacity, the target gives a concrete score-ranked rule with tight instability one and exact variability one, witnessing the source statement that such a rank model can represent a 1-variable, 1-unstable rule.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 19

**Source locator:** cited publication, lines 261-269

## Verbatim source input

```
\begin{restatable}[ML Representation]{proposition}{mlrepresentation}
\label{prop:ml-representation}
A model of the form~\eqref{eq:ml-independent} which makes independent predictions can only represent 0-unstable choice functions.
A model of the form~\eqref{eq:ml-rank} which ranks applicants can represent a 1-variable, 1-unstable choice function. No such model can represent a
↪ choice function with variability or instability greater than one.
\end{restatable}
```

This result shows that standard ML practice coherently applies to admissions data only in limited settings.  
The intuition for this result hinges on the fact that all 1-unstable and 1-variable choice functions correspond to total preference orderings over applicants  
↪ (formalized in Theorem~\ref{thm:variability}).  
Then the proposition follows from noticing that total preference orderings correspond to embedding applicants in  $\mathbb{R}$  (by their scores).

[Next verbatim source excerpt]

```
\subsection{Proof of \Cref{prop:ml-representation}} \label{sec:ml-prop-proof}
```

Recall \Cref{prop:ml-representation}:  
\mlrepresentation\*

First consider using an ML model to make decisions; then it can be seen as a choice function with various properties.  
Thus we say that an ML model can represent a choice function with various properties if it can have those properties when viewed as a choice function. %

The first statement follows by noting that any rule of the form~\eqref{eq:ml-independent} must either admit or reject an applicant regardless of the  
↪ applicant pool, so it must be independent and hence 0-unstable by \Cref{thm:independent-zero-unstable}. %  
The second and third statements follow by noting that scores in~\eqref{eq:ml-rank} induce a total order by embedding applicants in  $\mathbb{R}$  and  
↪ selecting accordingly. Therefore any rule of the form~\eqref{eq:ml-rank} is  $q$ -representative (by definition) and hence has instability and variability  
↪ exactly one (by \Cref{thm:ranking-variability}), but no greater.

**Settled source reading** The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE\_CLARIFICATIONS.md).

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [Fintype α] {C : DGD26AdmissionsPredictability.PaperChoiceRule α} (q : ℕ)
  (operationalScore : α → ℝ) (hinjective : Function.Injective operationalScore),
  0 < q →
  DGD26AdmissionsPredictability.paper_choice_function_feasible C →
  DGD26AdmissionsPredictability.paper_definition_ml_representation
    (DGD26AdmissionsPredictability.paper_definition_rank_threshold_predictor q operationalScore hinjective) C →
  DGD26AdmissionsPredictability.paper_definition_d_instability 1 C
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_d\\_instability](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_ml\\_representation](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_rank\\_threshold\\_predictor](#)

## Lean-elaborated claim roles

1. parameter: Type u\_1
2. parameter: DecidableEq α
3. parameter: Fintype α
4. parameter: DGD26AdmissionsPredictability.PaperChoiceRule α
5. parameter: ℕ
6. parameter: α → ℝ
7. assumption: Function.Injective operationalScore
8. assumption: 0 < q
9. assumption: DGD26AdmissionsPredictability.paper\_choice\_function\_feasible C
10. assumption: DGD26AdmissionsPredictability.paper\_definition\_ml\_representation  
(DGD26AdmissionsPredictability.paper\_definition\_rank\_threshold\_predictor q operationalScore hinjective) C
11. parameter: Finset α
12. parameter: α
13. assumption: x ∉ X
14. conclusion: AppliedModelingLib.FiniteChoice.choiceDistance C X (insert x X) ≤ 1

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_ml\_rank\_threshold\_representation\_instability\_bound\_statement



**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Matches: a represented positive-capacity rank-threshold rule has instability at most one, which is the source no-greater-than-one claim. The queue explicitly treats injective operationalScore only as an ex-ante tie-break, not raw-score uniqueness.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

|  |
|--|
|  |
|--|

# Review row 20

**Source locator:** cited publication, lines 261-269

## Verbatim source input

```
\begin{restatable}[ML Representation]{proposition}{mlrepresentation}
\label{prop:ml-representation}
A model of the form~\eqref{eq:ml-independent} which makes independent predictions can only represent 0-unstable choice functions.
A model of the form~\eqref{eq:ml-rank} which ranks applicants can represent a 1-variable, 1-unstable choice function. No such model can represent a
↪ choice function with variability or instability greater than one.
\end{restatable}
```

This result shows that standard ML practice coherently applies to admissions data only in limited settings.  
The intuition for this result hinges on the fact that all 1-unstable and 1-variable choice functions correspond to total preference orderings over applicants  
↪ (formalized in Theorem~\ref{thm:variability}).  
Then the proposition follows from noticing that total preference orderings correspond to embedding applicants in  $\mathbb{R}$  (by their scores).

[Next verbatim source excerpt]

```
\subsection{Proof of \Cref{prop:ml-representation}} \label{sec:ml-prop-proof}
```

Recall \Cref{prop:ml-representation}:  
\mlrepresentation\*

First consider using an ML model to make decisions; then it can be seen as a choice function with various properties.  
Thus we say that an ML model can represent a choice function with various properties if it can have those properties when viewed as a choice function. %

The first statement follows by noting that any rule of the form~\eqref{eq:ml-independent} must either admit or reject an applicant regardless of the  
↪ applicant pool, so it must be independent and hence 0-unstable by \Cref{thm:independent-zero-unstable}. %  
The second and third statements follow by noting that scores in~\eqref{eq:ml-rank} induce a total order by embedding applicants in  $\mathbb{R}$  and  
↪ selecting accordingly. Therefore any rule of the form~\eqref{eq:ml-rank} is  $q$ -representative (by definition) and hence has instability and variability  
↪ exactly one (by \Cref{thm:ranking-variability}), but no greater.

**Settled source reading** The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE\_CLARIFICATIONS.md).

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] {C : DGD26AdmissionsPredictability.PaperChoiceRule α}
(q : ℕ) (operationalScore : α → ℝ) (hinjective : Function.Injective operationalScore),
0 < q →
  DGD26AdmissionsPredictability.paper_choice_function_feasible C →
  DGD26AdmissionsPredictability.paper_definition_ml_representation
  (DGD26AdmissionsPredictability.paper_definition_rank_threshold_predictor q operationalScore hinjective) C →
  DGD26AdmissionsPredictability.paper_definition_variability_at_most 1 C
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_ml\\_representation](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_rank\\_threshold\\_predictor](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_variability\\_at\\_most](#)

## Lean-elaborated claim roles

1. parameter: Type u\_1
2. parameter: DecidableEq α
3. parameter: Fintype α
4. parameter: DGD26AdmissionsPredictability.PaperChoiceRule α
5. parameter: ℕ
6. parameter: α → ℝ
7. assumption: Function.Injective operationalScore
8. assumption: 0 < q
9. assumption: DGD26AdmissionsPredictability.paper\_choice\_function\_feasible C
10. assumption: DGD26AdmissionsPredictability.paper\_definition\_ml\_representation  
(DGD26AdmissionsPredictability.paper\_definition\_rank\_threshold\_predictor q operationalScore hinjective) C
11. parameter: Finset α
12. conclusion: (AppliedModelingLib.FiniteChoice.borderlineSet C X).card ≤ 1

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_ml\_rank\_threshold\_representation\_variability\_bound\_statement

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Matches: a represented positive-capacity rank-threshold rule has variability at most one, which is the source no-greater-than-one claim. The queue explicitly confines operationalScore injectivity to the tie-breaking refinement.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

|  |
|--|
|  |
|--|

# Review row 21

**Source locator:** cited publication, lines 347-358

## Verbatim source input

```
\begin{restatable}[]{\proposition}{empiricalmethods}
\label{prop:program-variability}
All considered choice functions ({Ed. Opt}, {Screened/Open}, {Ed. Opt with DIA}, and {Screened/Open with DIA}) are $1$-unstable. Furthermore, their
 $\leftrightarrow$  variability is (tightly) equal to their number of queues:

\begin{itemize}[noitemsep,topsep=0pt,parsep=0pt,partopsep=0pt]
\item Screened/Open programs are $1$-variable
\item Screened/Open with DIA programs are $2$-variable
\item Ed. Opt. programs are $3$-variable
\item Ed. Opt with DIA programs are $6$-variable
\end{itemize}

\end{restatable}
```

## Expanded PaperInterface specification

```
DGD26AdmissionsPredictability.paper_definition_d_instability 1
  DGD26AdmissionsPredictability.paper_definition_screened_open_program_choice  $\wedge$ 
DGD26AdmissionsPredictability.paper_definition_d_instability 1
  DGD26AdmissionsPredictability.paper_definition_screened_open_dia_program_choice  $\wedge$ 
DGD26AdmissionsPredictability.paper_definition_d_instability 1
  DGD26AdmissionsPredictability.paper_definition_educational_option_program_choice  $\wedge$ 
DGD26AdmissionsPredictability.paper_definition_d_instability 1
  DGD26AdmissionsPredictability.paper_definition_educational_option_dia_program_choice
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.paper\\_definition\\_d\\_instability](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_educational\\_option\\_dia\\_program\\_choice](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_educational\\_option\\_program\\_choice](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_screened\\_open\\_dia\\_program\\_choice](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_screened\\_open\\_program\\_choice](#)

## Lean-elaborated claim roles

```
1. conclusion: DGD26AdmissionsPredictability.paper_definition_d_instability 1
  DGD26AdmissionsPredictability.paper_definition_screened_open_program_choice  $\wedge$ 
DGD26AdmissionsPredictability.paper_definition_d_instability 1
  DGD26AdmissionsPredictability.paper_definition_screened_open_dia_program_choice  $\wedge$ 
DGD26AdmissionsPredictability.paper_definition_d_instability 1
  DGD26AdmissionsPredictability.paper_definition_educational_option_program_choice  $\wedge$ 
DGD26AdmissionsPredictability.paper_definition_d_instability 1
  DGD26AdmissionsPredictability.paper_definition_educational_option_dia_program_choice
```

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_program\_classes\_one\_instability\_statement

## Recorded source-to-Spec screening Verdict: matches

**Reason:** Independent outcome-blind review: under the current source-model procedure convention, the Spec is the source aggregate's conjunction of 1-instability for the documented one-, two-, three-, and six-queue abstract procedures, not an empirical class-wide assertion.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

## Reviewer annotation

# Review row 22

**Source locator:** cited publication, lines 347-358

## Verbatim source input

```
\begin{restatable}[]{\proposition}{empiricalmethods}
\label{prop:program-variability}
All considered choice functions ({Ed. Opt}, {Screened/Open}, {Ed. Opt with DIA}, and {Screened/Open with DIA}) are $1$-unstable. Furthermore, their
 $\leftrightarrow$  variability is (tightly) equal to their number of queues:

\begin{itemize}[noitemsep,topsep=0pt,parsep=0pt,partopsep=0pt]
\item Screened/Open programs are $1$-variable
\item Screened/Open with DIA programs are $2$-variable
\item Ed. Opt. programs are $3$-variable
\item Ed. Opt with DIA programs are $6$-variable
\end{itemize}

\end{restatable}
```

## Expanded PaperInterface specification

```
DGD26AdmissionsPredictability.paper_definition_variability_exactly 1
DGD26AdmissionsPredictability.paper_definition_screened_open_program_choice
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.paper\\_definition\\_screened\\_open\\_program\\_choice](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_variability\\_exactly](#)

## Lean-elaborated claim roles

```
1. conclusion: AppliedModelingLib.FiniteChoice.VariabilityAtMost 1
   DGD26AdmissionsPredictability.paper_definition_screened_open_program_choice  $\wedge$ 
 $\exists (X : \text{Finset } (\text{Fin } 2)),$ 
   (AppliedModelingLib.FiniteChoice.borderlineSet
    DGD26AdmissionsPredictability.paper_definition_screened_open_program_choice X).card =
1
```

## Lean proof endpoint (built separately)

```
DGD26AdmissionsPredictability.PaperInterface.paper_screened_open_variability_one_statement
```

**Recorded source-to-Spec screening Verdict:** matches  
**Reason:** Independent outcome-blind review: under the current source-model procedure convention, the source’s one-ranking conclusion matches the abstract one-queue procedure’s exact-variability Spec, not an empirical implementation claim.  
**Reviewer check:** ☐ Matches source input  
*If left unchecked, explain the discrepancy or uncertainty below.*  
**Reviewer annotation**

# Review row 23

**Source locator:** cited publication, lines 347-358

## Verbatim source input

```
\begin{restatable}[]{\proposition}{empiricalmethods}
\label{prop:program-variability}
All considered choice functions ({Ed. Opt}, {Screened/Open}, {Ed. Opt with DIA}, and {Screened/Open with DIA}) are $1$-unstable. Furthermore, their
↔ variability is (tightly) equal to their number of queues:

\begin{itemize}[noitemsep,topsep=0pt,parsep=0pt,partopsep=0pt]
\item Screened/Open programs are $1$-variable
\item Screened/Open with DIA programs are $2$-variable
\item Ed. Opt. programs are $3$-variable
\item Ed. Opt with DIA programs are $6$-variable
\end{itemize}

\end{restatable}
```

## Expanded PaperInterface specification

```
DGD26AdmissionsPredictability.paper_definition_variability_exactly 2
DGD26AdmissionsPredictability.paper_definition_screened_open_dia_program_choice
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.paper\\_definition\\_screened\\_open\\_dia\\_program\\_choice](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_variability\\_exactly](#)

## Lean-elaborated claim roles

```
1. conclusion: AppliedModelingLib.FiniteChoice.VariabilityAtMost 2
   DGD26AdmissionsPredictability.paper_definition_screened_open_dia_program_choice ^
  ∃ (X : Finset (Fin 4)),
    (AppliedModelingLib.FiniteChoice.borderlineSet
     DGD26AdmissionsPredictability.paper_definition_screened_open_dia_program_choice X).card =
    2
```

## Lean proof endpoint (built separately)

```
DGD26AdmissionsPredictability.PaperInterface.paper_screened_open_dia_variability_two_statement
```

**Recorded source-to-Spec screening Verdict:** matches  
**Reason:** Independent outcome-blind review: under the current source-model procedure convention, the source two-queue DIA tightness argument matches the abstract two-queue procedure’s exact-variability Spec, with no simulator or roster claim.  
**Reviewer check:** ☐ Matches source input  
*If left unchecked, explain the discrepancy or uncertainty below.*  
**Reviewer annotation**

# Review row 24

**Source locator:** cited publication, lines 347-358

## Verbatim source input

```
\begin{restatable}[]{proposition}{empiricalmethods}
\label{prop:program-variability}
All considered choice functions ({Ed. Opt}, {Screened/Open}, {Ed. Opt with DIA}, and {Screened/Open with DIA}) are $1$-unstable. Furthermore, their
↔ variability is (tightly) equal to their number of queues:

\begin{itemize}[noitemsep,topsep=0pt,parsep=0pt,partopsep=0pt]
\item Screened/Open programs are $1$-variable
\item Screened/Open with DIA programs are $2$-variable
\item Ed. Opt. programs are $3$-variable
\item Ed. Opt with DIA programs are $6$-variable
\end{itemize}

\end{restatable}
```

## Expanded PaperInterface specification

```
DGD26AdmissionsPredictability.paper_definition_variability_exactly 3
DGD26AdmissionsPredictability.paper_definition_educational_option_program_choice
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.paper\\_definition\\_educational\\_option\\_program\\_choice](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_variability\\_exactly](#)

## Lean-elaborated claim roles

```
1. conclusion: AppliedModelingLib.FiniteChoice.VariabilityAtMost 3
   DGD26AdmissionsPredictability.paper_definition_educational_option_program_choice ∧
   ∃ (X : Finset (Fin 6)),
   (AppliedModelingLib.FiniteChoice.borderlineSet
    DGD26AdmissionsPredictability.paper_definition_educational_option_program_choice X).card =
   3
```

## Lean proof endpoint (built separately)

```
DGD26AdmissionsPredictability.PaperInterface.paper_educational_option_variability_three_statement
```

**Recorded source-to-Spec screening Verdict:** matches  
**Reason:** Independent outcome-blind review: under the current source-model procedure convention, the source’s three-category displacement argument matches the exact-variability Spec for the documented abstract three-queue procedure, not an empirical implementation claim.  
**Reviewer check:** ☐ Matches source input  
*If left unchecked, explain the discrepancy or uncertainty below.*  
**Reviewer annotation**

# Review row 25

**Source locator:** cited publication, lines 347-358

## Verbatim source input

```
\begin{restatable}[]{\proposition}{empiricalmethods}
\label{prop:program-variability}
All considered choice functions ({Ed. Opt}, {Screened/Open}, {Ed. Opt with DIA}, and {Screened/Open with DIA}) are $1$-unstable. Furthermore, their
↔ variability is (tightly) equal to their number of queues:

\begin{itemize}[noitemsep,topsep=0pt,parsep=0pt,partopsep=0pt]
\item Screened/Open programs are $1$-variable
\item Screened/Open with DIA programs are $2$-variable
\item Ed. Opt. programs are $3$-variable
\item Ed. Opt with DIA programs are $6$-variable
\end{itemize}

\end{restatable}
```

## Expanded PaperInterface specification

```
DGD26AdmissionsPredictability.paper_definition_variability_exactly 6
DGD26AdmissionsPredictability.paper_definition_educational_option_dia_program_choice
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.paper\\_definition\\_educational\\_option\\_dia\\_program\\_choice](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_variability\\_exactly](#)

## Lean-elaborated claim roles

```
1. conclusion: AppliedModelingLib.FiniteChoice.VariabilityAtMost 6
   DGD26AdmissionsPredictability.paper_definition_educational_option_dia_program_choice ^
   ∃ (X : Finset (Fin 12)),
   (AppliedModelingLib.FiniteChoice.borderlineSet
    DGD26AdmissionsPredictability.paper_definition_educational_option_dia_program_choice X).card =
   6
```

## Lean proof endpoint (built separately)

```
DGD26AdmissionsPredictability.PaperInterface.paper_educational_option_dia_variability_six_statement
```

**Recorded source-to-Spec screening Verdict:** matches  
**Reason:** Independent outcome-blind review: under the current source-model procedure convention, the source’s six-queue displacement construction matches the exact-variability Spec for the documented abstract six-queue procedure; it makes no empirical implementation claim.  
**Reviewer check:** ☐ Matches source input  
*If left unchecked, explain the discrepancy or uncertainty below.*  
**Reviewer annotation**



# Review row 26

**Source locator:** cited publication, lines 998-1003

## Verbatim source input

```
\begin{restatable}[Additive Variability Bound]{theorem}{addvar}
```

```
\label{thm:additive-variability}
```

If  $\text{\textit{choice}}$  is a sequential composition of  $q$ -acceptant,  $1$ -unstable functions with variabilities  $m_1, \dots, m_n$ , then  $\text{\textit{choice}}$  has variability at most  $\sum_{i=1}^n m_i$ .

```
\end{restatable}
```

Note that this is only an upper bound; we could trivially construct cases where this is not tight. For example, compose a  $\text{\textit{choice}}$  of two

- $q$ -representative choice functions with the same underlying ordering. On the other hand, it is also easy to construct instances in which this upper
- bound is tight. Consider the composition of  $m$  queues which each rank a student by a different subject area exam. Then the addition of a new
- student to the applicant pool could displace any of  $m$  students, depending on the subject area.

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] {qs ms : List ℕ}
{Cs : List (DGD26AdmissionsPredictability.PaperChoiceRule α)},
List.Forall₂
  (fun (q : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
    DGD26AdmissionsPredictability.paper_choice_function_feasible C ∧
    DGD26AdmissionsPredictability.paper_definition_q_acceptance q C ∧
    DGD26AdmissionsPredictability.paper_definition_d_instability 1 C)
qs Cs →
List.Forall₂
  (fun (m : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
    DGD26AdmissionsPredictability.paper_definition_variability_exactly m C)
ms Cs →
DGD26AdmissionsPredictability.paper_definition_variability_at_most ms.sum
(DGD26AdmissionsPredictability.paper_definition_sequential_composition Cs)
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_d\\_instability](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_acceptance](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_sequential\\_composition](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_variability\\_at\\_most](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_variability\\_exactly](#)

## Lean-elaborated claim roles

1. parameter: Type u\_1
2. parameter: DecidableEq α
3. parameter: Fintype α
4. parameter: List ℕ
5. parameter: List ℕ
6. parameter: List (DGD26AdmissionsPredictability.PaperChoiceRule α)
7. assumption: List.Forall₂  
(fun (q : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>  
 DGD26AdmissionsPredictability.paper\_choice\_function\_feasible C ∧  
 DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance q C ∧  
 DGD26AdmissionsPredictability.paper\_definition\_d\_instability 1 C)  
qs Cs
8. assumption: List.Forall₂  
(fun (m : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>  
 DGD26AdmissionsPredictability.paper\_definition\_variability\_exactly m C)  
ms Cs
9. parameter: Finset α
10. conclusion: (AppliedModelingLib.FiniteChoice.borderlineSet  
(DGD26AdmissionsPredictability.paper\_definition\_sequential\_composition Cs) X).card ≤  
ms.sum

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_sequential\_additive\_variability\_bound\_statement

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Matches: a residual-pool sequential composition of q-acceptant 1-unstable components with exact variabilities  $m_i$  has variability at most their sum, exactly the source additive bound.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 27

**Source locator:** cited publication, lines 850-879

## Verbatim source input

```
\begin{restatable}{theorem}{appendremove}
\label{thm:append-remove-size}
For  $q$ -acceptant, 1-unstable  $\mathcal{X}$ , and  $|\mathcal{X}| \geq 2q$ , the maximum size of the waitlisted set is equal to the maximum size of the borderline set;
 $\hookrightarrow$  that is,
\begin{equation}
\max_{\mathcal{X} \subseteq \mathcal{X}} |V_{\text{choice}(\mathcal{X})}| = \max_{\mathcal{X} \subseteq \mathcal{X}} |A_{\text{choice}(\mathcal{X})}|
\end{equation}
\end{restatable}
```

In short, if there is a set  $X$  with a waitlisted set of size  $u$ , there must be a set  $X_0$  with a borderline set of size (at least)  $u$ , and vice versa. %

```
\begin{proof}
To prove equality, we will prove that 1)  $\max_{\mathcal{X} \subseteq \mathcal{X}} |V_{\text{choice}(\mathcal{X})}| \geq \max_{\mathcal{X} \subseteq \mathcal{X}} |A_{\text{choice}(\mathcal{X})}|$  and then
 $\hookrightarrow$  that 2)  $\max_{\mathcal{X} \subseteq \mathcal{X}} |V_{\text{choice}(\mathcal{X})}| \leq \max_{\mathcal{X} \subseteq \mathcal{X}} |A_{\text{choice}(\mathcal{X})}|$ .

\textbf{Part 1).}
To prove the first statement, assume that there is some  $X$  where the waitlisted set is of cardinality at least  $u$ , so  $x^*_1, \dots, x^*_u \in A_{\text{choice}(X)}$ .

Then (recall that  $\mathcal{X}$  is consistent) there are  $X_1, \dots, X_u \subseteq X$  where  $X_i = X \setminus \{x^*_i\}$  such that  $\text{choice}(X_i) = x^*_i \cup \text{choice}(X) \setminus x^*_i$  for  $i \in \{1, \dots, u\}$ . Then consider  $X_0 = \bigcap_{i=1}^u X_i = X \setminus \{x^*_1, \dots, x^*_u\}$ , and  $\text{choice}(X_0)$ . It
 $\hookrightarrow$  must be that  $\text{choice}(X_0) = \{x^*_1, \dots, x^*_u\} \cup \text{choice}(X) \setminus \{x^*_1, \dots, x^*_u\}$  by substitutability. I claim that  $\{x^*_1, \dots, x^*_u\} \subseteq \text{choice}(X_0)$ , which means  $|V_{\text{choice}(X_0)}| \geq u$ , which would complete the proof.

Assume for contradiction that this is untrue;  $\{x^*_1, \dots, x^*_u\} \not\subseteq \text{choice}(X_0)$ . Then some  $x^*_i$ , say  $x^*_u$  (without loss of
 $\hookrightarrow$  generality), is not in  $\text{choice}(X_0)$ , which means that  $x^*_u \in \text{choice}(X) \setminus \text{choice}(X_0)$ .
Consider the makeup of  $\text{choice}(X_0)$ . Recall that  $\{x^*_i\} \cup X_0 = \bigcap_{i=1}^u X_i = X \setminus \{x^*_1, \dots, x^*_{u-1}\}$ .
 $\hookrightarrow$  Because of substitutability, as a subset of  $X$ , we have  $\text{choice}(X) \setminus \{x^*_1, \dots, x^*_{u-1}\} \subseteq \text{choice}(X_0)$ . From
 $\hookrightarrow$  substitutability, as a subset of from  $X_1$  through  $X_{u-1}$ ,  $\{x^*_2, \dots, x^*_{u-1}\}$  are likewise chosen.
However, consider the size of  $\text{choice}(X_0)$ .  $\text{choice}(X) \setminus \{x^*_1, \dots, x^*_{u-1}\}$  is of size  $q - (u-1)$ , because  $x^*_i \in \text{choice}(X)$ 
 $\hookrightarrow$  for all  $i$ . We also know that  $|\{x^*_1, \dots, x^*_{u-1}\}| = u-1$ , and as no  $x^*_i$  is in  $\text{choice}(X)$  by definition, all of these elements are distinct.
 $\hookrightarrow$  As  $x^*_u$  remains by assumption, this adds up to  $|\text{choice}(X_0)| = q+1$ , which violates  $q$ -acceptance and completes the proof.

\textbf{Part 2).}
The proof for 2) is extremely similar to the former. Begin with  $X_0$  with  $V_{\text{choice}(X_0)} = \{x^*_1, \dots, x^*_m\}$ , with  $x^*_1, \dots, x^*_m$  such that
 $\hookrightarrow \text{choice}(X_1) = \{x^*_1\} \cup \text{choice}(X_0) \setminus \{x^*_1\}$ . Let  $X = \bigcup_{i=1}^m X_i = X_0 \cup \{x^*_1, \dots, x^*_m\}$  and  $X_i = X \setminus \{x^*_i\}$ . Note that  $X_i = X_1 \cup \dots \cup X_{i-1} \cup X_{i+1} \cup \dots \cup X_m$ . %
Because of 1)-instability,  $|\text{choice}(X_0) \setminus \text{choice}(X)| \leq |X_0 \setminus \text{choice}(X)| = m$ . Because of substitutability, any  $x \in \text{choice}(X)$  is either in
 $\hookrightarrow \text{choice}(X_0)$  or not in  $X_0$  and therefore in  $\{x^*_1, \dots, x^*_m\}$ .

I claim  $\text{choice}(X) = \{x^*_1, \dots, x^*_m\} \cup \text{choice}(X_0) \setminus \{x^*_1, \dots, x^*_m\}$ . Firstly,  $x^*_i \notin \text{choice}(X)$  because  $x^*_i \notin \text{choice}(X_1)$  and  $X_1 \subseteq X$ , so that would be a substitutability violation. As  $\text{choice}(X_0) \setminus \{x^*_1, \dots, x^*_m\}$  has  $q-m$ 
 $\hookrightarrow$  elements, the only possible elements to fill  $\text{choice}(X)$  must be the  $m$  elements of  $\{x^*_1, \dots, x^*_m\}$ .

The same argument holds for any  $X_i$ , but use  $X_m$  without loss of generality for ease of notation;  $X_j \subseteq X_m$  for  $m \neq j$ , so  $x^*_j \notin \text{choice}(X_m)$  by substitutability. With  $\text{choice}(X_0) \setminus \{x^*_1, \dots, x^*_{m-1}\}$  being the only  $q-m+1$  elements of  $X_0$  that can
 $\hookrightarrow$  be in  $\text{choice}(X_m)$ , the other  $m-1$  elements must be  $\{x^*_1, \dots, x^*_{m-1}\}$ . Then consider the definition of  $A_{\text{choice}(X)}$  and see that
 $\hookrightarrow \text{choice}(X) \setminus \text{choice}(X_0) = \{x^*_1, \dots, x^*_m\}$  for any  $i$ , with each  $x^*_i$  being unique.
\end{proof}
```

## Expanded PaperInterface specification

```
∀ {α : Type u 1} [inst : DecidableEq α] [inst 1 : Fintype α] {m q : ℕ}
{C : DGD26AdmissionsPredictability.PaperChoiceRule α},
DGD26AdmissionsPredictability.paper_choice_function_feasible C →
DGD26AdmissionsPredictability.paper_definition_q_acceptance q C →
DGD26AdmissionsPredictability.paper_definition_d_instability 1 C →
2 * q ≤ Fintype.card α →
(DGD26AdmissionsPredictability.paper_definition_general_variability_exactly m C ↔
DGD26AdmissionsPredictability.paper_definition_variability_exactly m C)
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_d\\_instability](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_general\\_variability\\_exactly](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_acceptance](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_variability\\_exactly](#)

## Lean-elaborated claim roles

1. parameter: Type u 1
2. parameter: DecidableEq α
3. parameter: Fintype α
4. parameter: ℕ

5. parameter:  $\mathbb{N}$   
 6. parameter:  $\text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha$   
 7. assumption:  $\text{DGD26AdmissionsPredictability.paper\_choice\_function\_feasible } C$   
 8. assumption:  $\text{DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance } q \ C$   
 9. assumption:  $\text{DGD26AdmissionsPredictability.paper\_definition\_d\_instability } 1 \ C$   
 10. assumption:  $2 * q \leq \text{Fintype.card } \alpha$   
 11. conclusion:  $\text{DGD26AdmissionsPredictability.paper\_definition\_general\_variability\_exactly } m \ C \leftrightarrow \text{DGD26AdmissionsPredictability.paper\_definition\_variability\_exactly } m \ C$

## Lean proof endpoint (built separately)

$\text{DGD26AdmissionsPredictability.PaperInterface.paper\_append\_remove\_variability\_exact\_equivalence\_statement}$

## Recorded source-to-Spec screening Verdict: matches

**Reason:** Matches: the finite  $q$ -acceptant, 1-unstable target with  $2q$  applicants available is the source equality between the maximum borderline and waitlisted-set sizes, expressed as equivalence of the two exact-variability definitions.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

## Reviewer annotation

# Review row 28

**Source locator:** cited publication, lines 576-580

## Verbatim source input

```
\begin{restatable}[Triangle Inequality]{theorem}{triangle}
\label{thm:triangle-ineq}
For any choice function  $\text{\textit{choice}}$  and sets  $X_1 \subseteq X_2 \subseteq X_3$ ,  $\text{\textit{choice}}$  obeys the triangle inequality;

$$r_{\text{\textit{choice}}}(X_1, X_3) \leq r_{\text{\textit{choice}}}(X_1, X_2) + r_{\text{\textit{choice}}}(X_2, X_3)$$

\end{restatable}
```

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) {X₁ X₂ X₃ : Finset α},
X₁ ⊆ X₂ →
X₂ ⊆ X₃ →
DGD26AdmissionsPredictability.paper_definition_choice_distance C X₁ X₃ ≤
DGD26AdmissionsPredictability.paper_definition_choice_distance C X₁ X₂ +
DGD26AdmissionsPredictability.paper_definition_choice_distance C X₂ X₃
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_choice\\_distance](#)

## Lean-elaborated claim roles

1. parameter: Type u\_1
2. parameter: DecidableEq α
3. parameter: DGD26AdmissionsPredictability.PaperChoiceRule α
4. parameter: Finset α
5. parameter: Finset α
6. parameter: Finset α
7. assumption: X₁ ⊆ X₂
8. assumption: X₂ ⊆ X₃
9. conclusion: (DGD26AdmissionsPredictability.paper\_definition\_choice\_distance C X₁ X₃).le (DGD26AdmissionsPredictability.paper\_definition\_choice\_distance C X₁ X₂ + DGD26AdmissionsPredictability.paper\_definition\_choice\_distance C X₂ X₃)

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_choice\_distance\_triangle\_statement

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Matches: the target is the source triangle inequality for choice distance on nested finite applicant sets.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

## Review row 29

**Source locator:** cited publication, lines 777-794

### Verbatim source input

```

\begin{restatable}[Even Instability is Inconsistent]{theorem}{inconsistentstable}
\label{thm:even-inconsistency}
For any  $q$ -acceptant  $\mathcal{A}$ ,  $\mathcal{A}$  is inconsistent if and only if there exist  $X, X' = X \cup \{x\}$  where  $r_{\mathcal{A}}(X, X')$  is positive and even.
\end{restatable}

\textit{Intuition}: for  $\mathcal{A}$  to have an even but nonzero choice distance  $r_{\mathcal{A}}$ , the indicator term in  $\text{lem:unstable-calc}$  must be zero; the
 $\hookrightarrow$  added  $x$  is not selected. But that means  $x$  is a rejected element that still has an impact, which violates consistency.
\begin{proof}
\textbf{Informal}
First, we prove that an even and positive value of  $r_{\mathcal{A}}$  implies inconsistency. For  $\mathcal{A}$  to have an even but nonzero choice distance
 $\hookrightarrow$   $r_{\mathcal{A}}$ , the indicator term in  $\text{lem:unstable-calc}$  must be zero; the added  $x$  is not selected. But that means  $x$  affects selection
 $\hookrightarrow$  without actually being chosen, which violates consistency.

Next, prove that consistency limits the values of  $r_{\mathcal{A}}$ . If  $\mathcal{A}$  is not consistent, then there is some  $X, X'$  where the new element  $x$  is not
 $\hookrightarrow$  chosen (so the indicator term in  $\text{lem:unstable-calc}$  is zero and  $r_{\mathcal{A}}$  is even), but changes the chosen elements. This makes  $r_{\mathcal{A}}$ 
 $\hookrightarrow$  is positive under a fixed capacity constraint.

\textbf{Formal}
Consider  $\text{lem:unstable-calc}$ . See that for  $r_{\mathcal{A}} \neq 0$ ,  $r_{\mathcal{A}}$  is odd if and only if  $x \notin \mathcal{A}(X')$  and even if and only if  $x \in \mathcal{A}(X')$ .
 $\hookrightarrow$   $\mathcal{A}(X')$ . If  $x \notin \mathcal{A}(X')$ , then  $\mathcal{A}(X') \subseteq X$ . %
So if  $x \notin \mathcal{A}(X')$  but  $n > 0$ ,  $\mathcal{A}(X') \subseteq X$  but  $\mathcal{A}(X') \neq \mathcal{A}(X)$ , which proves inconsistency.
Similarly, if  $\mathcal{A}$  is inconsistent, then  $\mathcal{A}(X') \subseteq X$  but  $\mathcal{A}(X') \neq \mathcal{A}(X)$ . %
As  $\mathcal{A}(X') \neq \mathcal{A}(X)$ ,  $n > 0$ , but for  $\mathcal{A}(X') \subseteq X$  to be true,  $x \notin \mathcal{A}(X')$ , and so  $r_{\mathcal{A}}(X, X')$  is even.
\end{proof}

```

## Expanded PaperInterface specification

$$\begin{aligned} & \forall \{ \alpha : \text{Type } u_1 \} [\text{inst} : \text{DecidableEq } \alpha] \{ q : \mathbb{N} \} \{ C : \text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha \}, \\ & \text{DGD26AdmissionsPredictability.paper\_choice\_function\_feasible } C \rightarrow \\ & \text{DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance } q \ C \rightarrow \\ & (\neg \text{DGD26AdmissionsPredictability.paper\_definition\_consistency } C \leftrightarrow \\ & \quad \exists (X : \text{Finset } \alpha), \\ & \quad \exists x \notin X, \\ & \quad 0 < \text{DGD26AdmissionsPredictability.paper\_definition\_choice\_distance } C \ X \ (\text{insert } x \ X) \wedge \\ & \quad \exists (k : \mathbb{N}), \text{DGD26AdmissionsPredictability.paper\_definition\_choice\_distance } C \ X \ (\text{insert } x \ X) = 2 * k) \end{aligned}$$

## Retained governing declarations

- DGD26AdmissionsPredictability.PaperChoiceRule
- DGD26AdmissionsPredictability.paper\_choice\_function\_feasible
- DGD26AdmissionsPredictability.paper\_definition\_choice\_distance
- DGD26AdmissionsPredictability.paper\_definition\_consistency
- DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance

### Lean-elaborated claim roles

1. parameter: Type  $u_1$
2. parameter: DecidableEq  $\alpha$
3. parameter:  $\mathbb{N}$
4. parameter: DGD26AdmissionsPredictability.PaperChoiceRule  $\alpha$
5. assumption: DGD26AdmissionsPredictability.paper\_choice\_function\_feasible C
6. assumption: DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance q C
7. conclusion:  $\neg$ DGD26AdmissionsPredictability.paper\_definition\_consistency C  $\leftrightarrow$   
 $\exists (X : \text{Finset } \alpha),$   
 $\exists x \notin X,$   
 $0 < \text{DGD26AdmissionsPredictability.paper\_definition\_choice\_distance } C \ X \ (\text{insert } x \ X) \wedge$   
 $\exists (k : \mathbb{N}), \text{DGD26AdmissionsPredictability.paper\_definition\_choice\_distance } C \ X \ (\text{insert } x \ X) = 2 * k$

### Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper even distance iff inconsistent statement

Recorded source-to-Spec screening Verdict: matches

**Reason:** Matches: under q-acceptance, inconsistency is equivalent to a fresh insertion with positive even choice distance, exactly as the source theorem states.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

### Reviewer annotation

# Review row 30

**Source locator:** cited publication, lines 683-707

## Verbatim source input

```
\begin{restatable}{theorem}{Independent Rules are Substitutable and Monotonic}
\label{thm:independent-sub-mon}
$\choice$ is independent if and only if it is both substitutable and monotonic.
\end{restatable}

\textit{Intuition}: The only way for adding applicants to neither hurt those already accepted or help those already rejected is to not affect them at all.
\begin{proof}
  Assume $\choice$ is independent.

  \textbf{Informal}
  Consider how independence implies substitutability and monotonicity. Independence means any element is always accepted or always rejected
  → regardless of applicant pool. Then any $x$ that is accepted in $X_1$ is still accepted in any subset of $X_1$, which satisfies the definition of
  → substitutability, and is accepted in any superset of $X_1$, which satisfies monotonicity.

  To prove that substitutability and monotonicity combine to form independence, consider the behavior they cover. Recall the intuition behind
  → substitutability --- removing applicants does not lead to rejecting those accepted, and the intuition behind monotonicity --- adding applicants does
  → not lead to rejecting those accepted. Given both properties, nothing can affect an acceptance decision at all.

  \textbf{Formal}
  First, we assume independence and prove $0$-stability.
  Recall \Cref{lem:substitutability-term}: $\choice$ is substitutable if and only if $|X_1 \cap \choice(X_2) \setminus \choice(X_1)| = 0$ for all $X_1$
  → $\subseteq X_2$. Then for any $x \in X_1$, either $x \in \choice(X_1) \cap \choice(X_2)$ or $x \notin \choice(X_1) \cap \choice(X_2)$, so $X_1 \cap \choice(X_2)$
  → $= \choice(X_1)$. Then $|X_1 \cap \choice(X_2) \setminus \choice(X_1)| = 0$ and so $\choice$ is substitutable. Similarly, recall
  → \Cref{lem:monotonicity-term}: $\choice$ is monotonic if and only if $|X_1 \setminus \choice(X_2)| = 0$ for all $X_1 \subseteq X_2$. Any
  → element $x \in \choice(X_1)$ is in $X_2$ because $\choice(X_1) \subseteq X_1 \subseteq X_2$. By definition of independence, $x \in \choice(X)$ for
  → all $X$ where $x \in X$, so if $x \in \choice(X_1)$, $x \in \choice(X_2)$, so $\choice(X_1) \subseteq \choice(X_2)$.

  To prove that substitutability and monotonicity together imply independence, consider any $x \in X$. If $x \in \choice(X)$, we can show that $x \in \choice(X')$
  → $\choice(X')$ for any $X'$ where $x \in X'$. By monotonicity, $x \in \choice(X' \cup X)$, and so by substitutability, $x \in \choice(X')$. If $x \notin \choice(X)$,
  → $\choice(X)$, we can prove $x \notin \choice(X')$ for any $X'$. If we assumed for contradiction that $x \in \choice(X')$, then the same argument as
  → above proves $x \in \choice(X)$, leading to contradiction.
\end{proof}

\begin{restatable}{corollary}{Only Independent Rules are 0-unstable}
\label{thm:independent-zero-unstable}
$\choice$ is $0$-unstable if and only if it is independent, and therefore no $q$-acceptant $\choice$ can be $0$-unstable.
\end{restatable}
```

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α),
  DGD26AdmissionsPredictability.paper_choice_function_feasible C →
  (DGD26AdmissionsPredictability.paper_definition_independence C ↔
   DGD26AdmissionsPredictability.paper_definition_substitutability C ∧
   DGD26AdmissionsPredictability.paper_definition_monotonicity C)
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_independence](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_monotonicity](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_substitutability](#)

## Lean-elaborated claim roles

1. parameter: Type u\_1
2. parameter: DecidableEq α
3. parameter: DGD26AdmissionsPredictability.PaperChoiceRule α
4. assumption: DGD26AdmissionsPredictability.paper\_choice\_function\_feasible C
5. conclusion: DGD26AdmissionsPredictability.paper\_definition\_independence C ↔  
DGD26AdmissionsPredictability.paper\_definition\_substitutability C ∧  
DGD26AdmissionsPredictability.paper\_definition\_monotonicity C

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_independent\_substitutable\_monotonic\_statement

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Matches: for a feasible choice rule, the target is precisely the source equivalence between independence and the conjunction of substitutability and monotonicity.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**



# Review row 31

**Source locator:** cited publication, lines 280-289

## Verbatim source input

```
\begin{restatable}{theorem}{thminstability}
\label{thm:instability}
A  $q$ -acceptant choice function
\begin{enumerate}
\item cannot be 0-unstable,
\item is exactly 1-unstable if and only if it is substitutable,
\item can be tightly  $d$ -unstable for every  $1 \leq d \leq 2q$ .
\end{enumerate}
\end{restatable}%
```

**Formalized review target** The archival source above is not asserted equivalent to this formalized target.

For a feasible  $q$ -acceptant choice rule  $C$  on a universe with an input pool strictly larger than positive capacity  $q$ ,  $C$  is not 0-unstable.

**Archival source anchor:** cited publication, lines 281-285

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {q : ℕ} {C : DGD26AdmissionsPredictability.PaperChoiceRule α},
DGD26AdmissionsPredictability.paper_choice_function_feasible C →
DGD26AdmissionsPredictability.paper_definition_q_acceptance q C →
0 < q → ∀ {U : Finset α}, q < U.card → ¬DGD26AdmissionsPredictability.paper_definition_zero_instability C
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_acceptance](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_zero\\_instability](#)

## Lean-elaborated claim roles

1. parameter: Type u\_1
2. parameter: DecidableEq α
3. parameter: ℕ
4. parameter: DGD26AdmissionsPredictability.PaperChoiceRule α
5. assumption: DGD26AdmissionsPredictability.paper\_choice\_function\_feasible C
6. assumption: DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance q C
7. assumption: 0 < q
8. parameter: Finset α
9. assumption: q < U.card
10. assumption: DGD26AdmissionsPredictability.paper\_definition\_zero\_instability C
11. conclusion: False

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_no\_zero\_instability\_under\_capacity\_statement

**Recorded source-to-Spec screening Verdict:** matches\_approved\_corrected\_target

**Reason:** Matches the approved corrected target: positive capacity and an offered pool larger than  $q$  make the no-zero conclusion nonvacuous, and the target states exactly that corrected source claim.

**Reviewer check:** ☐ Matches formalized target

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 32

**Source locator:** cited publication, lines 280-289

## Verbatim source input

```
\begin{restatable}{theorem}{thminstability}
\label{thm:instability}
A  $q$ -acceptant choice function
\begin{enumerate}
\item cannot be  $0$ -unstable,
\item is exactly  $1$ -unstable if and only if it is substitutable,
\item can be tightly  $d$ -unstable for every  $1 \leq d \leq 2q$ .
\end{enumerate}
\end{restatable}%
```

**Settled source reading** The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE\_CLARIFICATIONS.md).

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {q : ℕ} {C : DGD26AdmissionsPredictability.PaperChoiceRule α},
DGD26AdmissionsPredictability.paper_choice_function_feasible C →
DGD26AdmissionsPredictability.paper_definition_q_acceptance q C →
0 < q →
∀ {U : Finset α},
q < U.card →
(DGD26AdmissionsPredictability.paper_definition_substitutability C ↔
DGD26AdmissionsPredictability.paper_definition_d_instability 1 C ∧
¬DGD26AdmissionsPredictability.paper_definition_zero_instability C)
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_d\\_instability](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_acceptance](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_substitutability](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_zero\\_instability](#)

## Lean-elaborated claim roles

1. parameter: Type u\_1
2. parameter: DecidableEq α
3. parameter: ℕ
4. parameter: DGD26AdmissionsPredictability.PaperChoiceRule α
5. assumption: DGD26AdmissionsPredictability.paper\_choice\_function\_feasible C
6. assumption: DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance q C
7. assumption: 0 < q
8. parameter: Finset α
9. assumption: q < U.card
10. conclusion: DGD26AdmissionsPredictability.paper\_definition\_substitutability C ↔  
DGD26AdmissionsPredictability.paper\_definition\_d\_instability 1 C ∧  
¬DGD26AdmissionsPredictability.paper\_definition\_zero\_instability C

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_substitutability\_one\_instability\_equivalence\_statement

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Matches: the source Theorem 1's exactly-1-unstable iff substitutable clause is expressed by the 1-bound plus nonzero-instability conjunction under its explicit positive nontrivial-domain context.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 33

**Source locator:** cited publication, lines 280-289

## Verbatim source input

```
\begin{restatable}{theorem}{thminstability}
\label{thm:instability}
A  $q$ -acceptant choice function
\begin{enumerate}
\item cannot be  $0$ -unstable,
\item is exactly  $1$ -unstable if and only if it is substitutable,
\item can be tightly  $d$ -unstable for every  $1 \leq d \leq 2q$ .
\end{enumerate}
\end{restatable}%

[Next verbatim source excerpt]

\subsection{Proof of \Cref{thm:instability}} \label{sec:instability-proof}

Recall \Cref{thm:instability}:
\thminstability*

Putting things together, the first statement of \Cref{thm:instability} comes from \Cref{thm:independent-zero-unstable} and the second from
 $\hookrightarrow$  \Cref{thm:sub-unstable-equiv}. The third
has roots in \Cref{lem:unstable-calc}, which allows us to calculate instability.

Rigorously, we can generate a  $d$ -unstable  $\text{\textit{choice}}$  for any  $1 < d \leq 2q$ . Take any  $1$ -unstable,  $q$ -acceptant  $\text{\textit{choice}}$  induced by a total order
 $\hookrightarrow$   $\text{\textit{succ}}$  (i.e, is a ranking). Let  $n$  be  $\frac{d}{2}$ , rounded down. Then let  $X^* = \{x^*_1, \dots, x^*_n\}$  be the  $n$  \textit{minimal} elements of
 $\hookrightarrow$   $\text{\textit{succ}}$  (or any  $n$  elements that are not among the  $n$  maximal elements, if there exist any.) Then modify  $\text{\textit{choice}}$  such that if  $X^* \subseteq X$ 
 $\hookrightarrow$   $X$  for any input  $X$ ,  $X^* \subseteq \text{\textit{choice}}(X)$ . Then  $\text{\textit{choice}}$  is tightly  $2n-1$ -unstable. To make  $\text{\textit{choice}}$   $2n$ -unstable, let there be some
 $\hookrightarrow$   $x'$  such that if  $X^* \subseteq X$  but  $x' \in X$  as well, then  $\text{\textit{choice}}$  element of  $X^*$  is in  $\text{\textit{choice}}(X)$ . %

The intuition for the construction is that  $X^*$  forms a highly desirable team, even though each element is not individually desirable, nor are partial
 $\hookrightarrow$  subsets of  $X^*$ .
In the inconsistent case (even instability)  $x'$  represents a candidate whose mere presence negates the value of  $X^*$  as a team, even when not
 $\hookrightarrow$  accepted.
```

## Expanded PaperInterface specification

```
∀ (q d : ℕ),
  1 ≤ d →
    d ≤ 2 * q →
      ∃ (β : Type) (inst : DecidableEq β) (C : DGD26AdmissionsPredictability.PaperChoiceRule β),
        DGD26AdmissionsPredictability.paper_choice_function_feasible C ∧
        DGD26AdmissionsPredictability.paper_definition_q_acceptance q C ∧
        DGD26AdmissionsPredictability.paper_definition_tight_d_instability d C
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_acceptance](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_tight\\_d\\_instability](#)

## Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: ℕ
3. assumption: 1 ≤ d
4. assumption: d ≤ 2 * q
5. conclusion: ∃ (β : Type) (inst : DecidableEq β) (C : DGD26AdmissionsPredictability.PaperChoiceRule β),
  DGD26AdmissionsPredictability.paper_choice_function_feasible C ∧
  DGD26AdmissionsPredictability.paper_definition_q_acceptance q C ∧
  DGD26AdmissionsPredictability.paper_definition_tight_d_instability d C
```

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_theorem1\_tight\_all\_d\_statement

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Matches: for every  $d$  with  $1 \leq d \leq 2q$ , the target witnesses a feasible  $q$ -acceptant choice rule tightly  $d$ -unstable, exactly the source existence clause.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 34

**Source locator:** cited publication, lines 1097-1109

## Verbatim source input

```
\begin{restatable}[Linear Assignment Instability]{theorem}{LAPstable}
\label{thm:lap-unstable}
Linear assignment problems are 1-unstable.
\end{restatable}
```

\textit{Intuition}: The ranking-like structure of linear assignment slots maintains substitutability, because each slot maintains a preference ordering over elements that is independent of the cohort.

```
\begin{proof}
Recall that for  $q$ -acceptant functions, substitutability and 1-instability are equivalent by \Cref{thm:sub-unstable-equiv}.
```

The proof is by contradiction; assume  $\text{\textit{choice}}$  is not substitutable. As per \Cref{lem:unsubstitutable}, there exists  $\text{\textit{choice}}(X), \text{\textit{choice}}(X')$  where some  $x^* \in X$  is not in  $\text{\textit{choice}}(X)$  but  $x^* \in \text{\textit{choice}}(X')$ .  
By \Cref{lem:lap-ordering},  $\text{\textit{choice}}(X) = \{x_{s_1}, \dots, x_{s_q}\}$  tells us we have  $x_{s_i} \succ_{s_i} x^*$  for all  $i \in \{1, \dots, q\}$ . For  $x^*$  to be in  $\text{\textit{choice}}(X')$ ,  $x_{s_i}$  must also be in  $\text{\textit{choice}}(X')$ , meaning that  $\text{\textit{choice}}(X) \subseteq \text{\textit{choice}}(X')$ . However, as  $x^* \in \text{\textit{choice}}(X')$ ,  $|\text{\textit{choice}}(X')| > |\text{\textit{choice}}(X)|$ , which violates  $q$ -acceptance, leading to contradiction.  
\end{proof}

**Settled source reading** The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE\_CLARIFICATIONS.md).

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {σ : Type u_2} [inst_1 : DecidableEq σ] [inst_2 : Fintype σ]
{operationalWeight : α → σ → ℝ}
(hwell :
  ∀ (X : Finset α),
    ∃ (A : DGD26AdmissionsPredictability.LAP.Assignment α σ),
      DGD26AdmissionsPredictability.LAP.Assignment.Feasible X A ∧
      DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling X A ∧
      DGD26AdmissionsPredictability.LAP.Assignment.ObjectiveOptimal X operationalWeight A ∧
      ∀ (B : DGD26AdmissionsPredictability.LAP.Assignment α σ),
        DGD26AdmissionsPredictability.LAP.Assignment.Feasible X B →
        DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling X B →
        DGD26AdmissionsPredictability.LAP.Assignment.ObjectiveOptimal X operationalWeight B →
        B.chosenSet = A.chosenSet),
  (∀ (s : σ), DGD26AdmissionsPredictability.LAP.Assignment.SlotNoTies operationalWeight s) →
  DGD26AdmissionsPredictability.paper_definition_d_instability 1
  (DGD26AdmissionsPredictability.paperLAPChoiceRule operationalWeight hwell))
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.LAP.Assignment](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.Feasible](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.ObjectiveOptimal](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.SlotNoTies](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.chosenSet](#)
- [DGD26AdmissionsPredictability.paperLAPChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_d\\_instability](#)

## Lean-elaborated claim roles

1. parameter: Type u\_1
2. parameter: DecidableEq α
3. parameter: Type u\_2
4. parameter: DecidableEq σ
5. parameter: Fintype σ
6. parameter: α → σ → ℝ
7. assumption: ∀ (X : Finset α),  
 ∃ (A : DGD26AdmissionsPredictability.LAP.Assignment α σ),  
 DGD26AdmissionsPredictability.LAP.Assignment.Feasible X A ∧  
 DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling X A ∧  
 DGD26AdmissionsPredictability.LAP.Assignment.ObjectiveOptimal X operationalWeight A ∧  
 ∀ (B : DGD26AdmissionsPredictability.LAP.Assignment α σ),  
 DGD26AdmissionsPredictability.LAP.Assignment.Feasible X B →  
 DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling X B →  
 DGD26AdmissionsPredictability.LAP.Assignment.ObjectiveOptimal X operationalWeight B →  
 B.chosenSet = A.chosenSet
8. assumption: ∀ (s : σ), DGD26AdmissionsPredictability.LAP.Assignment.SlotNoTies operationalWeight s
9. parameter: Finset α

10. parameter:  $\alpha$   
11. assumption:  $x \notin X$   
12. conclusion: `AppliedModelingLib.FiniteChoice.choiceDistance`  
`(DGD26AdmissionsPredictability.paperLAPChoiceRule operationalWeight hwell) X (insert x X) ≤`  
1

### Lean proof endpoint (built separately)

`DGD26AdmissionsPredictability.PaperInterface.paper_lap_assignment_one_instability_statement`

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Independent outcome-blind review: under the current source-model tie-break convention, the 1-instability Spec uses a fixed generic operational tie-break to make the source LAP selection single-valued; it does not assume raw-score injectivity or raw-primary uniqueness.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 35

**Source locator:** cited publication, lines 1111-1129

## Verbatim source input

```
\begin{restatable}[Linear Assignment Variability]{theorem}{LAPvariable}
\label{thm:lap-variability}
The variability of a linear assignment problem is upper bounded by the number of distinct preference orderings induced by its slots.
\end{restatable}

\textit{Intuition}:
The proof follows from the observation that, if slot  $s$  ranks some other element  $x$  in the chosen set higher than the element  $x_s$  currently assigned
→ to  $s$ ,  $x$  cannot be in the variable set. Then, if  $k$  slots (say, slots  $s_1, \dots, s_k$ , with assigned elements  $x_{s_1}, \dots, x_{s_k}$ ) induce the same
→ preference ordering over elements, then only the minimal element in  $\{x_{s_1}, \dots, x_{s_k}\}$  according to that preference ordering can be in
→  $V_{\text{choice}(X)}$ .

\begin{proof}
We want to show that variability is upper bounded by the number of unique orderings. Recall that the variable set  $V_{\text{choice}(X)} \subseteq \text{choice}(X)$  is at
→ most cardinality  $q$ .

I claim that for any  $X$  and  $\text{choice}(X)$ ,  $x_{s_i}$  is not in  $V_{\text{choice}(X)}$  if  $x_{s_i} \succ x_{s_j}$  for any  $j \neq i$ .
To be precise, if  $x_{s_i} \in \text{choice}(X)$ , and  $x_{s_i} \succ x_{s_j}$  for some  $j \neq i$ , then  $x_{s_i} \in \text{choice}(X')$  for any  $X' = X \cup \{x\}$ .

The proof of this claim is by contradiction: assume there exists  $\text{choice}(X)$ ,  $\text{choice}(X')$  where some  $x_{s_i} \succ x_{s_j}$  but  $x_{s_i} \notin \text{choice}(X')$ .
→ Firstly, because of the 1-instability proven in \Cref{thm:lap-unstable},  $|\text{choice}(X) \setminus \text{choice}(X')| = 1$ , which means by
→ intuitive to see that this generalizes: if  $s_1, \dots, s_j$  all induce the same ordering  $\text{succ}_{s_j}$ , there is exactly one minimal element in  $x_{s_1}, \dots, x_{s_j}$ ,
→ because any finite subset of a totally ordered set is well-ordered.
→ assumption,  $|\text{choice}(X) \setminus \text{choice}(X')| = |\{x_{s_i}\}|$ , which further implies  $x_{s_j} \in \text{choice}(X')$ . However, \Cref{lem:lap-ordering} then
→ applies to  $x_{s_j}$ , where  $x_{s_i} \succ x_{s_j}$  implies  $x_{s_i} \in \text{choice}(X')$ , leading to contradiction.

Secondly, note that if  $s_i, s_j$  induce the same total ordering, then either  $x_{s_i} \succ x_{s_j}$  or  $x_{s_j} \succ x_{s_i}$  (and the same
→ for  $\text{succ}_{s_i}$ ) for any  $\text{choice}(X)$ , as a property of any total ordering. Then one of either  $x_{s_j}$  or  $x_{s_i}$  are not in  $V_{\text{choice}(X)}$ . It is
→ intuitive to see that this generalizes: if  $s_1, \dots, s_j$  all induce the same ordering  $\text{succ}_{s_j}$ , there is exactly one minimal element in  $x_{s_1}, \dots, x_{s_j}$ ,
→ because any finite subset of a totally ordered set is well-ordered.
Say that  $x_{s_j}$  is the minimal element, w.l.o.g.; then none of  $x_{s_1}, \dots, x_{s_{j-1}}$  are in  $V_{\text{choice}(X)}$ . More generally, let there be  $m$ 
→ unique orderings, which we notate  $\text{succ}_1, \dots, \text{succ}_m$ . Let  $k \in \{1, \dots, m\}$  and let there be  $c_k$  slots that induce  $\text{succ}_k$ . Then we have
→ defined sets of size  $c_k - 1$  elements that cannot be in  $V_{\text{choice}(X)}$ . Then  $|V_{\text{choice}(X)}| \leq m$ , and therefore the overall variability is at most  $m$ .
```

**Settled source reading** The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE\_CLARIFICATIONS.md).

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] {σ : Type u_2} [inst_2 : DecidableEq σ]
[inst_3 : Fintype σ] {operationalWeight : α → σ → ℝ}
(hwell :
  ∀ (X : Finset α),
  ∃ (A : DGD26AdmissionsPredictability.LAP.Assignment α σ),
  DGD26AdmissionsPredictability.LAP.Assignment.Feasible X A ∧
  DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling X A ∧
  DGD26AdmissionsPredictability.LAP.Assignment.ObjectiveOptimal X operationalWeight A ∧
  ∀ (B : DGD26AdmissionsPredictability.LAP.Assignment α σ),
  DGD26AdmissionsPredictability.LAP.Assignment.Feasible X B →
  DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling X B →
  DGD26AdmissionsPredictability.LAP.Assignment.ObjectiveOptimal X operationalWeight B →
  B.chosenSet = A.chosenSet),
(∀ (s : σ), DGD26AdmissionsPredictability.LAP.Assignment.SlotNoTies operationalWeight s) →
  ∀ (X : Finset α),
  (DGD26AdmissionsPredictability.paper_definition_borderline_set
    (DGD26AdmissionsPredictability.paperLAPChoiceRule operationalWeight hwell) X).card ≤
  (Finset.image (DGD26AdmissionsPredictability.LAP.Assignment.slotOrderClass operationalWeight) Finset.univ).card
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.LAP.Assignment](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.Feasible](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.ObjectiveOptimal](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.SlotNoTies](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.chosenSet](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.sameSlotOrderSetoid](#)
- [DGD26AdmissionsPredictability.LAP.Assignment.slotOrderClass](#)
- [DGD26AdmissionsPredictability.paperLAPChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_borderline\\_set](#)

## Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: DecidableEq α
3. parameter: Fintype α
4. parameter: Type u_2
5. parameter: DecidableEq σ
6. parameter: Fintype σ
7. parameter: α → σ → ℝ
8. assumption: ∀ (X : Finset α),
  ∃ (A : DGD26AdmissionsPredictability.LAP.Assignment α σ),
    DGD26AdmissionsPredictability.LAP.Assignment.Feasible X A ∧
    DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling X A ∧
    DGD26AdmissionsPredictability.LAP.Assignment.ObjectiveOptimal X operationalWeight A ∧
    ∀ (B : DGD26AdmissionsPredictability.LAP.Assignment α σ),
      DGD26AdmissionsPredictability.LAP.Assignment.Feasible X B →
      DGD26AdmissionsPredictability.LAP.Assignment.CapacityFilling X B →
      DGD26AdmissionsPredictability.LAP.Assignment.ObjectiveOptimal X operationalWeight B →
      B.chosenSet = A.chosenSet
9. assumption: ∀ (s : σ), DGD26AdmissionsPredictability.LAP.Assignment.SlotNoTies operationalWeight s
10. parameter: Finset α
11. conclusion: (DGD26AdmissionsPredictability.paper_definition_borderline_set
  (DGD26AdmissionsPredictability.paperLAPChoiceRule operationalWeight hwell) X).card ≤
  (Finset.image (DGD26AdmissionsPredictability.LAP.Assignment.slotOrderClass operationalWeight) Finset.univ).card
```

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_lap\_assignment\_slot\_order\_class\_variability\_of\_unique\_global\_optima\_statement

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Independent outcome-blind review: under the current source-model tie-break convention, the slot-order-class bound is the source result; its uniqueness is only for the refined selection, not raw-primary optima.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 36

**Source locator:** cited publication, lines 910-923

## Verbatim source input

```
\begin{restatable}[$q$-Representativeness and $1$-Variability are Equivalent]{theorem}{rankingvariability}
\label{thm:ranking-variability}
$\choice$ is $q$-representative if and only if it is $q$-acceptant and $1$-unstable with variability $1$.
\end{restatable}
\begin{proof}
We prove this in two parts: first, that $q$-representativeness implies $1$-variability, and then the converse.

\paragraph{Direction 1} %
Note that $q$-representativeness implies $q$-acceptance by definition.

Next, we prove that $q$-representativeness implies $1$-instability. Assume for contradiction that some $q$-representative $\choice$ is not substitutable
 $\hookrightarrow$  (and therefore not $1$-unstable). Then there exists (by \Cref{lem:unsubstitutable}) some $x^*$, $x'$, $X$ and $X' = X \cup \{x'\}$ where $x^* \in X$ and $x^* \in \choice(X')$ but $x^* \notin \choice(X)$. By $q$-acceptance, there must be at least one $x$ in $\choice(X)$ that is not in $\choice(X')$ (which must take the place of $x^*$). However, $\choice(X')$ implies that $x^* \succ x$, while $\choice(X)$ implies that $x \succ x^*$
 $\hookrightarrow$  --- violating asymmetry and leading us to contradiction.

Lastly, proving that $q$-representative $\choice$ leads to $m=1$ is relatively intuitive. For any $X$, $\choice(X)$ are the $q$ highest-ranked elements of $X$. Denote $X = \{x_1, x_2, \dots, x_k\}$ where $|X| = k$ and $x_1 \succ x_2 \succ \dots \succ x_k$. Then $\choice(X) = \{x_1, \dots, x_{q^*}\}$ where $q^* = \min\{q, k\}$.
Assume for contradiction that there is a $q$-representative $\choice$ with variability $m > 1$. Then there exists some $X$ where $|V_\choice(X)| > 1$;
 $\hookrightarrow$  say that $\{x^*_a, x^*_b\} \subseteq V_\choice(X)$. Then there exists some $x'_a$ where $\choice(X) \setminus \choice(X \cup x'_a) = \{x^*_a\}$
 $\hookrightarrow$  and similar $x'_b$ for $x^*_b$. The first implies $x^*_b$ is ranked above $x^*_a$, but the second implies the opposite. Then choices cannot be made according to a static ranking and $\choice$ is not $q$-representative, completing the proof by contradiction. %
```

**Settled source reading** The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE\_CLARIFICATIONS.md).

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] {q : ℕ}
{C : DGD26AdmissionsPredictability.PaperChoiceRule α},
DGD26AdmissionsPredictability.paper_choice_function_feasible C →
DGD26AdmissionsPredictability.paper_definition_q_representativeness q C →
0 < q →
q < Fintype.card α →
DGD26AdmissionsPredictability.paper_definition_q_acceptance q C ∧
DGD26AdmissionsPredictability.paper_definition_d_instability 1 C ∧
DGD26AdmissionsPredictability.paper_definition_variability_exactly 1 C
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_d\\_instability](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_acceptance](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_representativeness](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_variability\\_exactly](#)

## Lean-elaborated claim roles

1. parameter: Type u\_1
2. parameter: DecidableEq α
3. parameter: Fintype α
4. parameter: ℕ
5. parameter: DGD26AdmissionsPredictability.PaperChoiceRule α
6. assumption: DGD26AdmissionsPredictability.paper\_choice\_function\_feasible C
7. assumption: DGD26AdmissionsPredictability.paper\_definition\_q\_representativeness q C
8. assumption: 0 < q
9. assumption: q < Fintype.card α
10. conclusion: DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance q C ∧  
DGD26AdmissionsPredictability.paper\_definition\_d\_instability 1 C ∧  
DGD26AdmissionsPredictability.paper\_definition\_variability\_exactly 1 C

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_q\_representative\_forward\_exact\_statement



**Recorded source-to-Spec screening Verdict:** matches

**Reason:** The source theorem's forward direction says q-representativeness entails q-acceptance, one-instability, and variability one. The target has exactly those conclusions from q-representativeness and feasibility on the queue's source-model nondegenerate domain.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

|  |
|--|
|  |
|--|

# Review row 37

**Source locator:** cited publication, lines 910-950

## Verbatim source input

```
\begin{restatable}[$q$-Representativeness and $1$-Variability are Equivalent]{theorem}{rankingvariability}
\label{thm:ranking-variability}
$\choice$ is $q$-representative if and only if it is $q$-acceptant and $1$-unstable with variability $1$.
\end{restatable}
\begin{proof}
We prove this in two parts: first, that $q$-representativeness implies $1$-variability, and then the converse.

\paragraph{Direction 1} %
Note that $q$-representativeness implies $q$-acceptance by definition.

Next, we prove that $q$-representativeness implies $1$-instability. Assume for contradiction that some $q$-representative $\choice$ is not substitutable
→ (and therefore not $1$-unstable). Then there exists (by \Cref{lem:unsubstitutable}) some $x^*$, $x'$, $X$ and $X' = X \cup \{x'\}$ where $x^* \in X$ and $x^* \in \choice(X')$ but $x^* \notin \choice(X)$. By $q$-acceptance, there must be at least one $x$ in $\choice(X)$ that is not in $\choice(X')$ (which must take the place of $x^*$). However, $\choice(X')$ implies that $x^* \succ x$, while $\choice(X)$ implies that $x \succ x^*$ --- violating asymmetry and leading us to contradiction.

Lastly, proving that $q$-representative $\choice$ leads to $m=1$ is relatively intuitive. For any $X$, $\choice(X)$ are the $q$ highest-ranked elements of $X$. Denote $X = \{x_1, x_2, \dots, x_k\}$ where $|X| = k$ and $x_1 \succ x_2 \succ \dots \succ x_k$. Then $\choice(X) = \{x_1, \dots, x_{q^*}\}$ where $q^* = \min\{q, k\}$.
Assume for contradiction that there is a $q$-representative $\choice$ with variability $m > 1$. Then there exists some $X$ where $|V_\choice(X)| > 1$; say that $\{x^*_a, x^*_b\} \subseteq V_\choice(X)$. Then there exists some $x'_a$ where $\choice(X) \setminus \choice(X \cup x'_a) = \{x^*_a\}$ and similar $x'_b$ for $x^*_b$. The first implies $x^*_b$ is ranked above $x^*_a$, but the second implies the opposite. Then choices cannot be made according to a static ranking and $\choice$ is not $q$-representative, completing the proof by contradiction. %

\paragraph{Direction 2}
Assume $q$-acceptance, variability 1, and instability 1. Now we prove $\choice$ is $q$-representative. To be precise, I take this to mean that $\choice$ is indistinguishable from some $q$-representative $\choice$; $\choice$ could be represented by some total ordering over elements of $X$. We show this strict total ordering constructively.

Define $\succ$ such that, if $x_a \in \choice(X)$ but $x_b \notin \choice(X)$ for any $x_a, x_b \in X$, $x_a \succ x_b$. Note that this implies that the existence of some set $A$ where $x_a \in \choice(A)$ and $A \setminus \choice(A) = \{x_b\}$ by consistency (e.g. $A = \choice(X) \cup \{x_b\}$). This also implies $A'$ where $x_a, x_b \in \choice(A')$, $V_\choice(A') = \{x_b\}$, where $A' = A \setminus \{x\}$ for any $x \neq x_a, x_b$.

Then, we need to prove this relation is asymmetric, transitive, and completeness (or rather, is indistinguishable from a complete relation when examining the induced choice function).

\paragraph{Asymmetry}
First, we show asymmetry. Assume for contradiction that $a \succ b$ and $b \succ a$. Then there exists $X_a$ where $a, b \in X_a$, $a \in \choice(X_a)$ and $b \notin \choice(X_a)$, and a similar $X_b$ where $a \notin \choice(X_b)$ and so on.

Let $X_a = \choice(X_a) \cup \{b\}$ (we can construct such out of any $X_a$, by consistency), where $A \setminus \choice(X_a) = \{b\}$, and denote $\choice(X_a) = \{x^*_a_1, \dots, x^*_a_{q-1}\}$. Construct $X_b$ similarly. Further, let $X'_a = X_a \setminus \{x^*_a_1\}$, and note that $V_\choice(X'_a) = \{b\}$.

Then from here, we add the elements of $X_b$ to $X'_a$, constructing $X_1, X_2$, and so forth.

Let $X_0 = X_a$ and sequentially construct a series of $X_t$. Add $x^*_b_t$ to $X_{t-1}$. If $x_b \notin \choice(X_{t-1} \cup \{x^*_b_t\})$, then let $X_t = X_{t-1} \cup \{x^*_b_t\}$, and $V_\choice(X_t) = \{b\}$ by \Cref{lem:consistent-remove}. If $x^*_b_t \in \choice(X_{t-1} \cup \{x^*_b_t\})$, then $b \notin \choice(X_{t-1} \cup \{x^*_b_t\})$ and $A \setminus \choice(X_{t-1} \cup \{x^*_b_t\}) = \{b\}$ (by \Cref{lem:a-v-equal}). Then let $X_t = \{x^*_b_t\} \cup X'_a \setminus \{x^*_a_i\}$ where $x^*_a_{i-1} \notin \choice(X_{t-1})$ but $x^*_a_i \in \choice(X_{t-1})$, and $V_\choice(X_t) = \{b\}$ (again, by \Cref{lem:a-v-equal}). Note that $a \in \choice(X_t)$, $b \in X_t$, and specifically $b \in V_\choice(X_t)$, for all $t \in \{0, \dots, q-2\}$.

Then $X_{q-2}$ will contain $x^*_b_1, \dots, x^*_b_{q-2}$. Let $X^* = X_{q-2} \cup \{x^*_b_{q-1}\}$. Note that adding $x^*_b_{q-1}$ to $X_{q-2}$ makes $X^* \supseteq X_b$. Then, if $a \in \choice(X^*)$, substitutability is violated, as $a \notin \choice(X_b)$. If $x^*_b_{q-1} \notin \choice(X^*)$, then $\choice(X_{q-2}) = \choice(X^*)$ by consistency, and $a \in \choice(X_{q-2})$. If instead $x^*_b_{q-1} \in \choice(X^*)$, recall that $V_\choice(X_{q-2}) = \{b\}$, so $a \notin \choice(X_{q-2})$, and $a$ remains in $\choice(X^*)$. Therefore, either possibility leads to contradiction, proving that $\succ$ is asymmetric.

\paragraph{Transitivity}
Say that $a \succ b$ and $b \succ c$ and we want to prove $a \succ c$. If $a \succ b$, there exists some $X$ where $a, b \in X$, $a \in \choice(X)$, and $b \notin \choice(X)$. Let $X$ be $\choice(X) \cup \{b\}$, where $a \in \choice(X)$, $b \notin \choice(X)$ by consistency. Then consider $X \cup \{c\}$. By asymmetry, $c \notin \succ b$, so $c \notin \choice(X \cup \{c\})$. By consistency, then, $\choice(X \cup \{c\}) = \choice(X)$, so $a \in \choice(X \cup \{c\})$ and $a \succ c$, completing the proof.

\paragraph{(Almost)-Completeness}
See that this relation must exist for all but $q$ elements --- and any arbitrary linear extension thereof generates the same choice function behavior.

For any $a, b$, let there be some $X_0$ where $a, b \notin \choice(X_0)$. Then remove elements of $\choice(X_t)$ in a sequence $X_0, \dots, X_t$ until $A \setminus \choice(X_t) = \{a\}$ or $\{b\}$. This must happen for some value of $t$ because when $|X_t| = q+1$, at least one of $a$ or $b$ are in $\choice(X_t)$, and due to $1$-instability, both $a$ and $b$ cannot be added to $\choice(X_t)$ at once. The only exception is if there is no such $X_0$ where $a, b \notin \choice(X_0)$, which can only be true if $a$ is such that $a \in \choice(X)$ for all $X$ (and similarly for $b$). (If $a \notin \choice(X)$ for some $X$, then substitutability implies $a \notin \choice(X \cup \{b\})$). However, there can only be $q$ such elements (by $q$-acceptance). Note that any linear extension of this partial order will produce the same choice function behavior; that is, any constructed total ordering that otherwise aligns with $\succ$ will produce the same choice behavior, and thus, $\choice$ can be represented by any such strict total ordering.
\end{proof}
```

**Settled source reading** The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE\_CLARIFICATIONS.md).

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] {q : ℕ}
{C : DGD26AdmissionsPredictability.PaperChoiceRule α},
```

DGD26AdmissionsPredictability.paper\_choice\_function\_feasible C →  
DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance q C →  
DGD26AdmissionsPredictability.paper\_definition\_d\_instability 1 C →  
DGD26AdmissionsPredictability.paper\_definition\_variability\_exactly 1 C →  
0 < q → q < Fintype.card α → DGD26AdmissionsPredictability.paper\_definition\_q\_representativeness q C

### Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_d\\_instability](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_acceptance](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_representativeness](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_variability\\_exactly](#)

### Lean-elaborated claim roles

1. parameter: Type u\_1  
2. parameter: DecidableEq α  
3. parameter: Fintype α  
4. parameter: ℕ  
5. parameter: DGD26AdmissionsPredictability.PaperChoiceRule α  
6. assumption: DGD26AdmissionsPredictability.paper\_choice\_function\_feasible C  
7. assumption: DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance q C  
8. assumption: DGD26AdmissionsPredictability.paper\_definition\_d\_instability 1 C  
9. assumption: DGD26AdmissionsPredictability.paper\_definition\_variability\_exactly 1 C  
10. assumption: 0 < q  
11. assumption: q < Fintype.card α  
12. conclusion: ∃ (r : α → α → Prop),  
AppliedModelingLib.FiniteChoice.StrictTotalOrder r ∧  
AppliedModelingLib.FiniteChoice.Feasible C ∧  
AppliedModelingLib.FiniteChoice.QAcceptant q C ∧ ∀ {X : Finset α} {x y : α}, x ∈ C X → y ∈ X → y ∉ C X → r x y

### Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_q\_representative\_converse\_exact\_statement

### Recorded source-to-Spec screening Verdict: matches

**Reason:** The source theorem's converse takes q-acceptance, variability one, and instability one to q-representativeness. The target has those premises and conclusion, together with feasibility and the positive nontrivial-capacity source-model domain under which the displacement construction is meaningful.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

### Reviewer annotation

# Review row 38

**Source locator:** cited publication, lines 962-996

## Verbatim source input

```
\paragraph{Sequentially Composed Substitutable Functions Are Substitutable}
Similarly, we show sequential compositions of substitutable functions are also substitutable.
\begin{restatable}[Composition Preserves Substitutability]{theorem}{seqsub}
\label{thm:composition-substitutable}
Sequential composition of substitutable functions yields a substitutable function.
\end{restatable}

\textit{Intuition}:
``Appending'' a substitutable choice function  $\text{\textit{\textit{choice}}}_2$  before another substitutable function  $\text{\textit{\textit{choice}}}_1$  affects the inputs to  $\text{\textit{\textit{choice}}}_1$  in a
 $\hookrightarrow$  structured way.
\begin{proof}
  Proof by induction.
  In the base case, for  $n = 1$ ,  $\text{\textit{\textit{choice}}}(X) = \text{\textit{\textit{choice}}}_1(X)$ , and as  $\text{\textit{\textit{choice}}}_1$  is given to be substitutable,  $\text{\textit{\textit{choice}}}$  is as well.

  For the inductive step, show that  $\text{\textit{\textit{choice}}}^{n+1}$  is substitutable when comprised of  $\text{\textit{\textit{choice}}}_{n+1}$ ,  $\text{\textit{\textit{choice}}}_n$ , ...,  $\text{\textit{\textit{choice}}}_1$ , assuming that
   $\hookrightarrow$   $\text{\textit{\textit{choice}}}^n$  comprised of  $\text{\textit{\textit{choice}}}_n, \dots, \text{\textit{\textit{choice}}}_1$  is substitutable.

  The definition of substitutability is that  $X \subseteq X'$  implies  $\text{\textit{\textit{choice}}}(X') \cap X \subseteq \text{\textit{\textit{choice}}}(X)$ . So we need to show  $\text{\textit{\textit{choice}}}^{n+1}(X') \cap X \subseteq \text{\textit{\textit{choice}}}^{n+1}(X)$ .

  First, note that  $\text{\textit{\textit{choice}}}^{n+1}(X) = \text{\textit{\textit{choice}}}_{n+1}(X) \cup \text{\textit{\textit{choice}}}^n(X \setminus \text{\textit{\textit{choice}}}_{n+1}(X))$ .
  Let  $S = X \setminus \text{\textit{\textit{choice}}}_{n+1}(X)$  and  $S' = X' \setminus \text{\textit{\textit{choice}}}_{n+1}(X')$ . As  $S \subseteq S'$ , we only need to show that no elements of  $S$ 
   $\hookrightarrow$  are elements of  $\text{\textit{\textit{choice}}}_{n+1}(X')$ . As  $\text{\textit{\textit{choice}}}_{n+1}$  is substitutable,  $\text{\textit{\textit{choice}}}_{n+1}(X') \cap X = \text{\textit{\textit{choice}}}_{n+1}(X)$ . So, any element of
   $\hookrightarrow$   $S$  that is in  $\text{\textit{\textit{choice}}}_{n+1}(X')$  must be in  $\text{\textit{\textit{choice}}}_{n+1}(X)$ , and therefore cannot be in  $S = X \setminus \text{\textit{\textit{choice}}}_{n+1}(X)$ . Therefore,  $S \subseteq S'$ .
   $\hookrightarrow$   $\text{\textit{\textit{choice}}}^n S \subseteq S'$ .

  Now, we can apply the induction hypothesis, so we assume  $\text{\textit{\textit{choice}}}^n$  is substitutable to get

  \begin{align*}
    & \text{\textit{\textit{choice}}}^n(S') \cap S \subseteq \text{\textit{\textit{choice}}}^n(S) \quad \& \text{\textit{\textit{induction hypothesis}}} \quad \& \\
    & \text{\textit{\textit{choice}}}^n(S') \cap \left( X \setminus \text{\textit{\textit{choice}}}_{n+1}(X) \right) \subseteq \text{\textit{\textit{choice}}}^n(S) \quad \& \text{\textit{\textit{substitute out } S}} \quad \& \left( \text{\textit{\textit{choice}}}^n(S') \cap \right. \\
    & \quad \left. X \right) \setminus \text{\textit{\textit{choice}}}_{n+1}(X) \subseteq \text{\textit{\textit{choice}}}^n(S) \quad \& \text{\textit{\textit{intersection commutes}}} \quad \& \\
    & \text{\textit{\textit{choice}}}_{n+1}(X) \cup \left( \left( \text{\textit{\textit{choice}}}^n(S') \cap X \right) \setminus \text{\textit{\textit{choice}}}_{n+1}(X) \right) \subseteq \text{\textit{\textit{choice}}}^n(S) \cup \text{\textit{\textit{choice}}}_{n+1}(X) \quad \& \\
    & \quad \hookrightarrow \text{\textit{\textit{choice}}}_{n+1}(X) \cup \text{\textit{\textit{choice}}}^n(S) \quad \& \text{\textit{\textit{add } \text{\textit{\textit{choice}}}_{n+1}(X)}} \quad \& \text{\textit{\textit{Note that } \text{\textit{\textit{choice}}}_{n+1}(X) \subseteq \text{\textit{\textit{choice}}}^n(S')}} \\
    & \quad \hookrightarrow \text{\textit{\textit{choice}}}_{n+1}(X) \cup \text{\textit{\textit{choice}}}^n(S), \text{ so the set difference can be simplified out: } \\
    & \text{\textit{\textit{choice}}}_{n+1}(X) \cup \left( \text{\textit{\textit{choice}}}^n(S') \cap X \right) \subseteq \text{\textit{\textit{choice}}}_{n+1}(X) \cup \text{\textit{\textit{choice}}}^n(S) \quad \& \text{\textit{\textit{equivalent}}} \quad \& \text{\textit{\textit{Note that}}} \\
    & \quad \hookrightarrow \text{ for any sets } A, B, C, (A \cup B) \cap C = (A \cap C) \cup (B \cap C), \text{ so: } \\
    & \quad \left( \text{\textit{\textit{choice}}}_{n+1}(X) \cup \text{\textit{\textit{choice}}}^n(S') \right) \cap X \subseteq \text{\textit{\textit{choice}}}_{n+1}(X) \cup \text{\textit{\textit{choice}}}^n(S) \quad \& \text{\textit{\textit{}}} \quad \& \\
    & \quad \left( \text{\textit{\textit{choice}}}_{n+1}(X) \cup \text{\textit{\textit{choice}}}^n(X \setminus \text{\textit{\textit{choice}}}_{n+1}(X')) \right) \cap X \subseteq \text{\textit{\textit{choice}}}_{n+1}(X) \cup \text{\textit{\textit{choice}}}^n(X \setminus \text{\textit{\textit{choice}}}_{n+1}(X)) \\
    & \quad \hookrightarrow \text{\textit{\textit{choice}}}_{n+1}(X) \cup \text{\textit{\textit{choice}}}^n(X \setminus \text{\textit{\textit{choice}}}_{n+1}(X)) \quad \& \text{\textit{\textit{substitute out } S, S'}} \quad \& \\
    & \text{\textit{\textit{choice}}}^{n+1}(X') \cap X \subseteq \text{\textit{\textit{choice}}}^{n+1}(X) \quad \& \text{\textit{\textit{definition of } \text{\textit{\textit{choice}}}^{n+1}}} \\
  \end{align*}

  Which is the definition of substitutability, completing the proof.
\end{proof}
```

## Expanded PaperInterface specification

```

 $\forall \{ \alpha : \text{Type } u_1 \} \{ \text{inst} : \text{DecidableEq } \alpha \} \{ \text{Cs} : \text{List } (\text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha) \},$ 
 $(\forall C \in \text{Cs}, \text{DGD26AdmissionsPredictability.paper\_choice\_function\_feasible } C) \rightarrow$ 
 $(\forall C \in \text{Cs}, \text{DGD26AdmissionsPredictability.paper\_definition\_substitutability } C) \rightarrow$ 
 $\text{DGD26AdmissionsPredictability.paper\_definition\_sequential\_composition } \text{Cs}$ 
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_sequential\\_composition](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_substitutability](#)

## Lean-elaborated claim roles

1. parameter:  $\text{Type } u_1$
2. parameter:  $\text{DecidableEq } \alpha$
3. parameter:  $\text{List } (\text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha)$
4. assumption:  $\forall C \in \text{Cs}, \text{DGD26AdmissionsPredictability.paper\_choice\_function\_feasible } C$
5. assumption:  $\forall C \in \text{Cs}, \text{DGD26AdmissionsPredictability.paper\_definition\_substitutability } C$
6. parameter:  $\text{Finset } \alpha$
7. parameter:  $\text{Finset } \alpha$
8. assumption:  $X_1 \subseteq X_2$
9. conclusion:  $X_1 \cap \text{DGD26AdmissionsPredictability.paper\_definition\_sequential\_composition } \text{Cs } X_2 \subseteq \text{DGD26AdmissionsPredictability.paper\_definition\_sequential\_composition } \text{Cs } X_1$

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_sequential\_composition\_substitutable\_statement

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Matches: the target states that the queue-context-bound residual-pool composition of feasible substitutable components remains substitutable, exactly the source composition theorem.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 39

**Source locator:** cited publication, lines 715-733

## Verbatim source input

```
\begin{restatable}[Substitutability and 1-Instability are Equivalent Under Capacity Constraints]{theorem}{subequiv}
\label{thm:sub-unstable-equiv}
Any  $q$ -acceptant choice function is  $1$ -unstable if and only if it is substitutable.
\end{restatable}

\begin{proof}
\textbf{Informal}
Intuitively,  $q$ -acceptance means that every acceptance decision that becomes a rejection must also be paired with a new acceptance to keep the size of
→ the accepted set the same. Substitutability implies that adding elements cannot cause new acceptances from rejections, so the only possible newly
→ accepted element is the newly added one.

Recall that the first term of  $r_{\text{choice}}$  is always zero only for substitutable  $\text{choice}$ . So, we only need to show  $|\text{choice}(X) \setminus \text{choice}(X')| \leq 1$ 
→ if and only if  $\text{choice}$  is substitutable. %
First, we show that  $\text{choice}$  being substitutable is \textit{sufficient} for  $1$ -instability. If  $\text{choice}$  is substitutable, then all elements in  $\text{choice}(X')$ 
→ except for  $x'$  must be in  $\text{choice}(X)$ . By  $q$ -acceptance,  $|\text{choice}(X')| = q$  (the  $|X| < q$  case is trivial), so  $\text{choice}(X)$  can have
→ at most one element not in  $\text{choice}(X')$ , making  $\text{choice}$   $1$ -unstable.

Second, we show that substitutability is \textit{necessary} for  $1$ -instability. If  $\text{choice}$  is not substitutable, there is some  $X \subseteq X'$  and  $x^* \in X$ 
→  $X$  such that  $x^* \notin \text{choice}(X)$  but  $x^* \in \text{choice}(X')$  (by \Cref{lem:unsubstitutable}). This creates one change. By  $q$ -acceptance, this
→ forces another candidate out of  $\text{choice}(X')$ , creating at least a second change, violating  $1$ -instability.

Together, these show that under  $q$ -acceptance, substitutability is necessary and sufficient for  $1$ -instability, completing the proof.
\end{proof}

\begin{restatable}{theorem}{BoundedDistance}
```

**Settled source reading** The result-specific conditions and corrections are stated in the [clarification memo](docs/SOURCE\_CLARIFICATIONS.md).

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {q : ℕ} {C : DGD26AdmissionsPredictability.PaperChoiceRule α},
DGD26AdmissionsPredictability.paper_choice_function_feasible C →
DGD26AdmissionsPredictability.paper_definition_q_acceptance q C →
0 < q →
  ∀ {U : Finset α},
  q < U.card →
    (DGD26AdmissionsPredictability.paper_definition_d_instability 1 C ∧
     →DGD26AdmissionsPredictability.paper_definition_zero_instability C ↔
     DGD26AdmissionsPredictability.paper_definition_substitutability C)
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_d\\_instability](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_acceptance](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_substitutability](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_zero\\_instability](#)

## Lean-elaborated claim roles

1. parameter: Type u\_1
2. parameter: DecidableEq α
3. parameter: ℕ
4. parameter: DGD26AdmissionsPredictability.PaperChoiceRule α
5. assumption: DGD26AdmissionsPredictability.paper\_choice\_function\_feasible C
6. assumption: DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance q C
7. assumption: 0 < q
8. parameter: Finset α
9. assumption: q < U.card
10. conclusion: DGD26AdmissionsPredictability.paper\_definition\_d\_instability 1 C ∧  
→DGD26AdmissionsPredictability.paper\_definition\_zero\_instability C ↔  
DGD26AdmissionsPredictability.paper\_definition\_substitutability C

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_qacceptant\_substitutable\_iff\_one\_statement

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Matches: with the queue's explicit positive nontrivial-domain context, the source's 1-instability/substitutability equivalence has the stated exactness conjunction, whose nonzero component is supplied by the source theorem's separate no-zero clause.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 40

**Source locator:** cited publication, lines 733-745

## Verbatim source input

```
\begin{restatable}{theorem}{BoundedDistance}
\label{thm:sub-accept-consistent}
Any  $q$ -acceptant, substitutable  $\text{choice}$  is consistent.
\end{restatable}

\begin{proof}
Consider  $X_1, X_2$  where  $\text{choice}(X_2) \subseteq X_1 \subseteq X_2$  for substitutable,  $q$ -acceptant  $\text{choice}$ . Substitutability guarantees that if  $x \in \text{choice}(X_2)$  and  $x \in X_1$ ,  $x \in \text{choice}(X_1)$ . By construction, this applies to all  $x \in \text{choice}(X_2)$ , because  $\text{choice}(X_2) \subseteq X_1$ ; therefore,  $\text{choice}(X_2) \subseteq \text{choice}(X_1)$ . As  $|\text{choice}(X_2)| = |\text{choice}(X_1)|$ , and  $\text{choice}(X_1), \text{choice}(X_2)$  are finite sets, this means that  $\text{choice}(X_2) = \text{choice}(X_1)$ , which is the definition of consistency.
\end{proof}

\subsection{\textit{d}-Instability} \label{sec:d-unstable-appendix}

\subsubsection{Proof of \Cref{lem:unstable-calc}} \label{sec:calc-proof}
```

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {q : ℕ} {C : DGD26AdmissionsPredictability.PaperChoiceRule α},
  DGD26AdmissionsPredictability.paper_choice_function_feasible C →
  DGD26AdmissionsPredictability.paper_definition_q_acceptance q C →
  DGD26AdmissionsPredictability.paper_definition_substitutability C →
  DGD26AdmissionsPredictability.paper_definition_consistency C
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_consistency](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_acceptance](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_substitutability](#)

## Lean-elaborated claim roles

1. parameter: Type u\_1
2. parameter: DecidableEq α
3. parameter: ℕ
4. parameter: DGD26AdmissionsPredictability.PaperChoiceRule α
5. assumption: DGD26AdmissionsPredictability.paper\_choice\_function\_feasible C
6. assumption: DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance q C
7. assumption: DGD26AdmissionsPredictability.paper\_definition\_substitutability C
8. parameter: Finset α
9. parameter: Finset α
10. assumption: C X<sub>2</sub> ⊆ X<sub>1</sub>
11. assumption: X<sub>1</sub> ⊆ X<sub>2</sub>
12. conclusion: C X<sub>2</sub> = C X<sub>1</sub>

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_q\_acceptant\_substitutable\_consistent\_statement

**Recorded source-to-Spec screening Verdict:** matches

**Reason:** Matches: the target is exactly the source theorem that a feasible  $q$ -acceptant substitutable choice rule is consistent.

**Reviewer check:** ☐ Matches source input

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**



# Review row 41

**Source locator:** cited publication, lines 315-322

## Verbatim source input

```
\begin{restatable}{theorem}{thmvariability}
\label{thm:variability}
Consider a  $q$ -acceptant, 1-unstable choice function  $\mathcal{C}$  which
can be represented as the sequential composition of  $n$  choice functions, each characterized by a total order.
Then  $\mathcal{C}$  has a variability  $m$ , where  $1 \leq m \leq n$ . Furthermore,  $\mathcal{C}$  has variability  $1$  if and only if it can be characterized by a
 $\hookrightarrow$  single total order,  $n=1$ .
\end{restatable}
In the appendix, we discuss when this upper bound is tight. We also extend this result to choice functions which are not explicitly defined as a fixed
 $\hookrightarrow$  sequential composition, such as linear assignment problems.
\Cref{thm:lap-variability} addresses the context-dependent sequential queues which result from such optimization based admissions.
```

[Next verbatim source excerpt]

```
\subsection{Proof of \Cref{thm:variability}} \label{sec:variability-proof}
```

```
Recall \Cref{thm:variability}:
\thmvariability*
```

We see that the second claim comes exactly from \Cref{thm:ranking-variability}, and that combining it with \Cref{thm:additive-variability} directly gives  
 $\hookrightarrow$  us the first claim.

**Formalized review target** The archival source above is not asserted equivalent to this formalized target.

For a sequential composition of total-order  $q$ -representative queues with positive total capacity on a universe larger than that capacity, there exists  $m$  with  
 $\hookrightarrow 1 \leq m \leq n$  such that the composition has exact variability  $m$ .

**Archival source anchor:** cited publication, lines 315-322

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] {qs : List ℕ}
{Cs : List (DGD26AdmissionsPredictability.PaperChoiceRule α)},
List.Forall₂
  (fun (q : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
    DGD26AdmissionsPredictability.paper_choice_function_feasible C ∧
    DGD26AdmissionsPredictability.paper_definition_q_representativeness q C)
  qs Cs →
0 < qs.sum →
qs.sum < Fintype.card α →
  ∃ (m : ℕ),
    1 ≤ m ∧
    m ≤ Cs.length ∧
    DGD26AdmissionsPredictability.paper_definition_variability_exactly m
    (DGD26AdmissionsPredictability.paper_definition_sequential_composition Cs)
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_representativeness](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_sequential\\_composition](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_variability\\_exactly](#)

## Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: DecidableEq α
3. parameter: Fintype α
4. parameter: List ℕ
5. parameter: List (DGD26AdmissionsPredictability.PaperChoiceRule α)
6. assumption: List.Forall₂
  (fun (q : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α) =>
    DGD26AdmissionsPredictability.paper_choice_function_feasible C ∧
    DGD26AdmissionsPredictability.paper_definition_q_representativeness q C)
  qs Cs
7. assumption: 0 < qs.sum
8. assumption: qs.sum < Fintype.card α
9. conclusion: ∃ (m : ℕ),
  1 ≤ m ∧
  m ≤ Cs.length ∧
  DGD26AdmissionsPredictability.paper_definition_variability_exactly m
  (DGD26AdmissionsPredictability.paper_definition_sequential_composition Cs)
```

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_sequential\_q\_representative\_variability\_range\_statement

**Recorded source-to-Spec screening Verdict:** matches\_approved\_corrected\_target

**Reason:** The archival theorem gives the range  $1 \leq m \leq n$  for sequential total-order queues. The target is the queue's approved corrected formulation: feasible q-representative queues, positive total capacity below the finite universe, and exact variability m in the same range. It preserves the source conclusion without claiming literal archival equivalence.

**Reviewer check:** ☐ Matches formalized target

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**

# Review row 42

**Source locator:** cited publication, lines 315-323

## Verbatim source input

```
\begin{restatable}{theorem}{thmvariability}
\label{thm:variability}
Consider a  $q$ -acceptant, 1-unstable choice function  $\mathcal{C}$  which
can be represented as the sequential composition of  $n$  choice functions, each characterized by a total order.
Then  $\mathcal{C}$  has a variability  $m$ , where  $1 \leq m \leq n$ . Furthermore,  $\mathcal{C}$  has variability  $1$  if and only if it can be characterized by a
 $\hookrightarrow$  single total order,  $n=1$ .
\end{restatable}
In the appendix, we discuss when this upper bound is tight. We also extend this result to choice functions which are not explicitly defined as a fixed
 $\hookrightarrow$  sequential composition, such as linear assignment problems.
\Cref{thm:lap-variability} addresses the context-dependent sequential queues which result from such optimization based admissions.
Importantly, Theorem~\ref{thm:variability} means that a choice function can be faithfully represented by a machine learning model if and \emph{only if}
 $\hookrightarrow$  it is characterized by a total ordering over  $\mathcal{X}$ .
```

[Next verbatim source excerpt]

```
\subsection{Proof of \Cref{thm:variability}} \label{sec:variability-proof}
```

Recall \Cref{thm:variability}:  
\thmvariability\*

We see that the second claim comes exactly from \Cref{thm:ranking-variability}, and that combining it with \Cref{thm:additive-variability} directly gives  
 $\hookrightarrow$  us the first claim.

**Formalized review target** The archival source above is not asserted equivalent to this formalized target.

For a feasible  $q$ -acceptant 1-unstable choice rule on a positive-capacity universe larger than  $q$ , exact variability one holds if and only if the rule is  
 $\hookrightarrow$   $q$ -representative, equivalently characterized by one strict total order.

**Archival source anchor:** cited publication, lines 315-323

## Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] {q : ℕ}
{C : DGD26AdmissionsPredictability.PaperChoiceRule α},
DGD26AdmissionsPredictability.paper_choice_function_feasible C  $\hookrightarrow$ 
DGD26AdmissionsPredictability.paper_definition_q_acceptance q C  $\hookrightarrow$ 
DGD26AdmissionsPredictability.paper_definition_d_instability 1 C  $\hookrightarrow$ 
0 < q  $\hookrightarrow$ 
q < Fintype.card α  $\hookrightarrow$ 
(DGD26AdmissionsPredictability.paper_definition_variability_exactly 1 C  $\leftrightarrow$ 
DGD26AdmissionsPredictability.paper_definition_q_representativeness q C)
```

## Retained governing declarations

- [DGD26AdmissionsPredictability.PaperChoiceRule](#)
- [DGD26AdmissionsPredictability.paper\\_choice\\_function\\_feasible](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_d\\_instability](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_acceptance](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_q\\_representativeness](#)
- [DGD26AdmissionsPredictability.paper\\_definition\\_variability\\_exactly](#)

## Lean-elaborated claim roles

1. parameter: Type u\_1
2. parameter: DecidableEq α
3. parameter: Fintype α
4. parameter: ℕ
5. parameter: DGD26AdmissionsPredictability.PaperChoiceRule α
6. assumption: DGD26AdmissionsPredictability.paper\_choice\_function\_feasible C
7. assumption: DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance q C
8. assumption: DGD26AdmissionsPredictability.paper\_definition\_d\_instability 1 C
9. assumption: 0 < q
10. assumption: q < Fintype.card α
11. conclusion: DGD26AdmissionsPredictability.paper\_definition\_variability\_exactly 1 C  $\leftrightarrow$  DGD26AdmissionsPredictability.paper\_definition\_q\_representativeness q C

## Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper\_variability\_one\_iff\_single\_order\_statement

**Recorded source-to-Spec screening Verdict:** matches\_approved\_corrected\_target

**Reason:** The archival  $m=1$  clause characterizes a rule by one total order. The target is the approved corrected version: under feasible  $q$ -acceptance and one-instability on the positive nontrivial domain, exact variability one holds exactly when the rule is  $q$ -representative, equivalently one strict total order.

**Reviewer check:** ☐ Matches formalized target

*If left unchecked, explain the discrepancy or uncertainty below.*

**Reviewer annotation**